深度学习平台与应用

第八讲：循环神经网络

循环神经网络

one to one

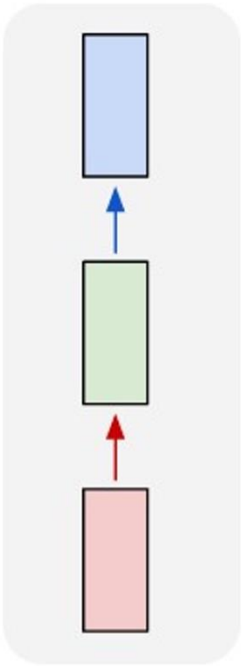


Image Pred
Image
Image Class

循环神经网络

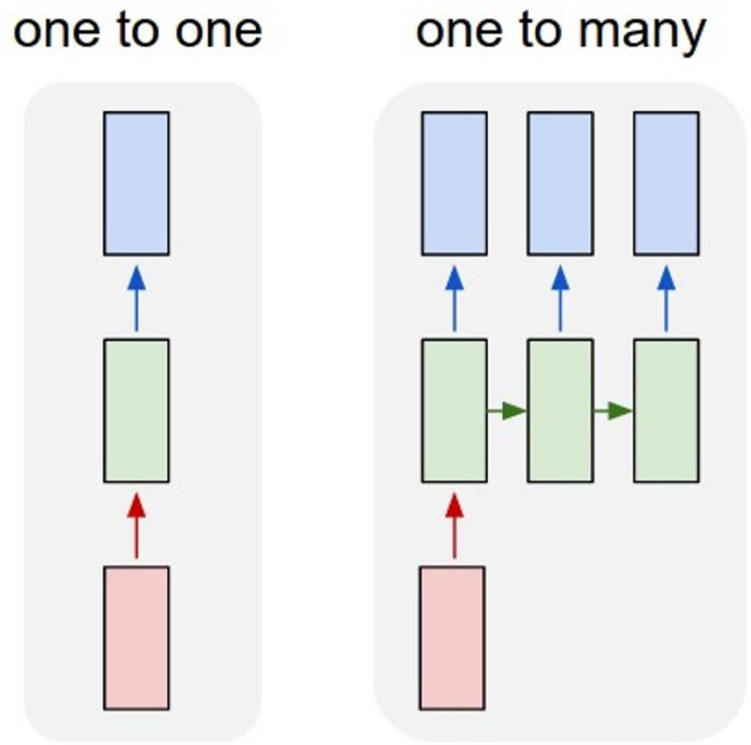


Image Pred	**Image Captioning**
Image	**Image**
Image Class	**Word sentence**

循环神经网络

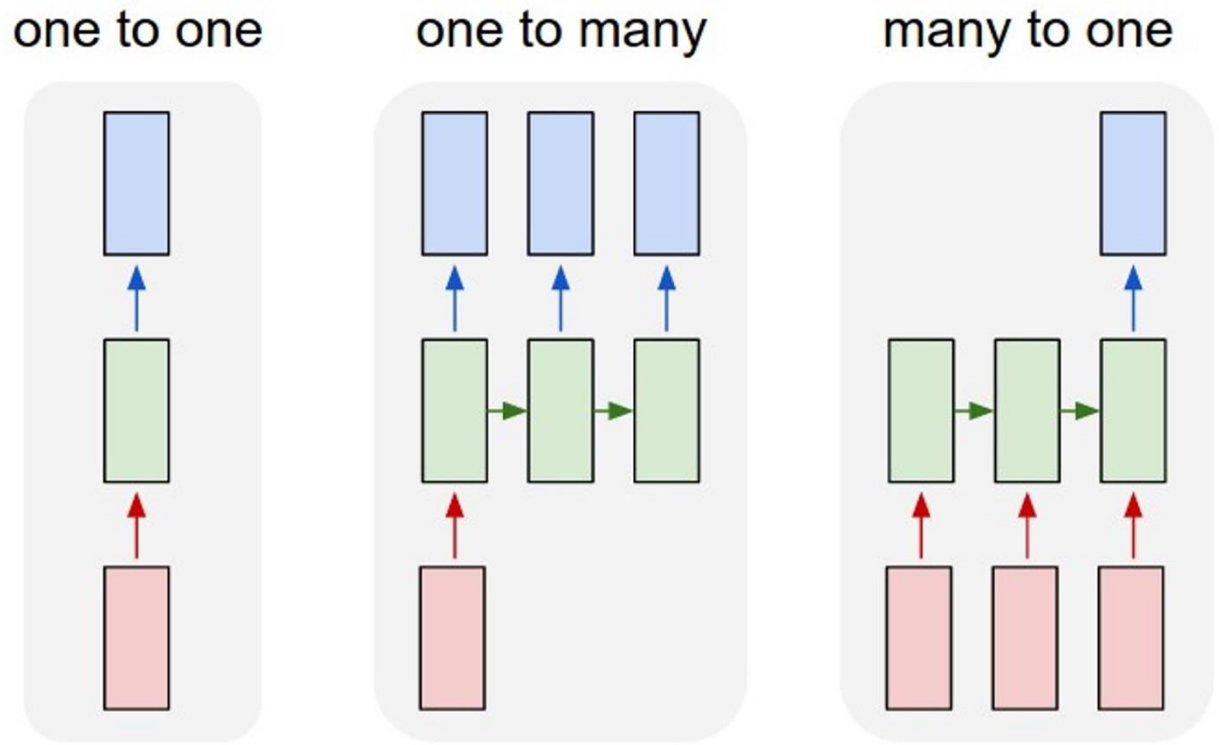


Image Pred	Image Captioning	Action Prediction
Image	Image	Video
Image Class	Word sentence	Action class

循环神经网络

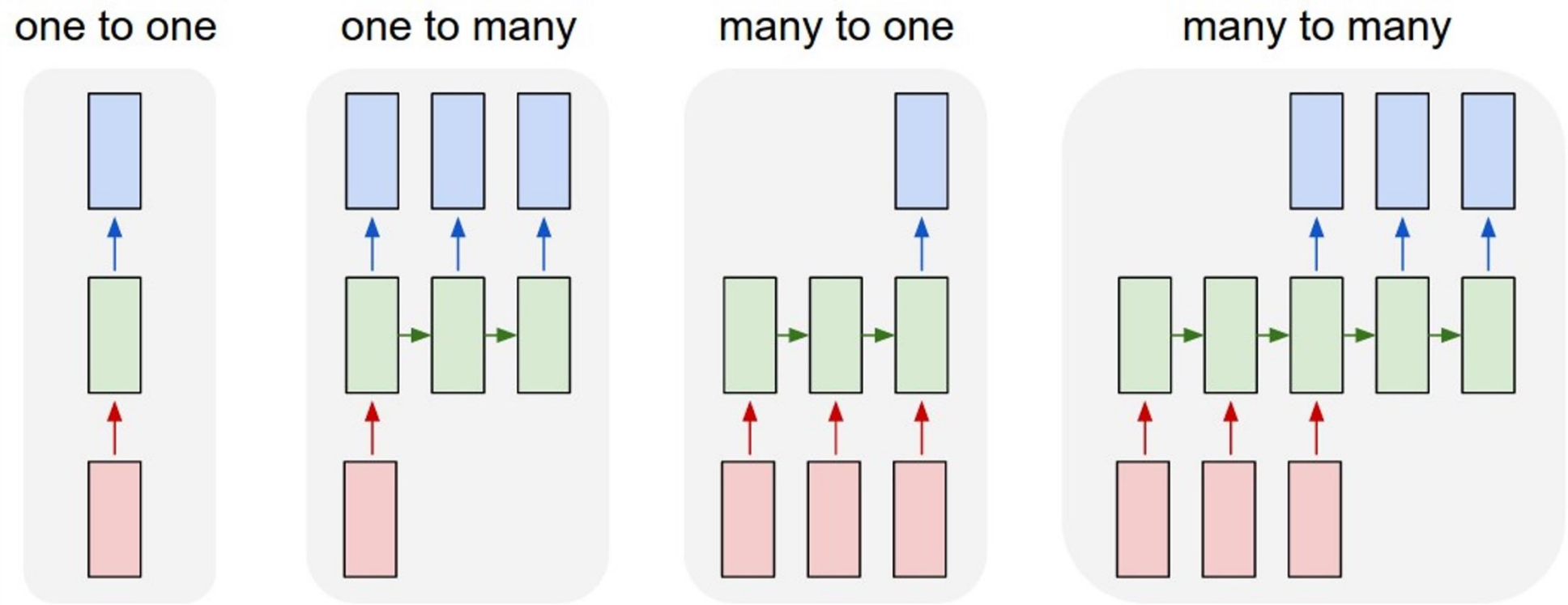


Image Pred	Image Captioning	Action Prediction	Video Captioning
Image	Image	Video	Video
Image Class	Word sentence	Action class	Video caption

循环神经网络

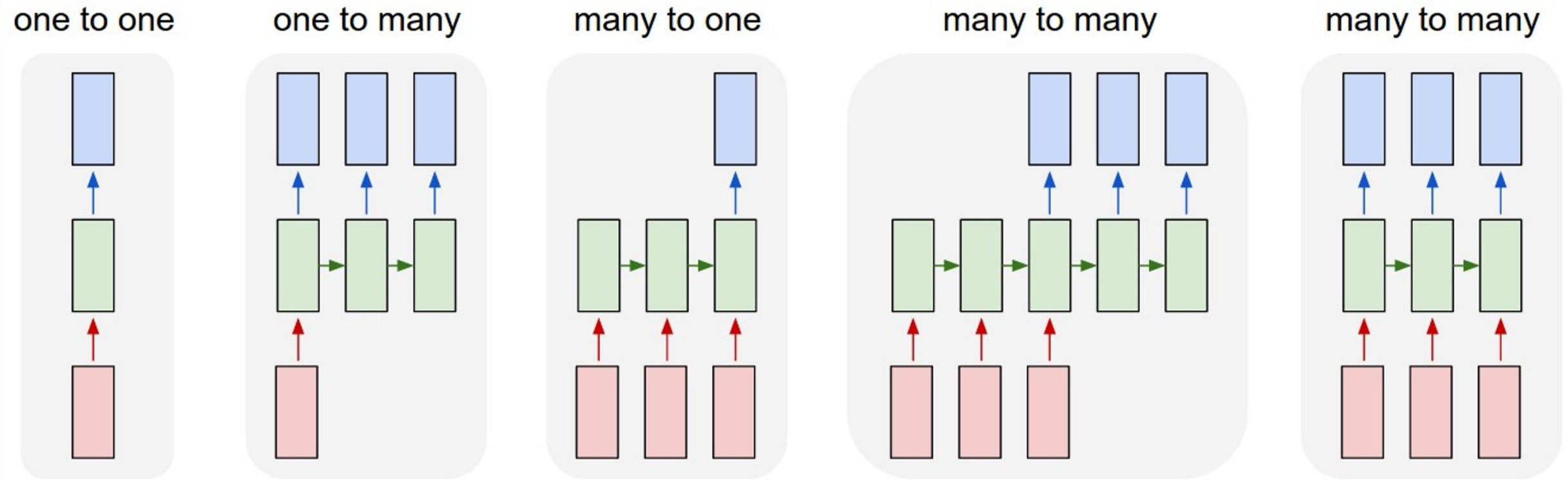
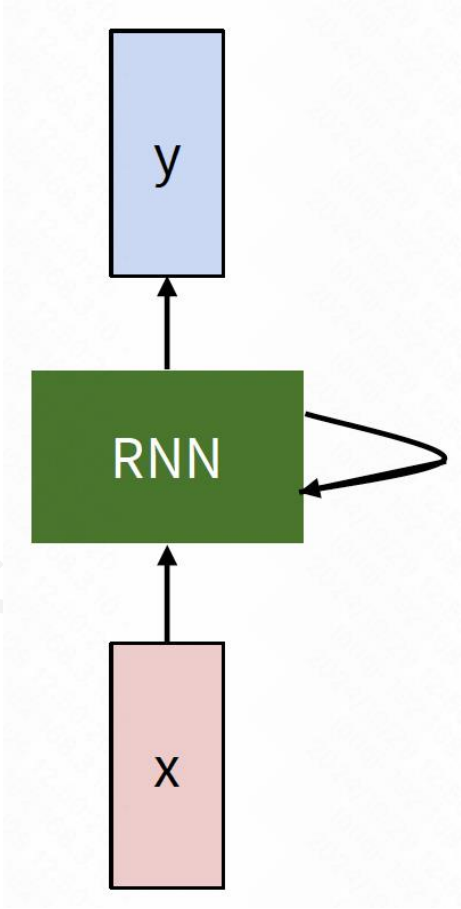


Image Pred	Image Captioning	Action Prediction	Video Captioning	Video Classification
Image	Image	Video	Video	Video
Image Class	Word sentence	Action class	Video caption	Frame class

循环神经网络

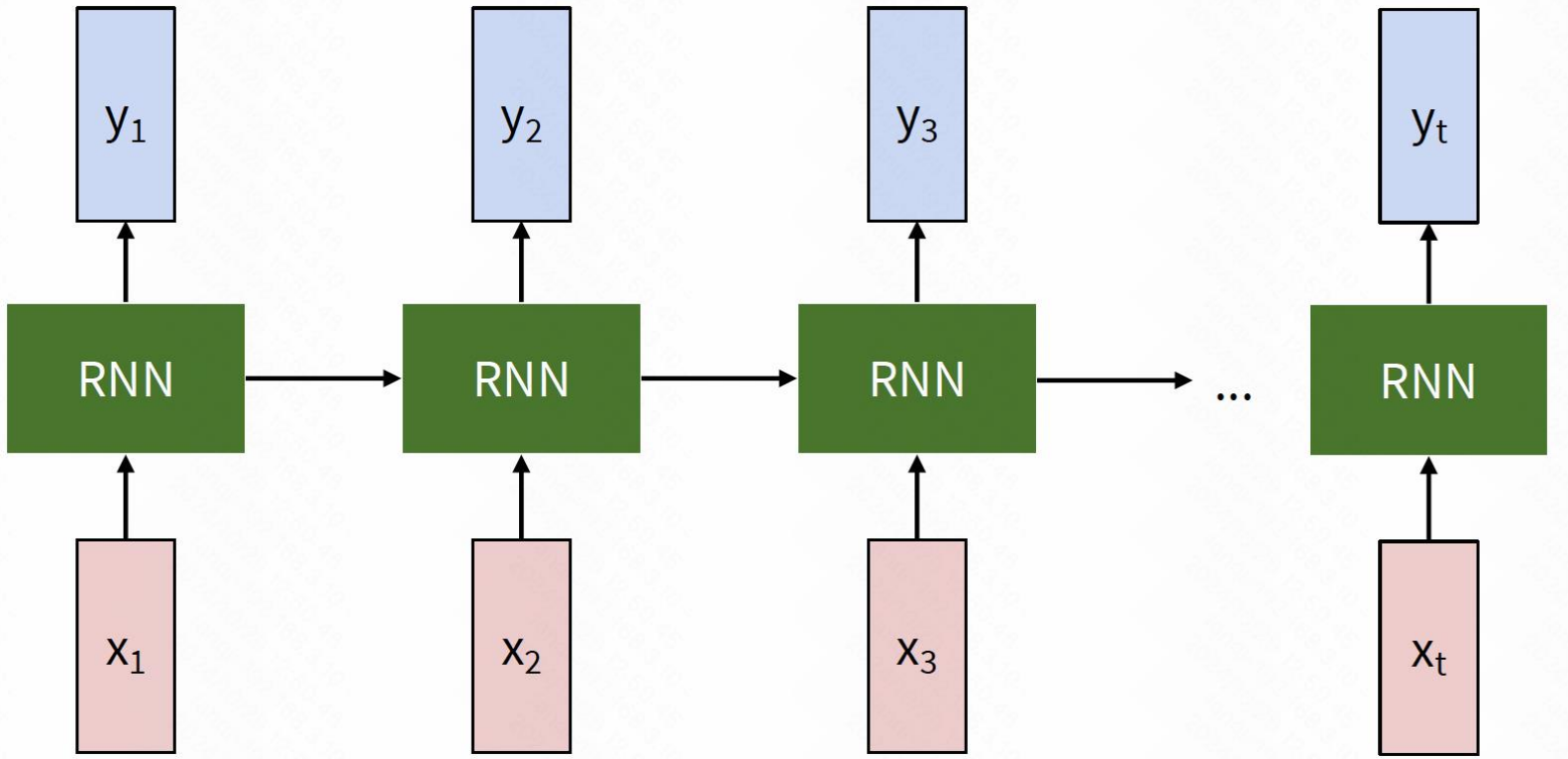
- 核心思想：RNN 有一个“内部状态”，随着序列处理而更新



循环神经网络

南京大学 NANJING UNIVERSITY

- 展开的 RNN



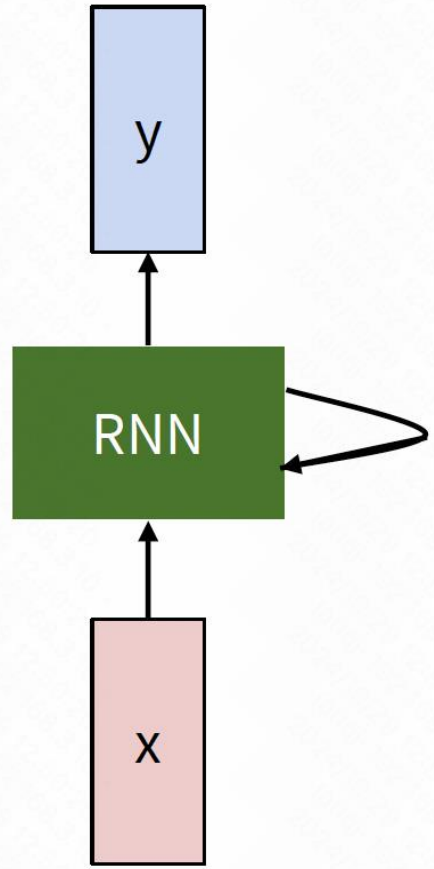
循环神经网络

- RNN 隐藏状态（hidden state）更新：
  - 使用递归公式处理每个时间步的隐藏状态

$$h_t = f_W(h_{t-1}, x_t)$$

new state　　old state　　input vector at some time step

some function with parameters W



循环神经网络

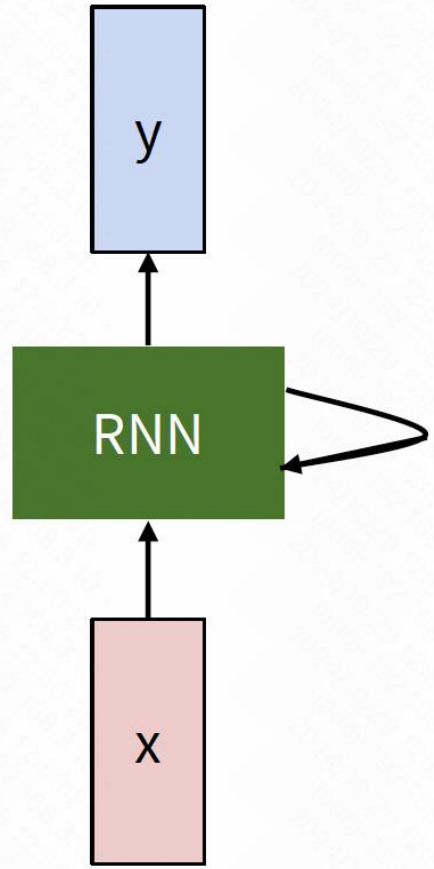
- **RNN 输出计算**

$$y_t = f_{W_{hy}}(h_t)$$

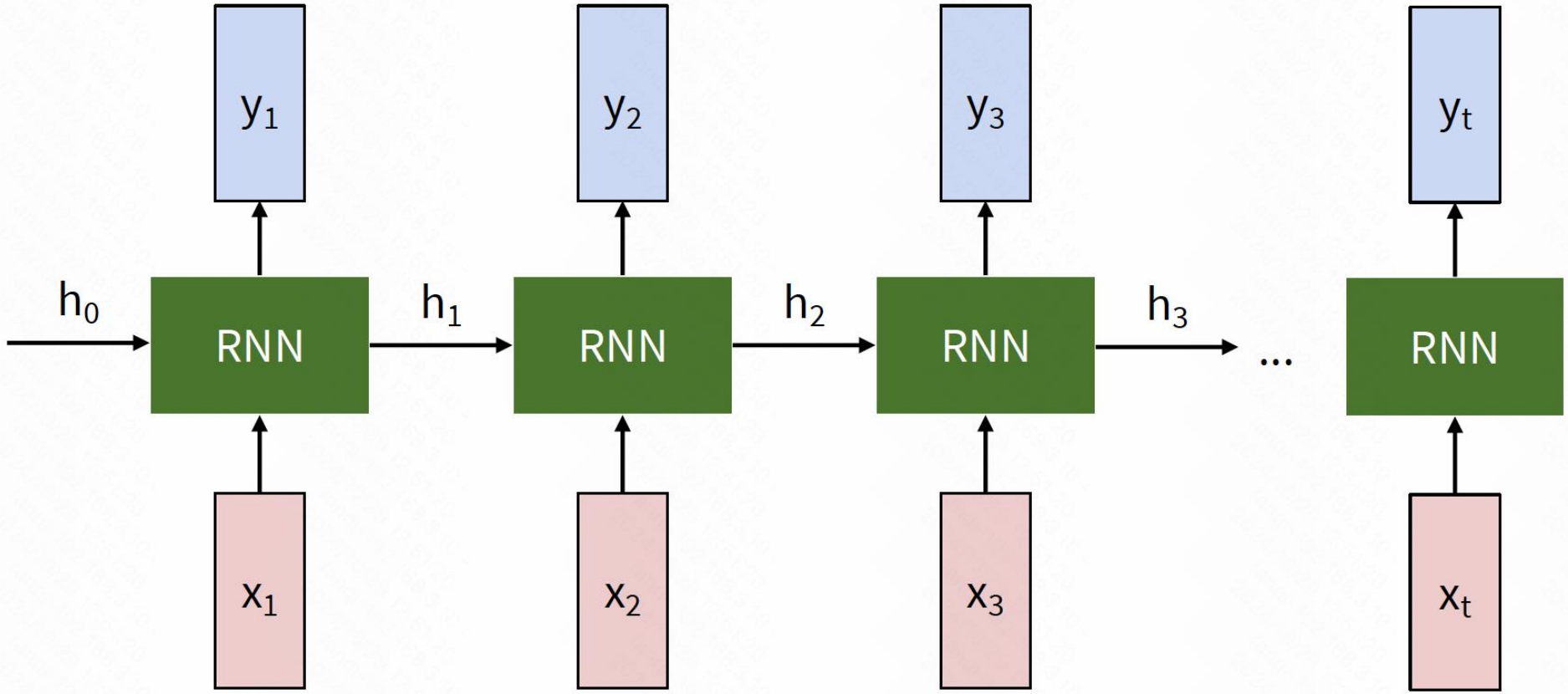
output

new state

another function
with parameters W_{hy}



循环神经网络

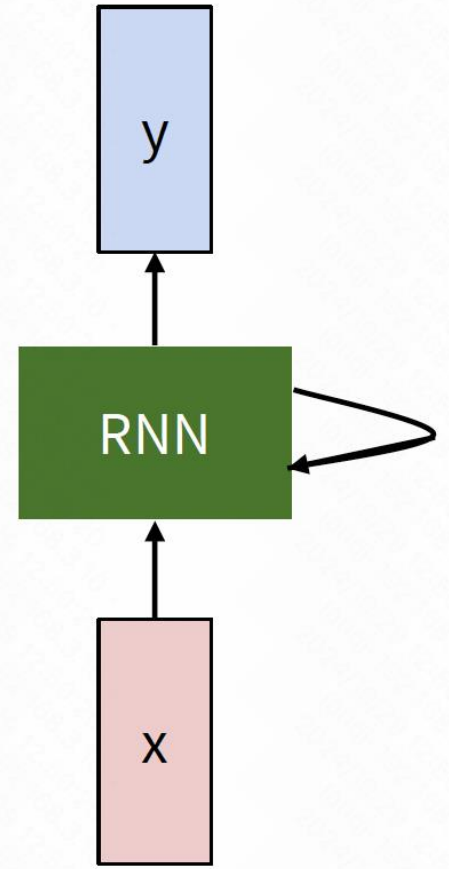


循环神经网络

- 注意：每个时间步的计算都是用相同的函数和参数

所有时间步共享的函数和参数

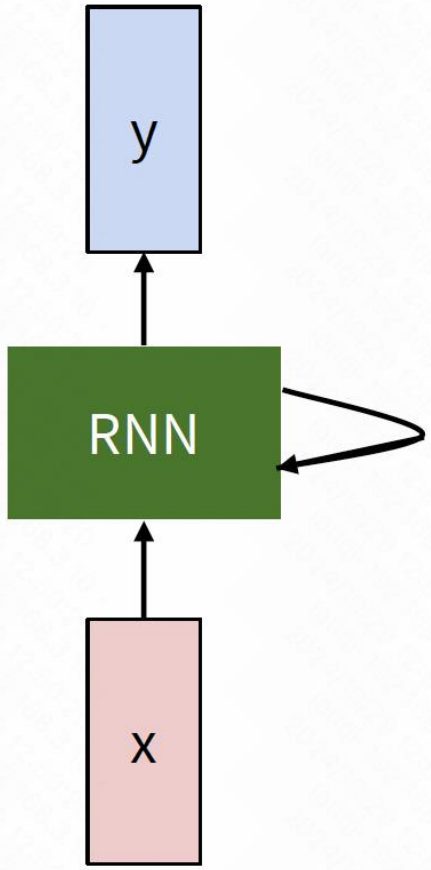
$$h_t = f_W(h_{t-1}, x_t)$$



循环神经网络

南京大学 NANJING UNIVERSITY

- 核心思想



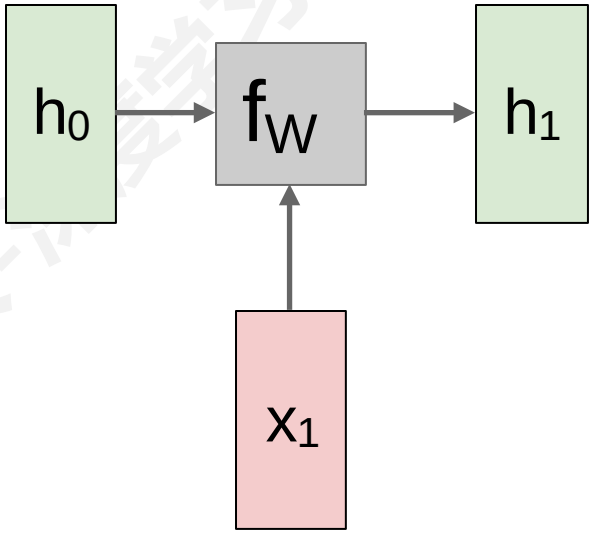
$$h_t = f_W(h_{t-1}, x_t)$$

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

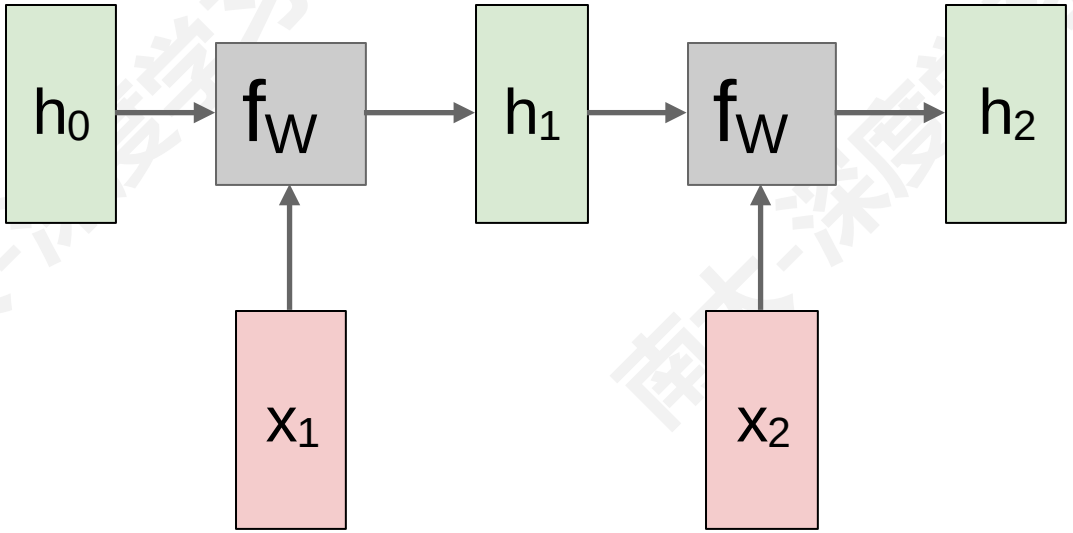
循环神经网络

- **RNN 计算图**



循环神经网络

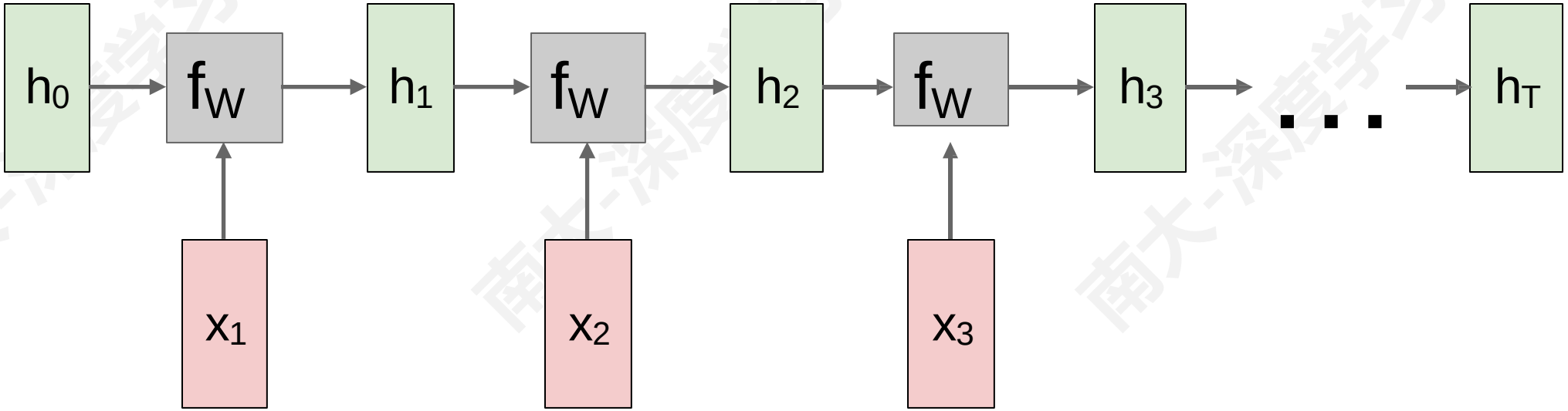
- **RNN 计算图**



循环神经网络

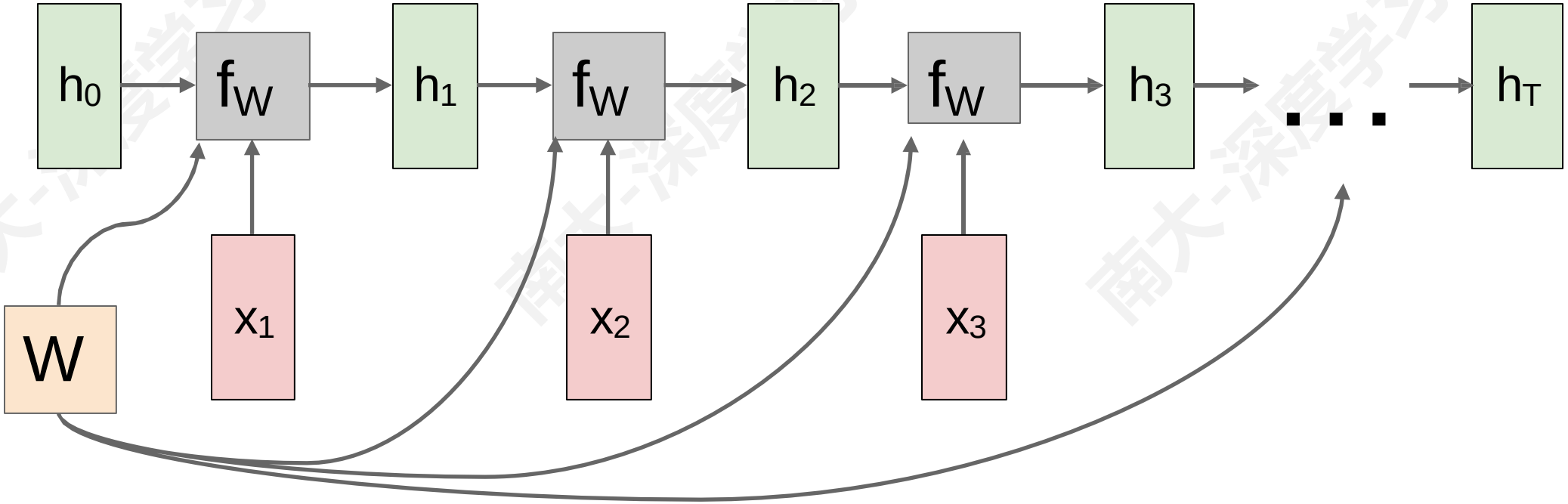
南京大学 NANJING UNIVERSITY

RNN 计算图



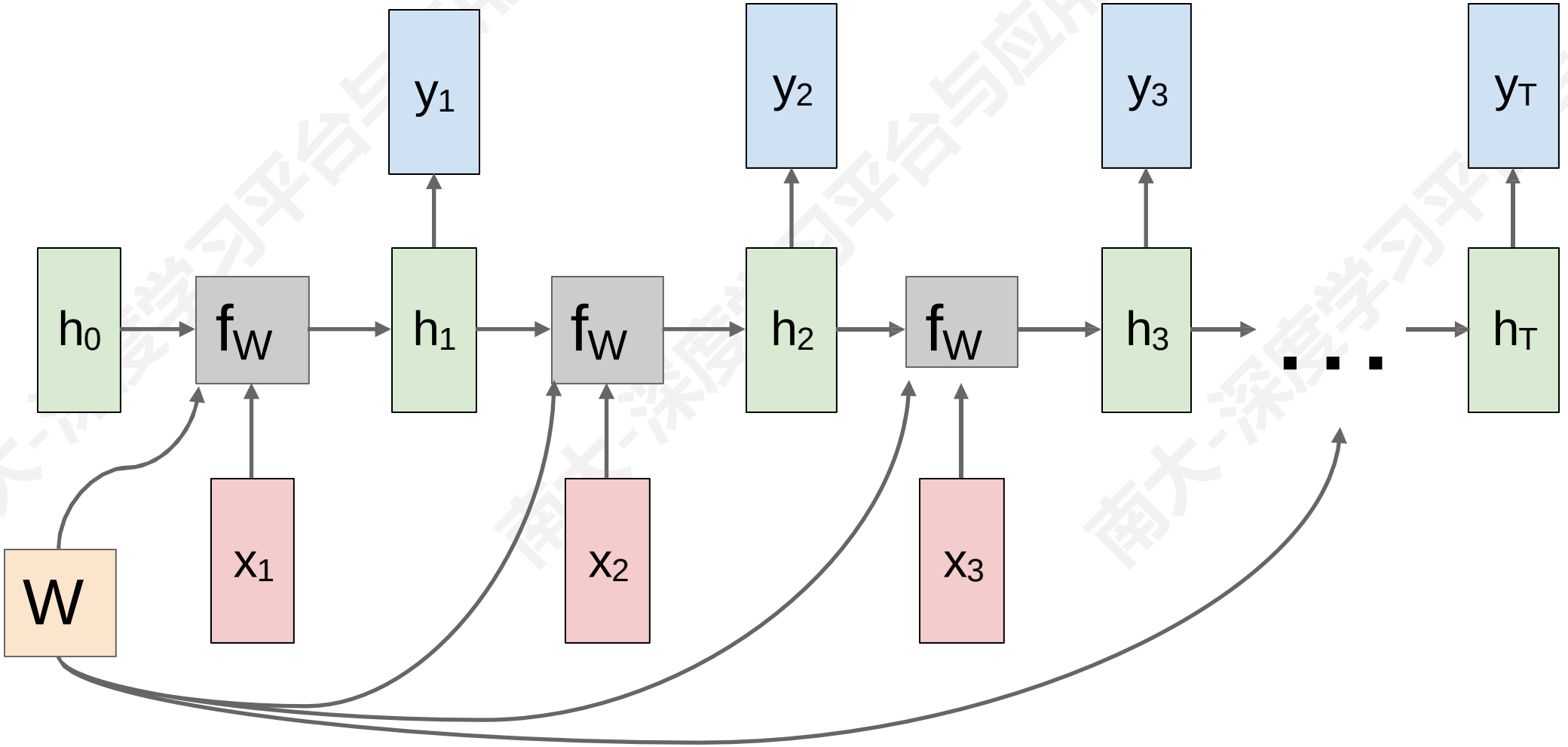
循环神经网络

- RNN 计算图
  - 每个时间步“重复使用”相同模型权重



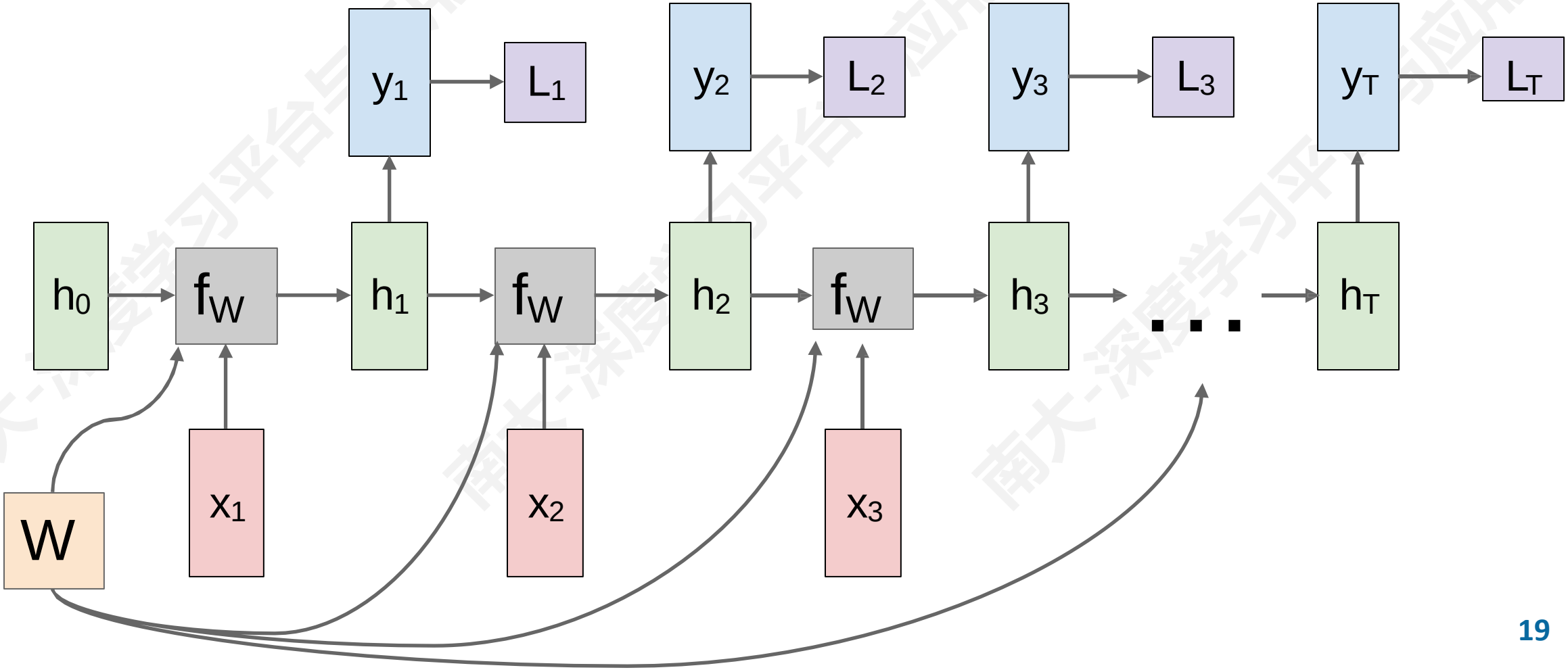
循环神经网络

- RNN 计算图（Many to Many）



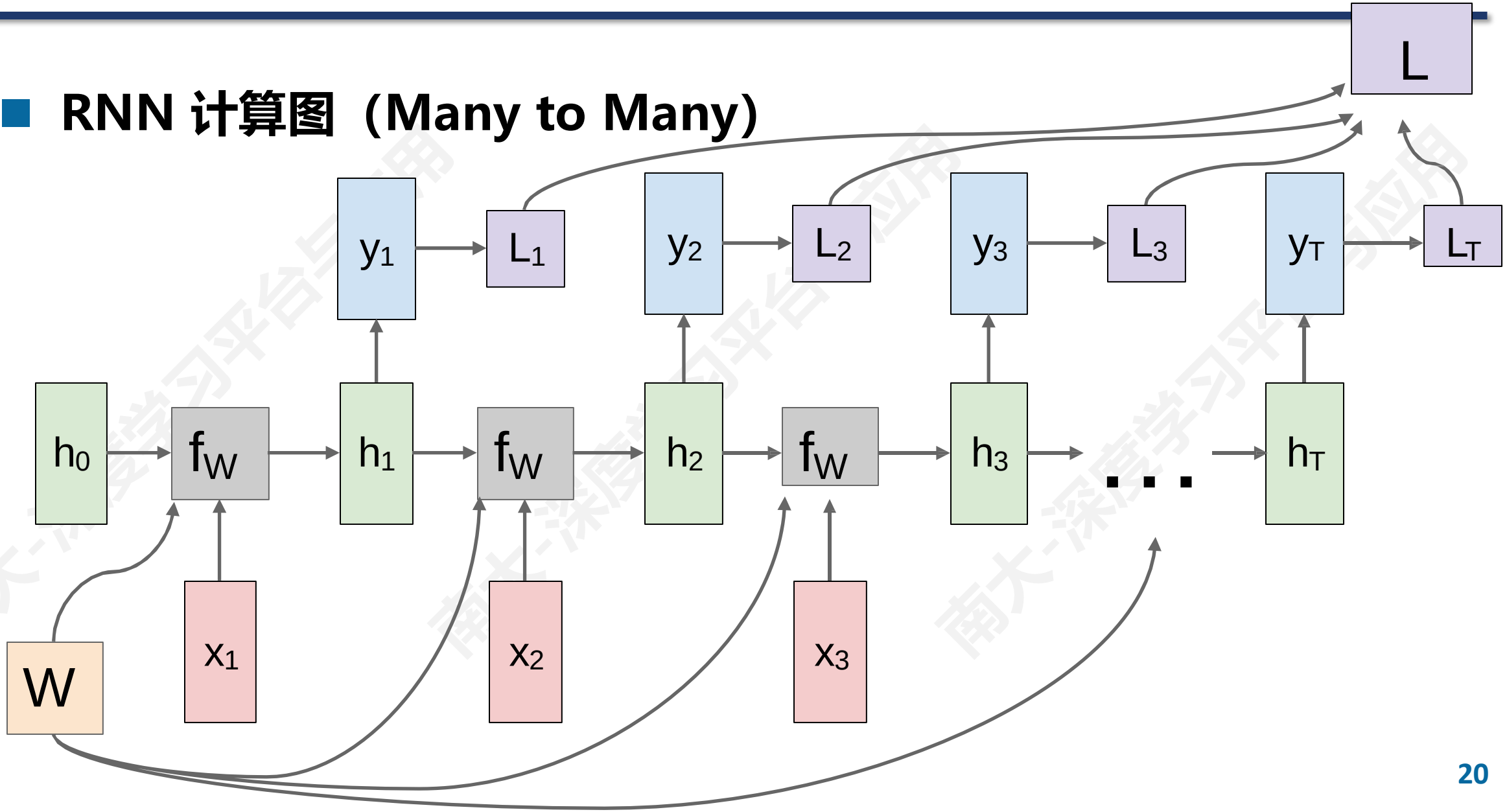
循环神经网络

- RNN 计算图（Many to Many）



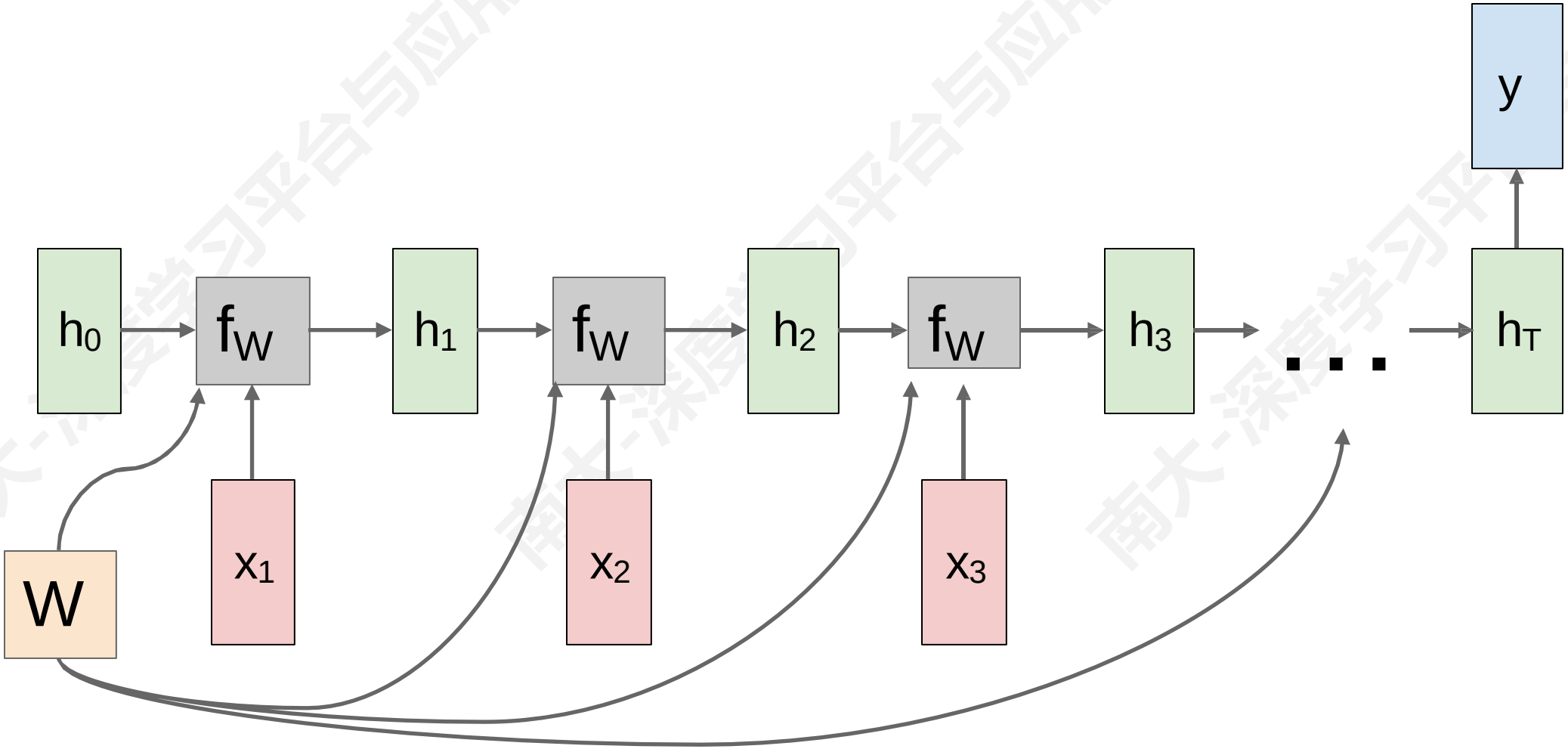
循环神经网络

- RNN 计算图（Many to Many）



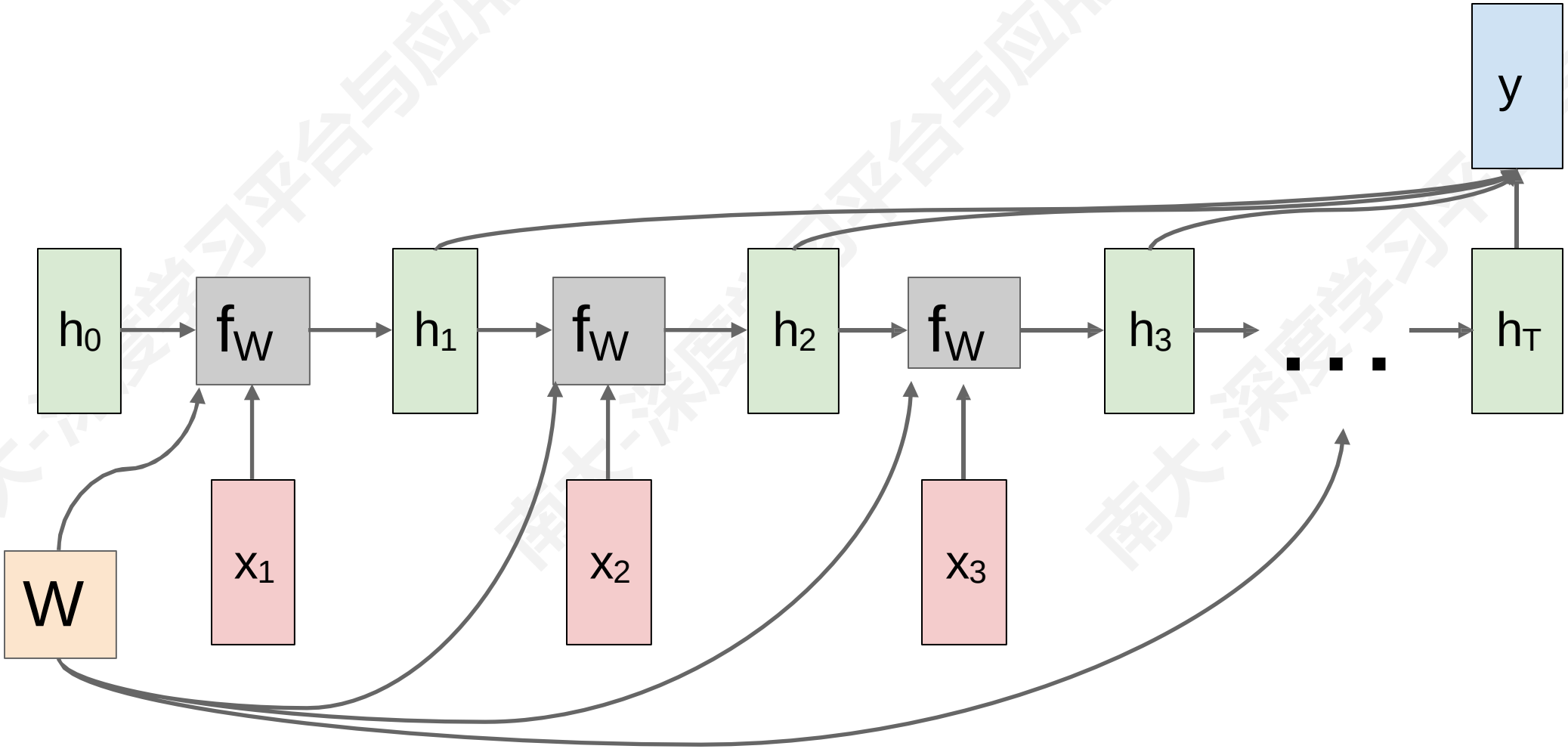
循环神经网络

- RNN 计算图（Many to One）



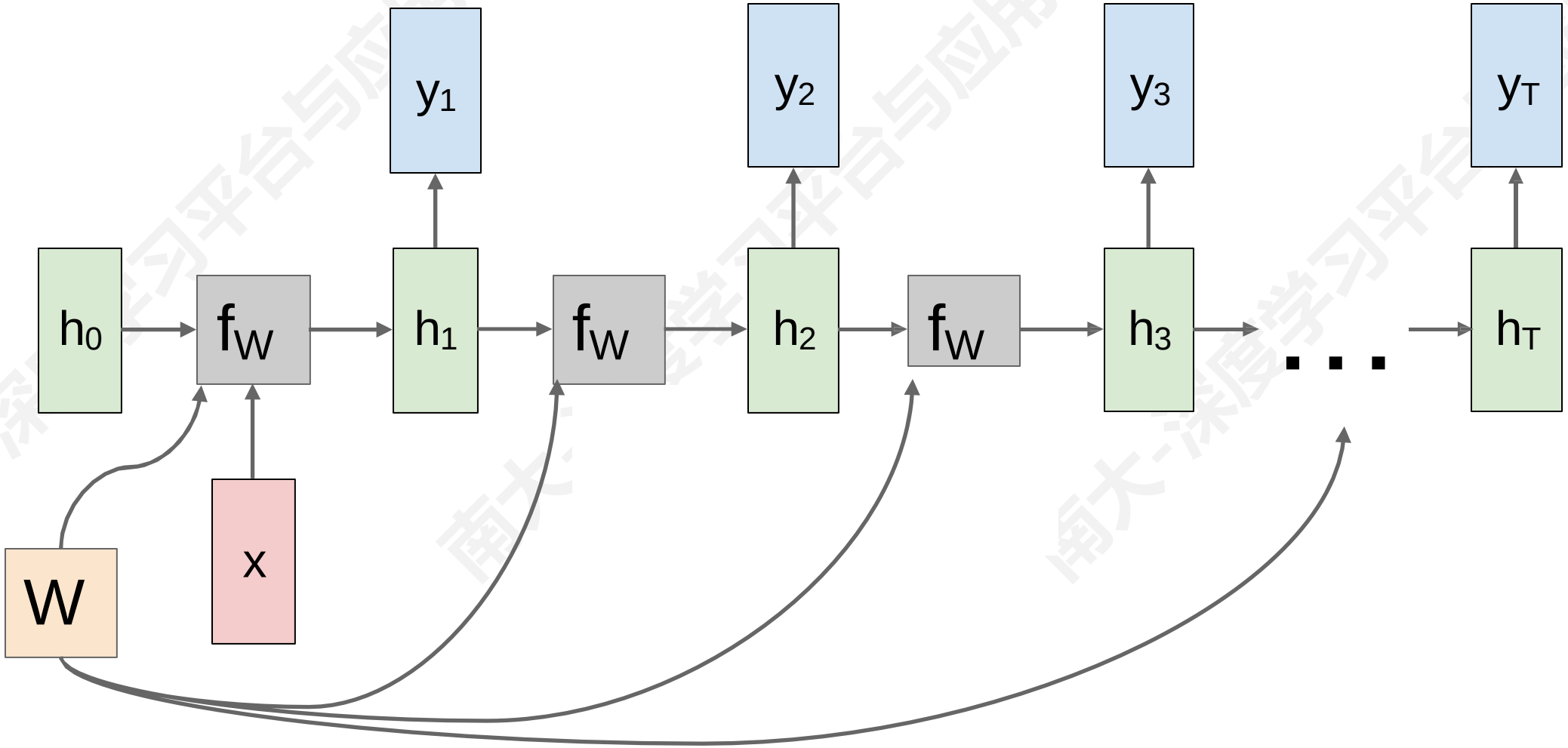
循环神经网络

- RNN 计算图（Many to One）



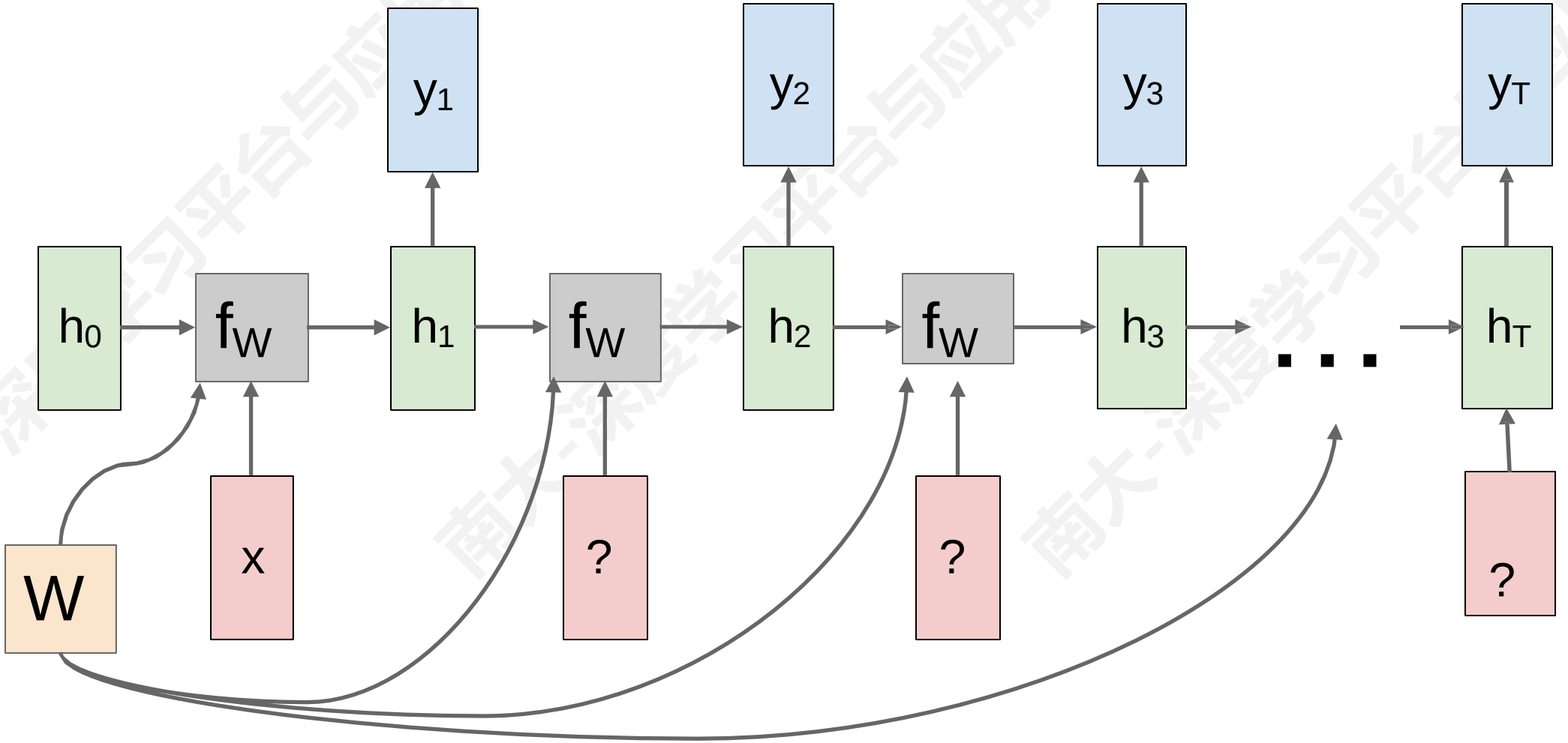
循环神经网络

- RNN 计算图（One to Many）



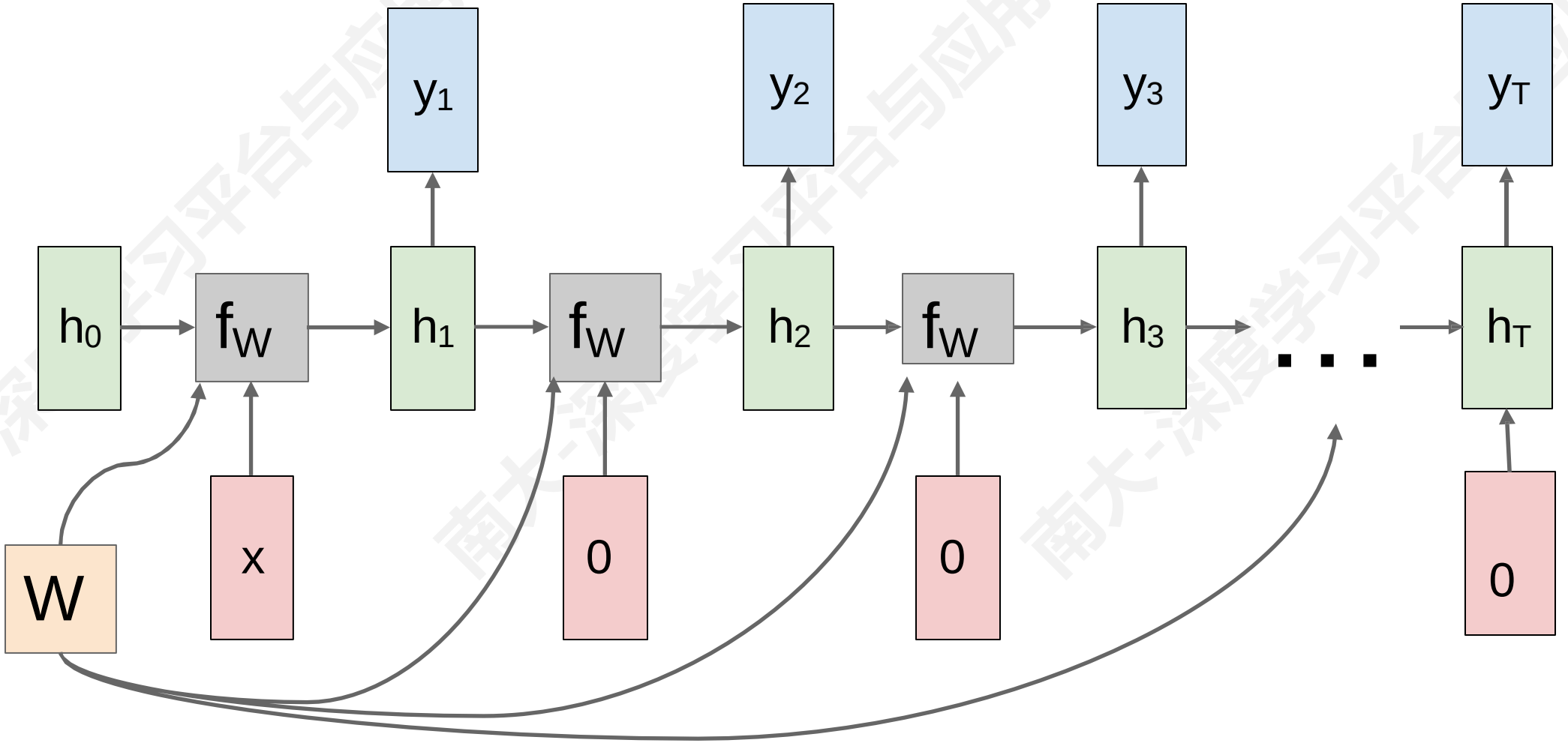
循环神经网络

- RNN 计算图（One to Many）



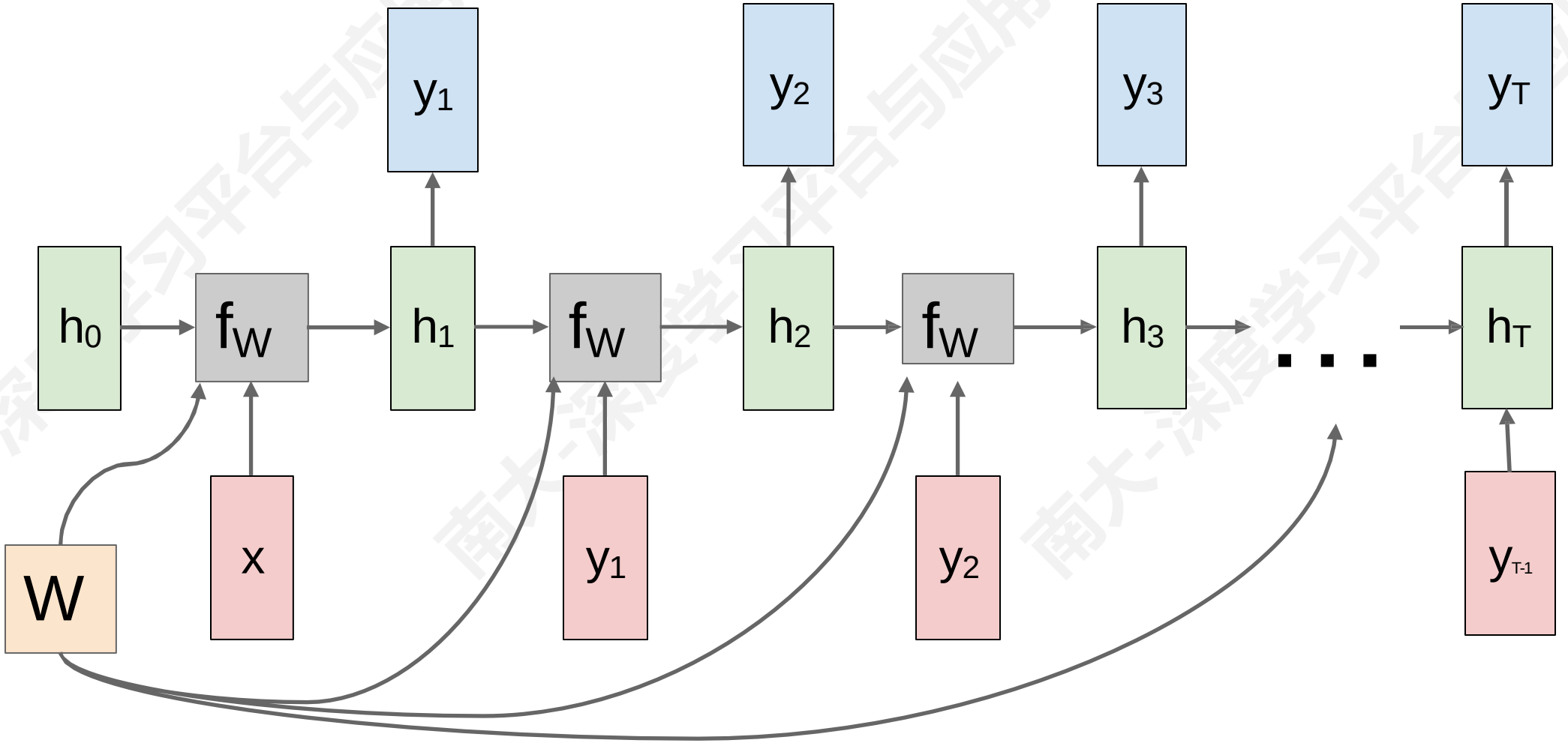
循环神经网络

- RNN 计算图（One to Many）



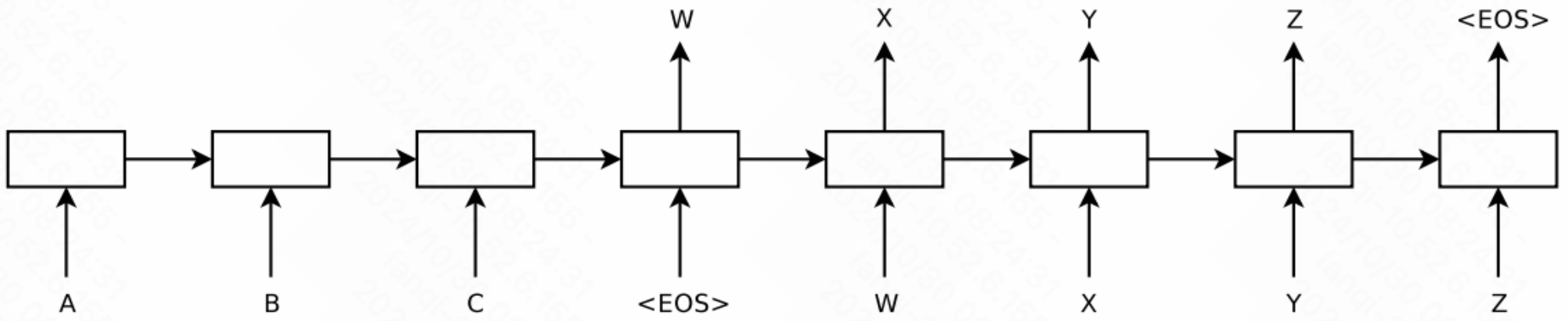
循环神经网络

- RNN 计算图（One to Many）



循环神经网络

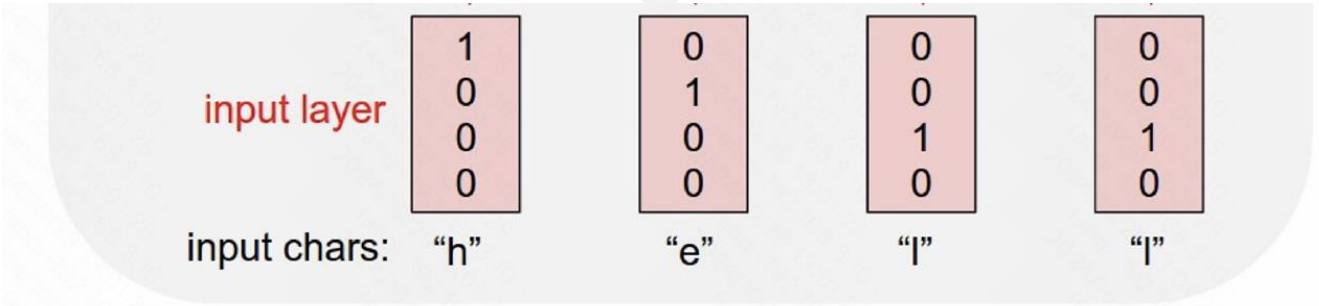
- Sequence to Sequence: Many-to-One + One-to-Many



Sequence to Sequence Learning with Neural Networks, NeurIPS 2014

循环神经网络

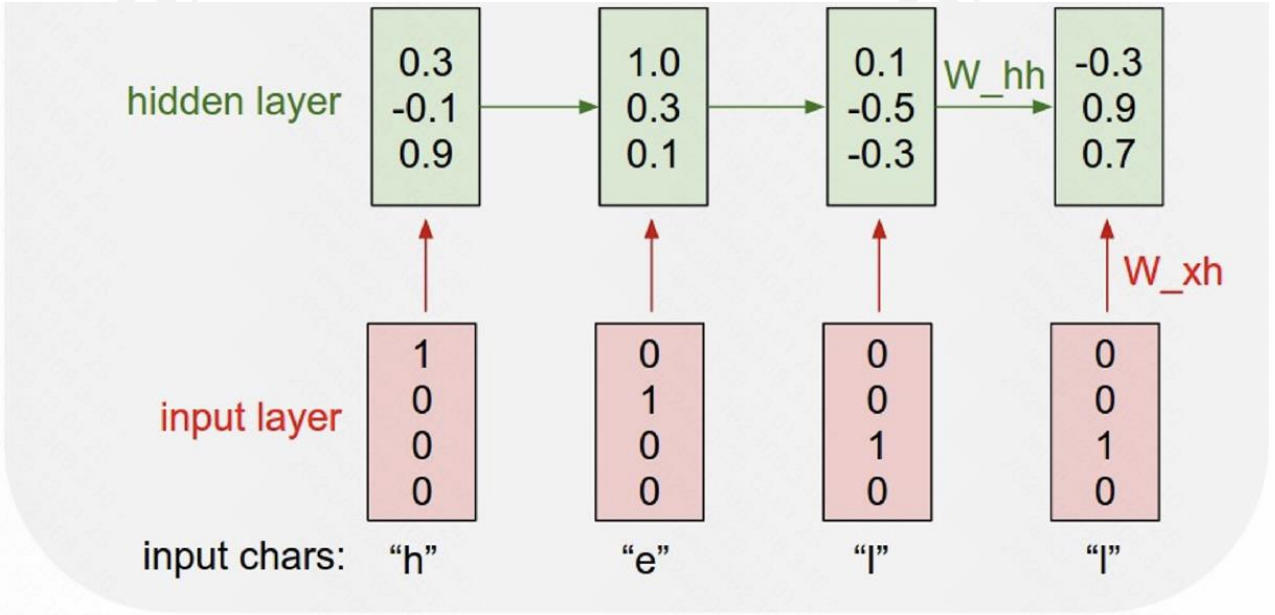
- 例子：预测字符
- 字符表：{h, e, l, o}
- 训练：”hello"



循环神经网络

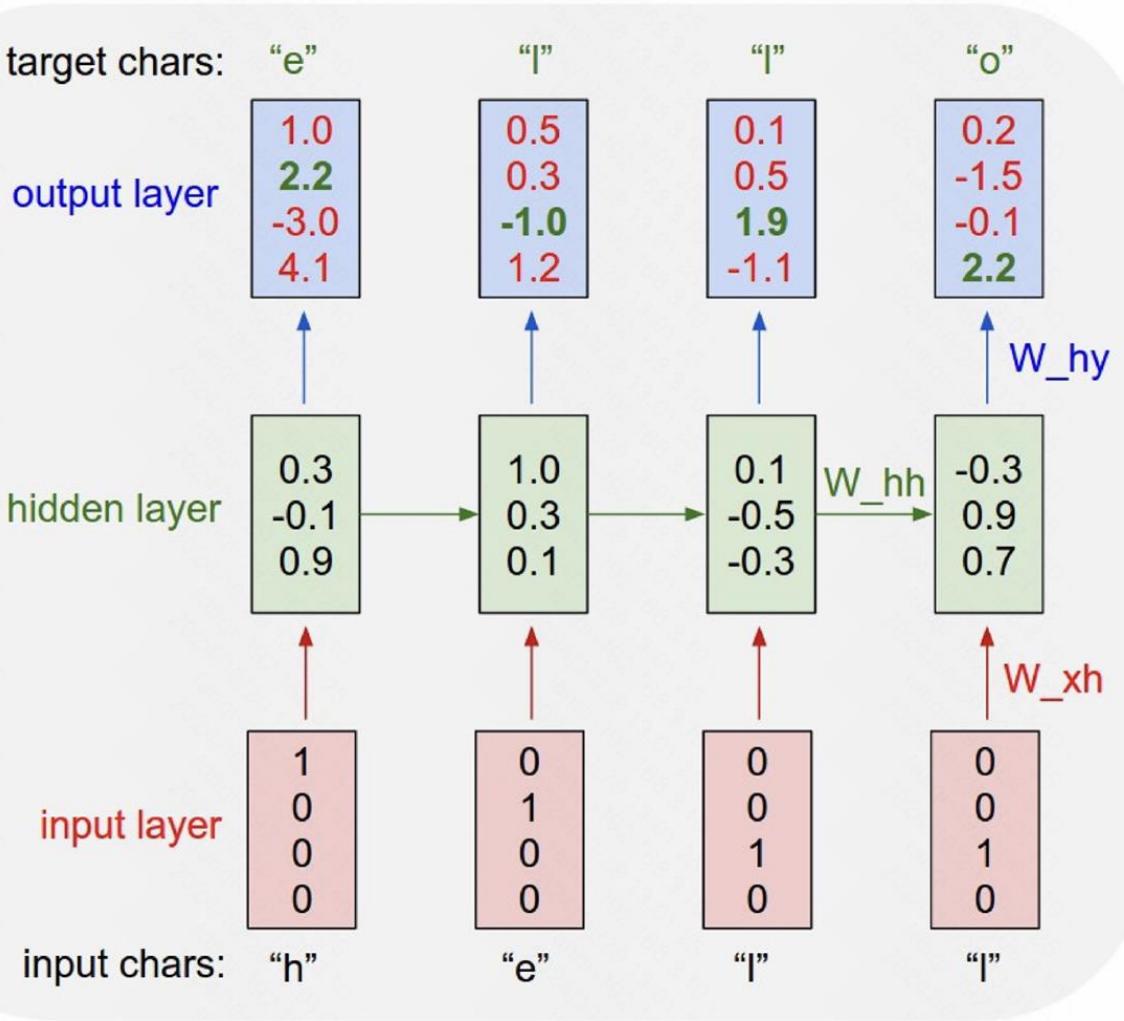
- 例子：预测字符
- 字符表：{h, e, l, o}
- 训练：” hello"

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$



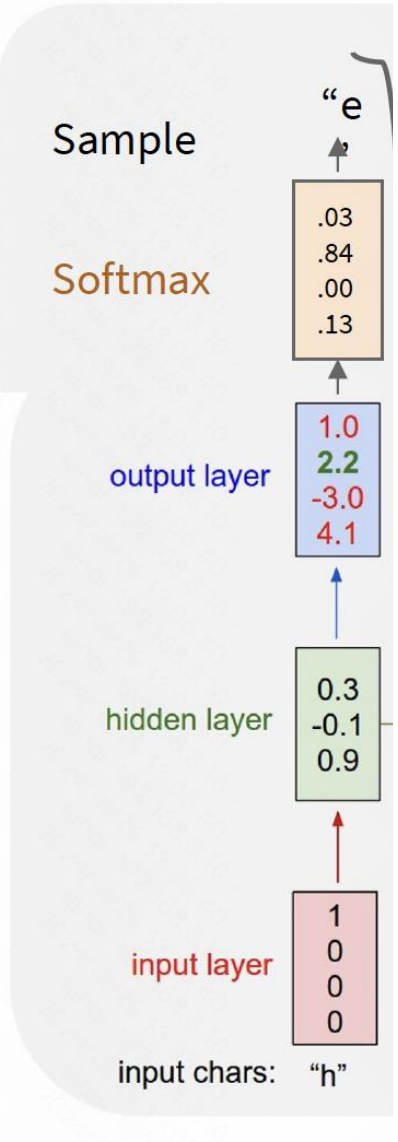
循环神经网络

- 例子：预测字符
- 字符表：{h, e, l, o}
- 训练：” hello"



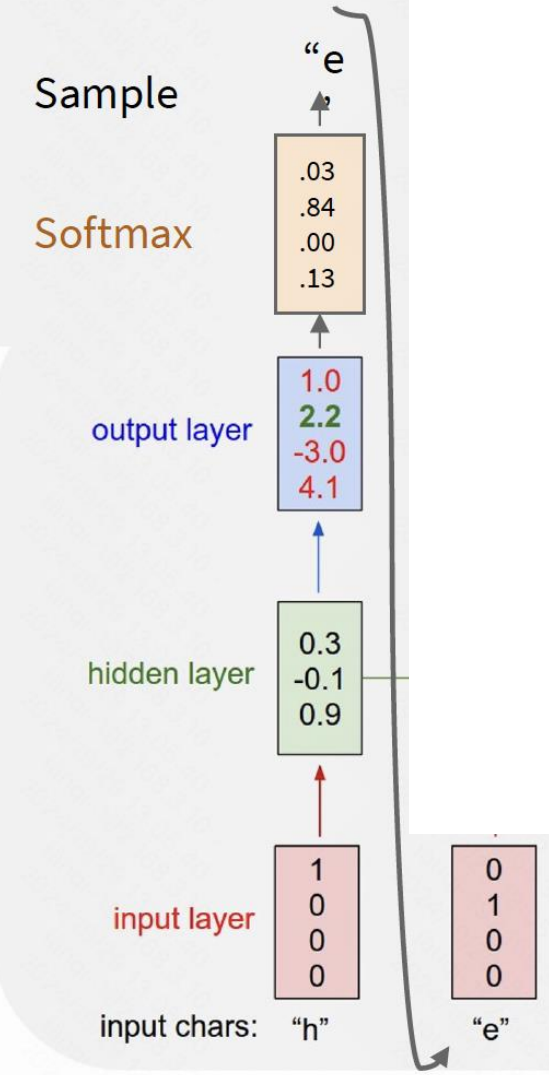
循环神经网络

- 例子：预测字符
- 字符表：{h, e, l, o}
- **测试采样**：每次预测一个结果，并将预测结果送入模型



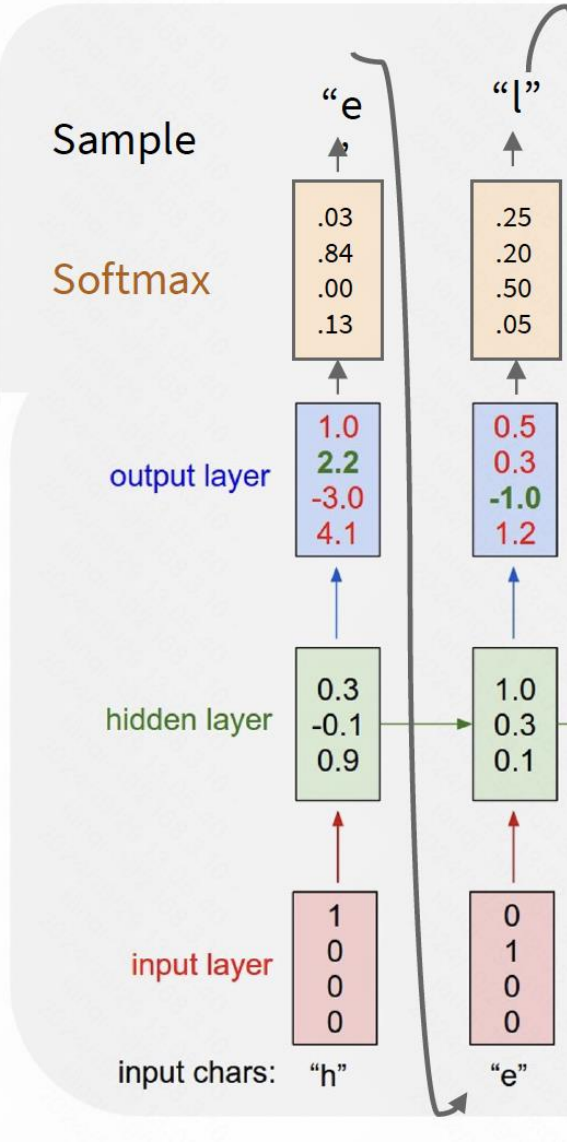
循环神经网络

- 例子：预测字符
- 字符表：{h, e, l, o}
- **测试采样**：每次预测一个结果，并将预测结果送入模型



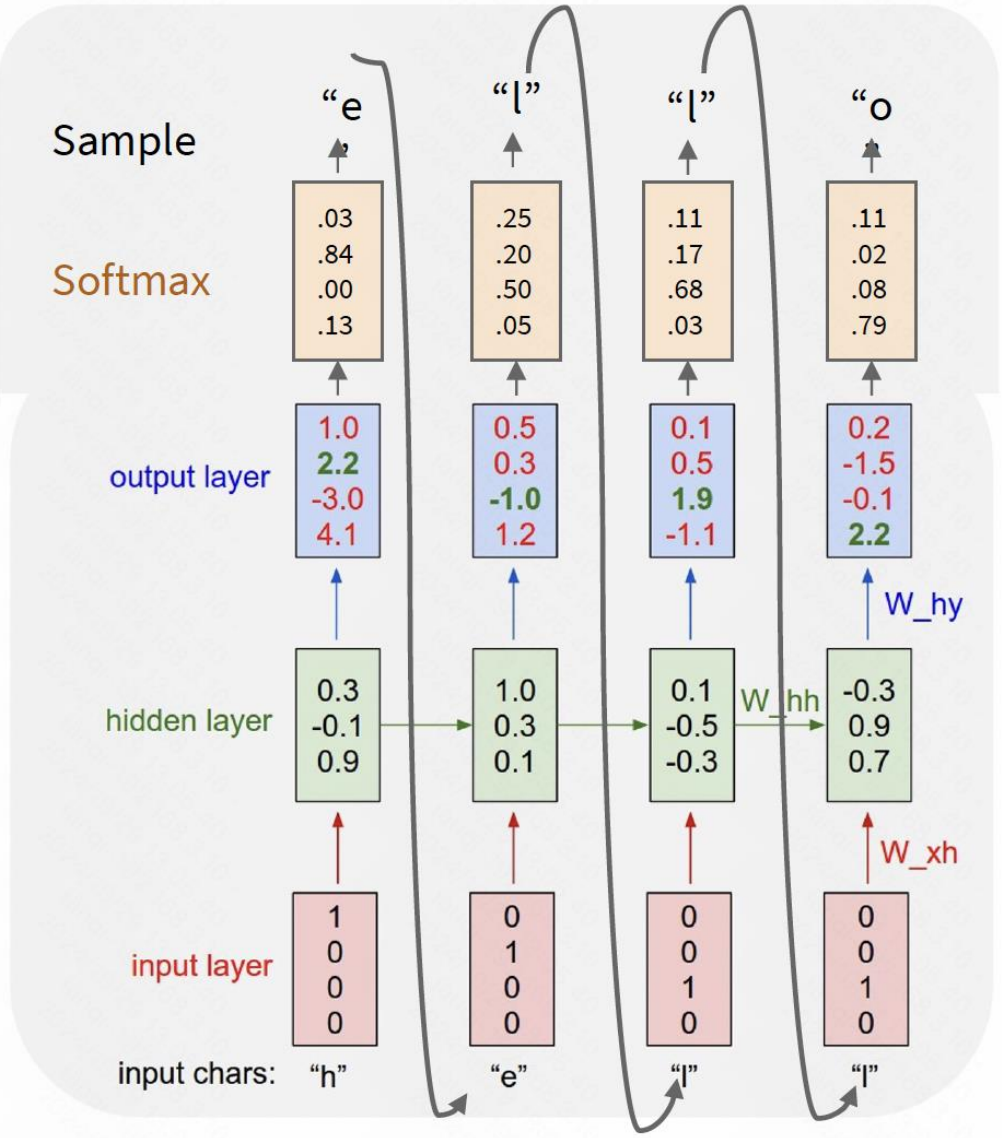
循环神经网络

- 例子：预测字符
- 字符表：{h, e, l, o}
- **测试采样**：每次预测一个结果，并将预测结果送入模型



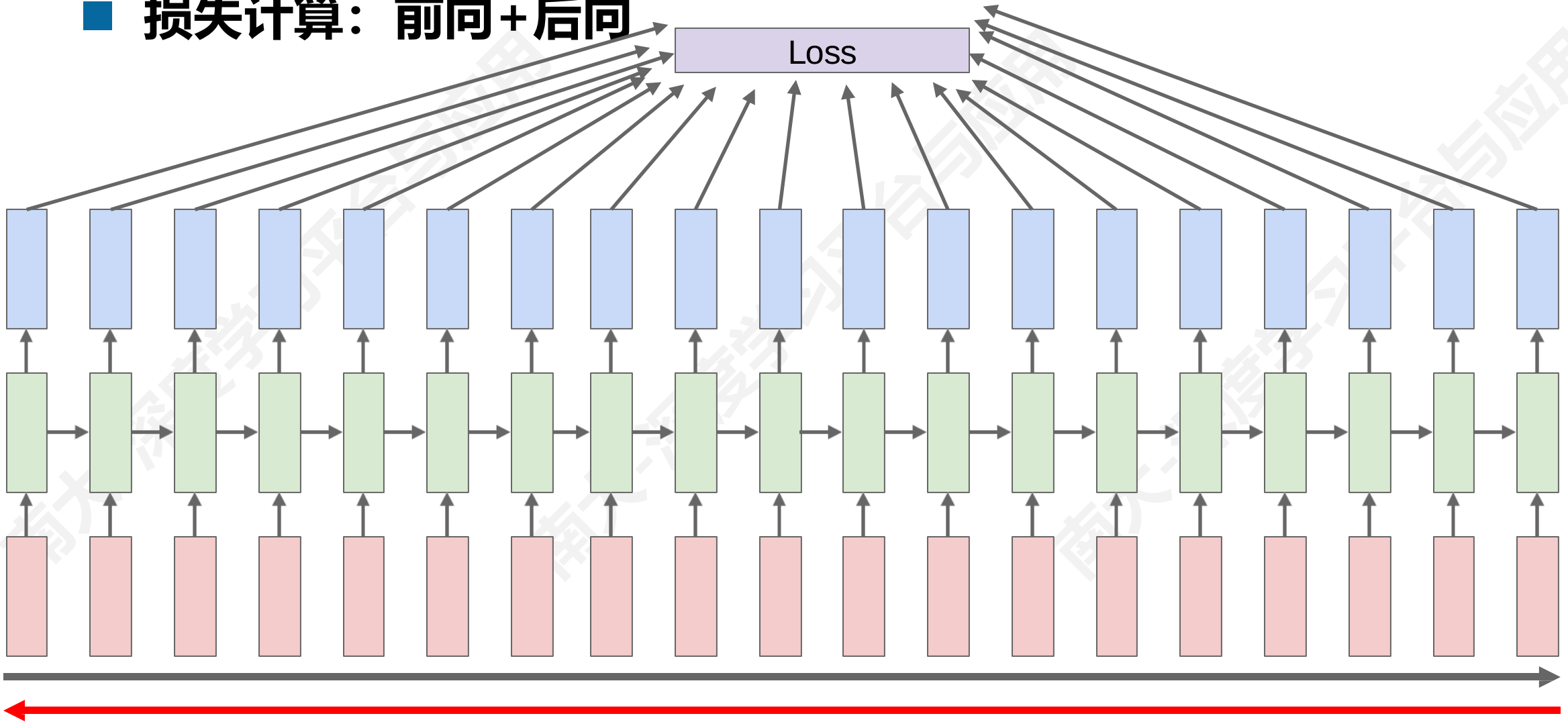
循环神经网络

- 例子：预测字符
- 字符表：{h, e, l, o}
- **测试采样**：每次预测一个结果，并将预测结果送入模型

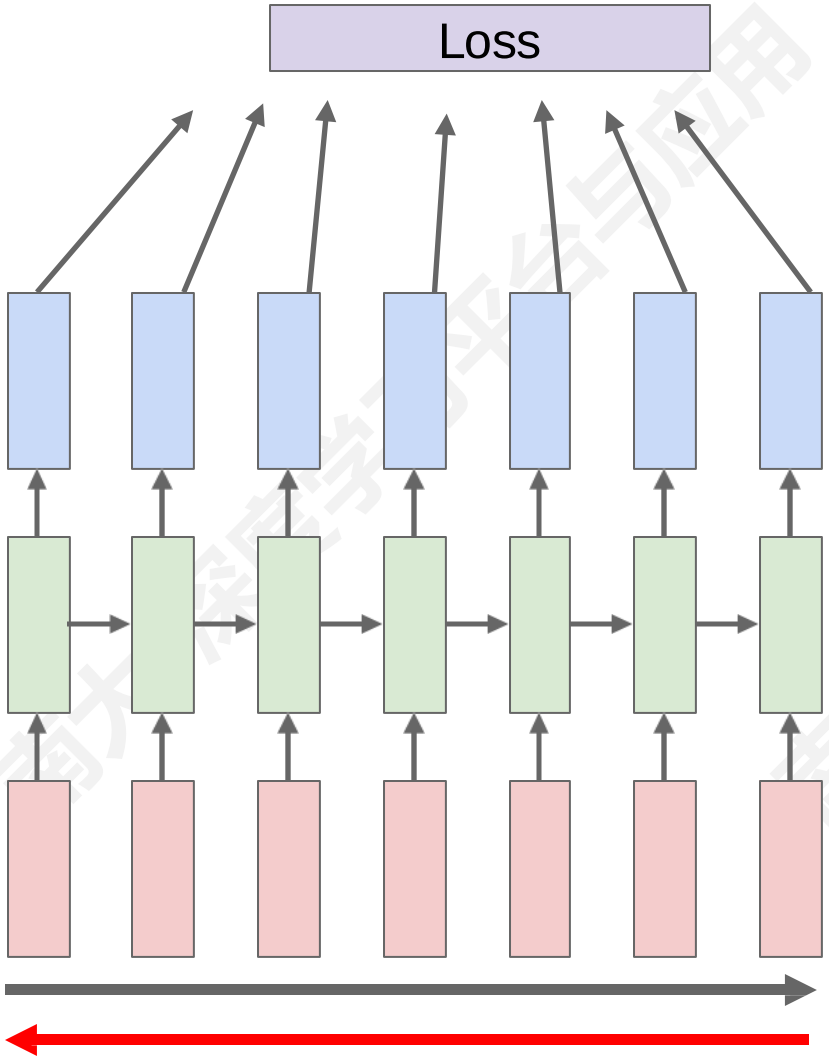


循环神经网络

- 损失计算：前向+后向



循环神经网络

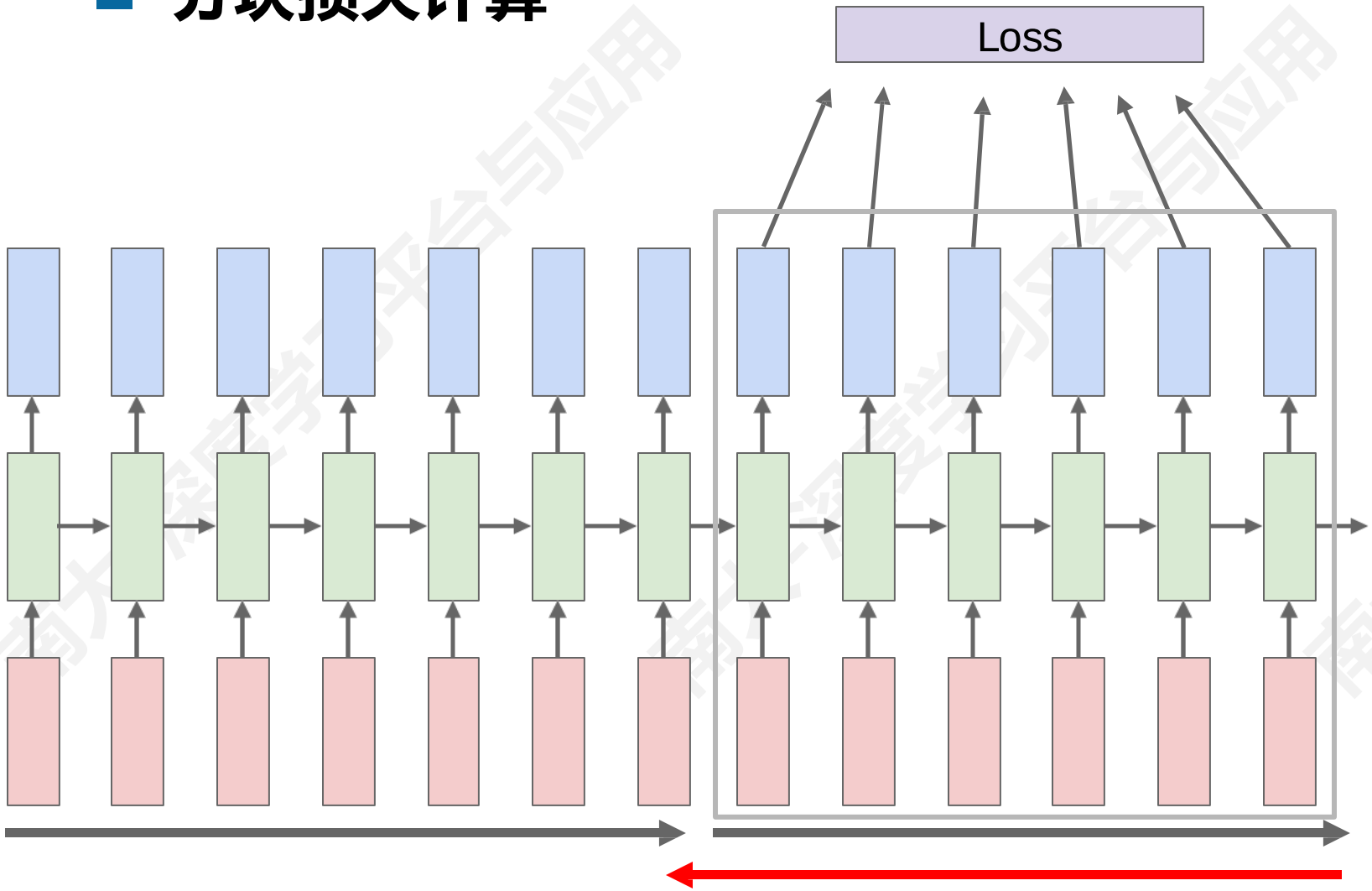


- 分块损失计算
- 计算所有时间步的前向传播
- 但只在每时间块内进行反向传播

循环神经网络

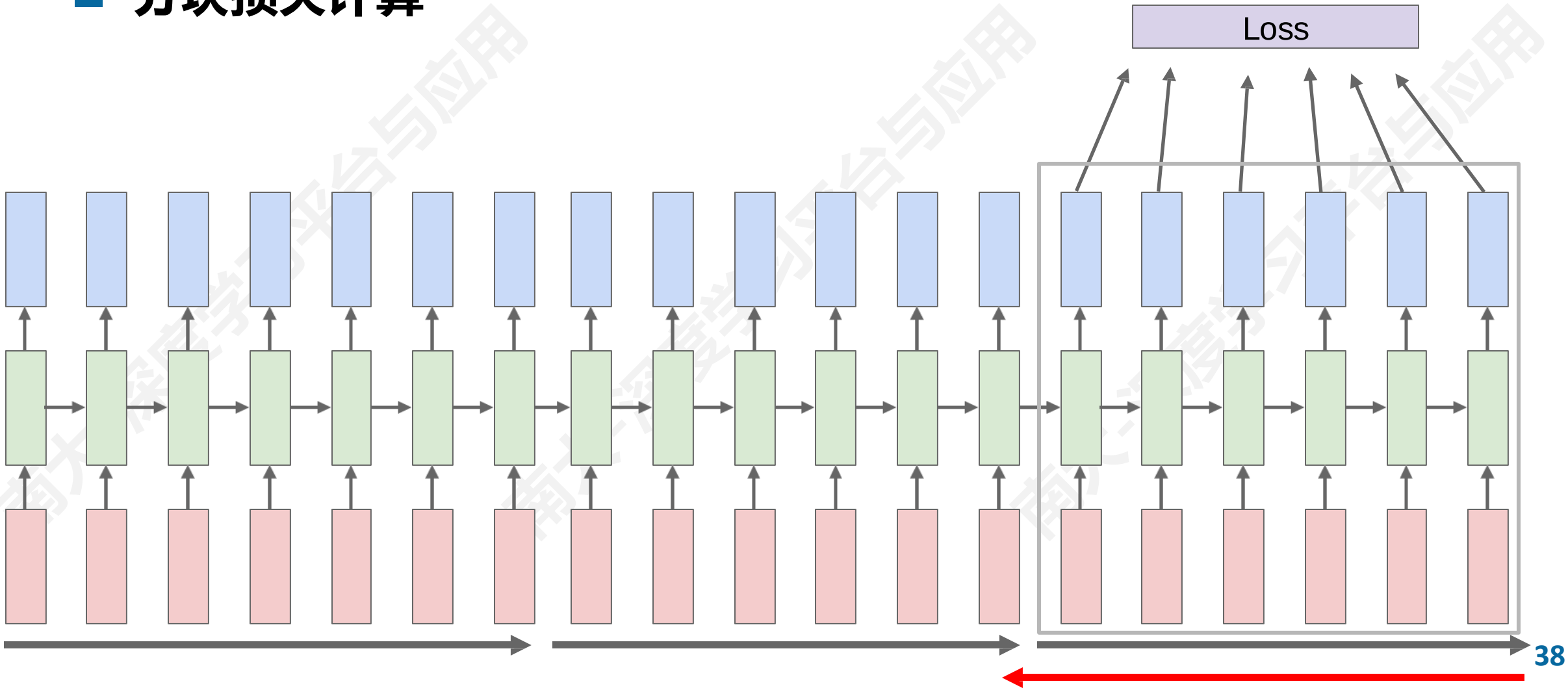
南京大学 NANJING UNIVERSITY

- 分块损失计算



循环神经网络

- 分块损失计算



循环神经网络

```
tyntd-iafhatawiaoihrdemot  lytdws  e ,tfti, astai f ogoh eoase rrranbyne 'nhthnee e
plia tklrgd t o idoe ns,smtt   h ne etie h,hregtrs nigtike,aoaenns lng
```

↓ 训练

```
"Tmont thithey" fomesscerliund
Keushey. Thom here
sheulke, anmerenith ol sivh I lalterthend Bleipile shuwy fil on aseterlome
coaniogennc Phe lism thond hon at. MeiDimorotion in ther thize."
```

↓ 继续训练

```
Aftair fall unsuch that the hall for Prince Velzonski's that me of
her hearly, and behs to so arwage fiving were to it beloge, pavu say falling misfort
how, and Gogition is so overelical and ofter.
```

↓ 继续训练

```
"Why do what that day," replied Natasha, and wishing to himself the fact the
princess, Princess Mary was easier, fed in had oftened him.
Pierre aking his soul came to the packs and drove up his father-in-law women.
```

循环神经网络

- 生成代码

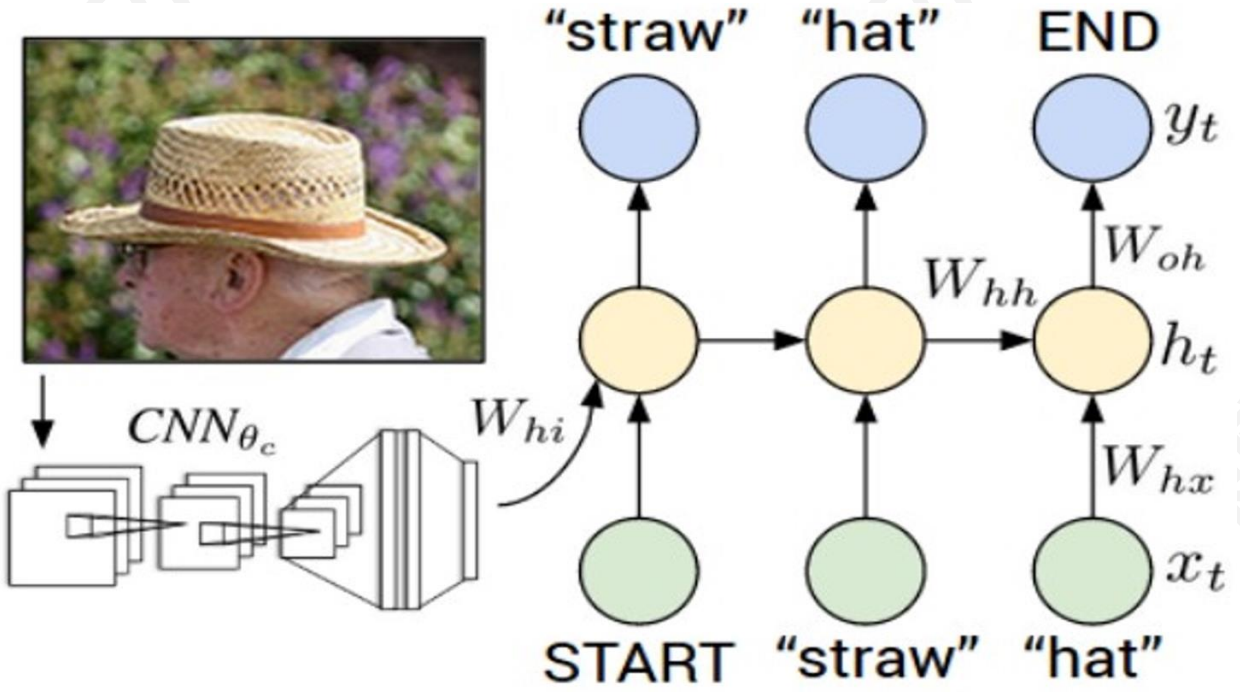
```
static void do_command(struct seq_file *m, void *v)
{
  int column = 32 << (cmd[2] & 0x80);
  if (state)
    cmd = (int)(int_state ^ (in_8(&ch->ch_flags) & Cmd) ? 2 : 1);
  else
    seq = 1;
  for (i = 0; i < 16; i++) {
    if (k & (1 << 1))
      pipe = (in_use & UMXTHREAD_UNCCA) +
        ((count & 0x00000000fffffff8) & 0x000000f) << 8;
    if (count == 0)
      sub(pid, ppc_md.kexec_handle, 0x20000000);
    pipe_set_bytes(i, 0);
  }
  /* Free our user pages pointer to place camera if all dash */
  subsystem_info = &of_changes[PAGE_SIZE];
  rek_controls(offset, idx, &soffset);
  /* Now we want to deliberately put it to device */
  control_check_polarity(&context, val, 0);
  for (i = 0; i < COUNTER; i++)
    seq_puts(s, "policy ");
}
```

循环神经网络

- RNN优势：
  - 可以处理任何长度的输入
  - 步骤t的计算（理论上）可以使用之前许多步骤的信息
  - 输入时间越长，模型尺寸就越大
  - 每个时间步都使用了相同的权重
- RNN缺点：
  - 循环计算速度慢
  - 在实践中，很难从多个步骤中获取信息（遗忘）

循环神经网络

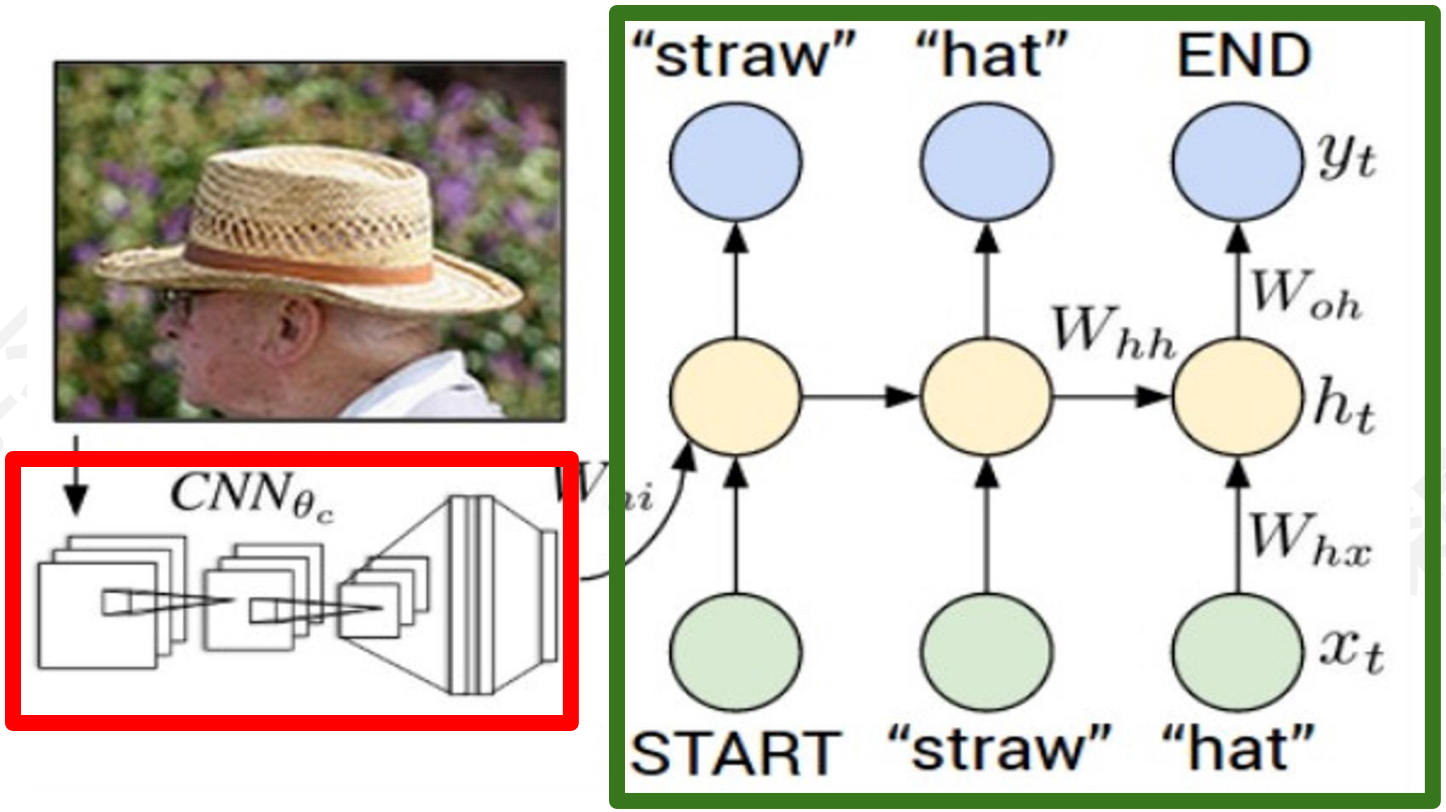
- 图像描述


"straw"
"hat"
END
y_t
W_{oh}
W_{hh}
h_t
W_{hx}
x_t
W_{hi}
CNN_{θ_c}
START
"straw"
"hat"

循环神经网络

- **图像描述**

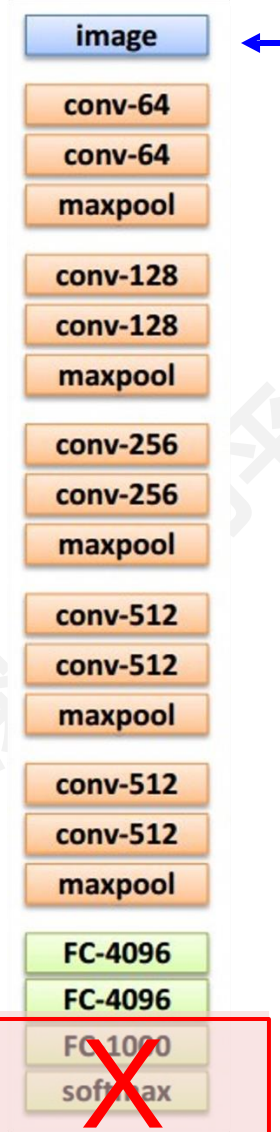
Recurrent Neural Network



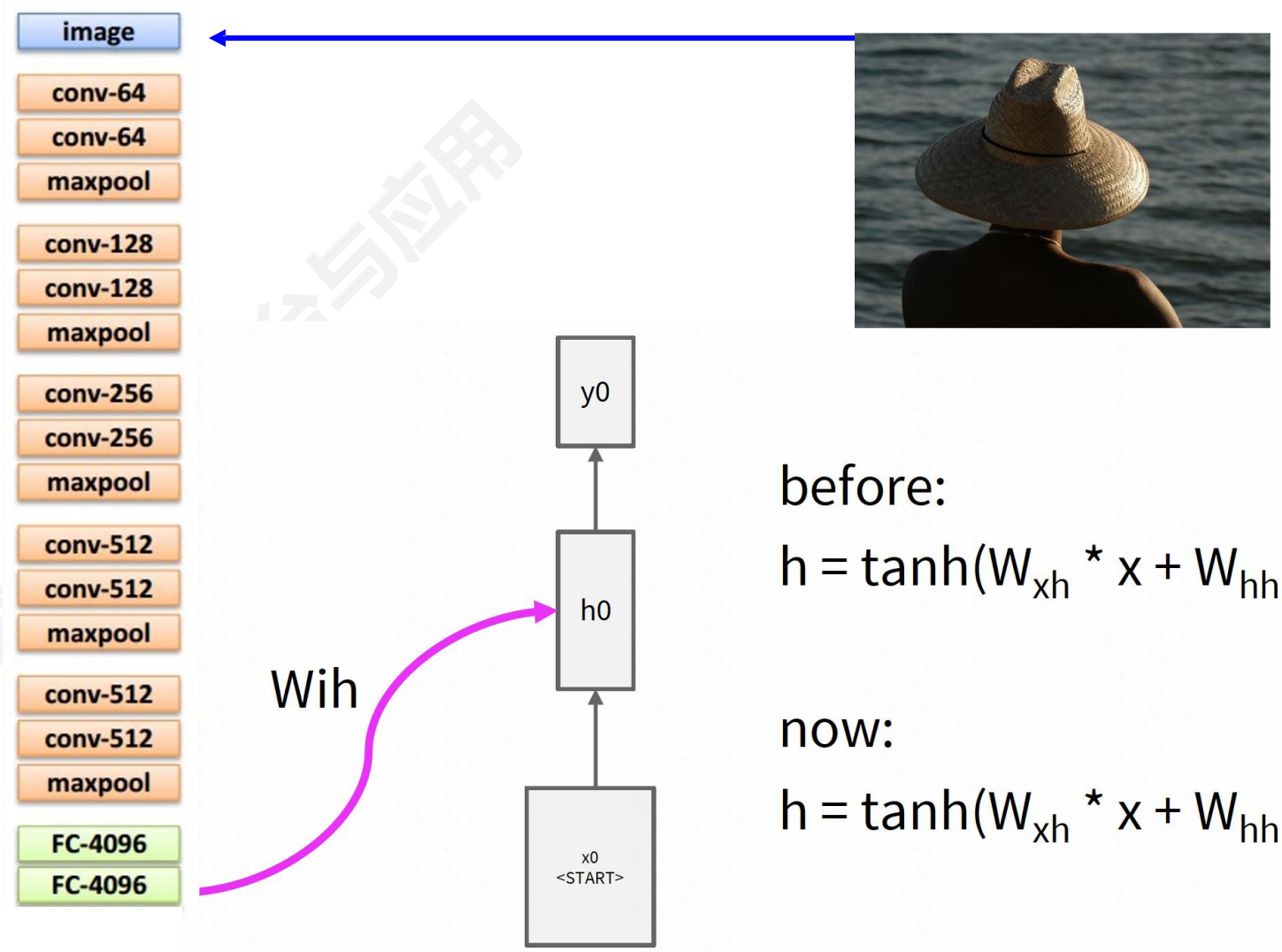
Convolutional Neural Network

循环神经网络

循环神经网络



循环神经网络



before:

$h = \tanh(W_{xh} * x + W_{hh} * h)$

now:

$h = \tanh(W_{xh} * x + W_{hh} * h + W_{ih} * v)$

循环神经网络

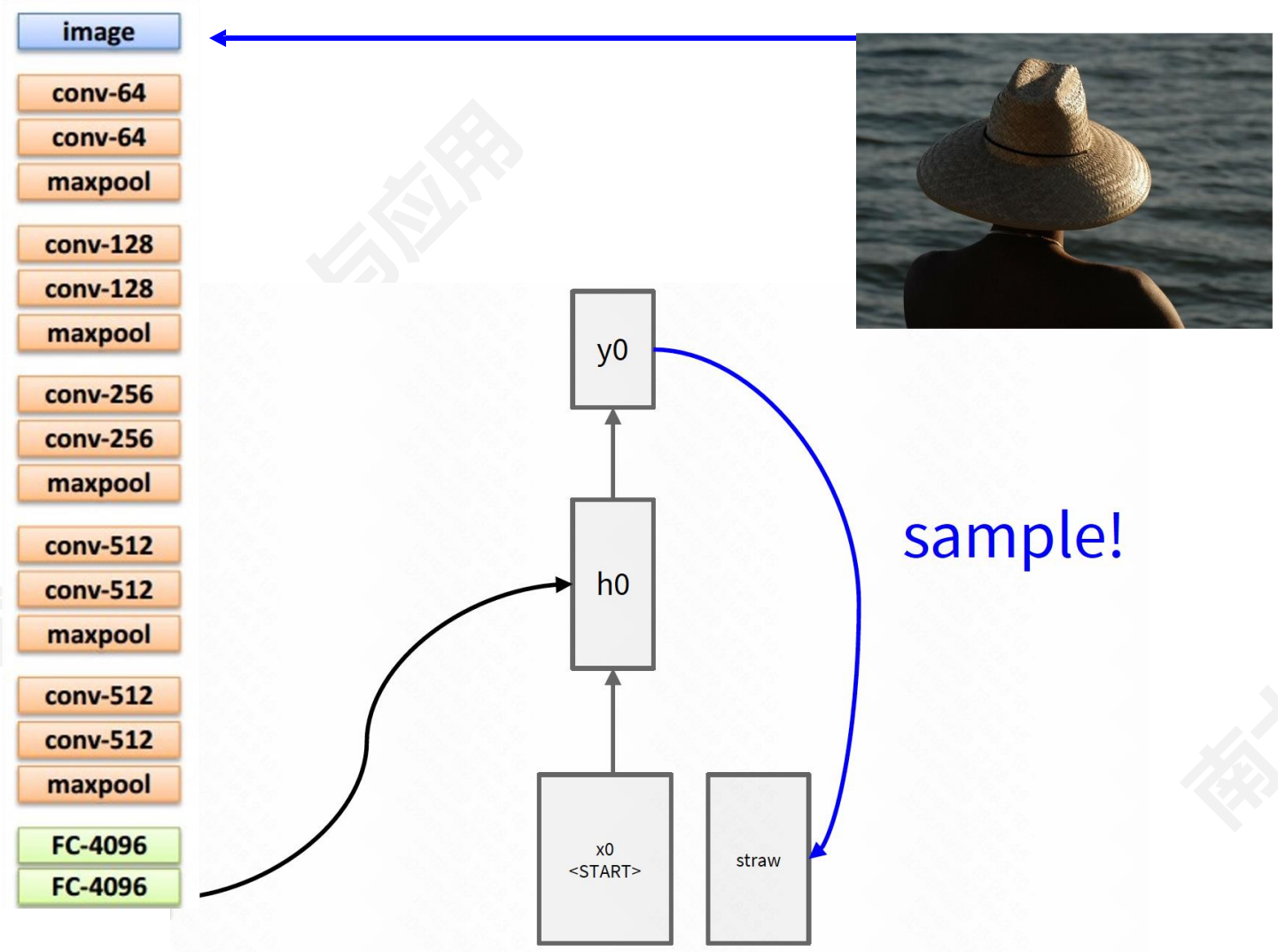


image
conv-64
conv-64
maxpool
conv-128
conv-128
maxpool
conv-256
conv-256
maxpool
conv-512
conv-512
maxpool
conv-512
conv-512
maxpool
FC-4096
FC-4096
y0
h0
x0
<START>
straw
sample!

循环神经网络

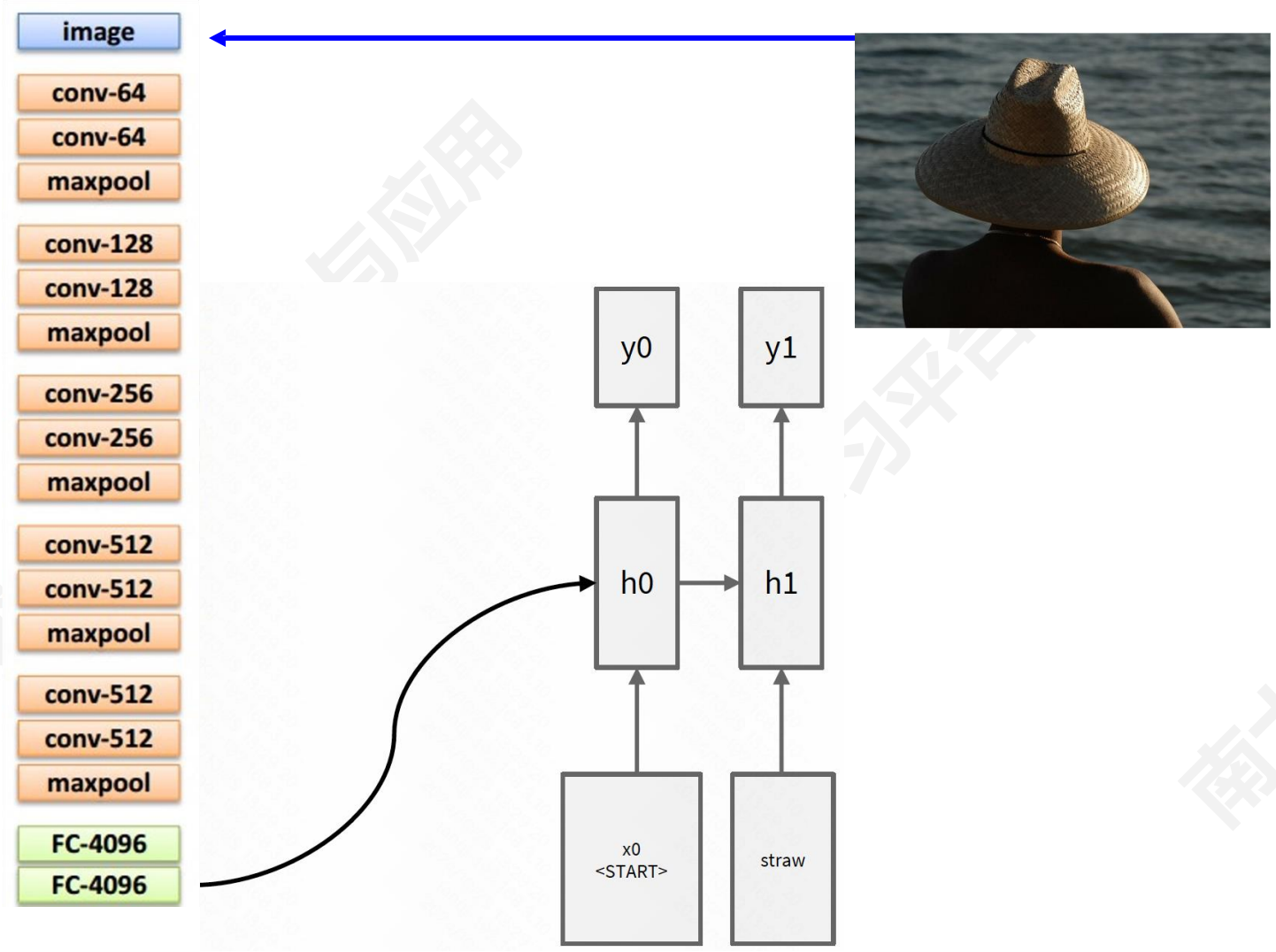


image
conv-64
conv-64
maxpool
conv-128
conv-128
maxpool
conv-256
conv-256
maxpool
conv-512
conv-512
maxpool
conv-512
conv-512
maxpool
FC-4096
FC-4096
y0
y1
h0
h1
x0
<START>
straw

循环神经网络

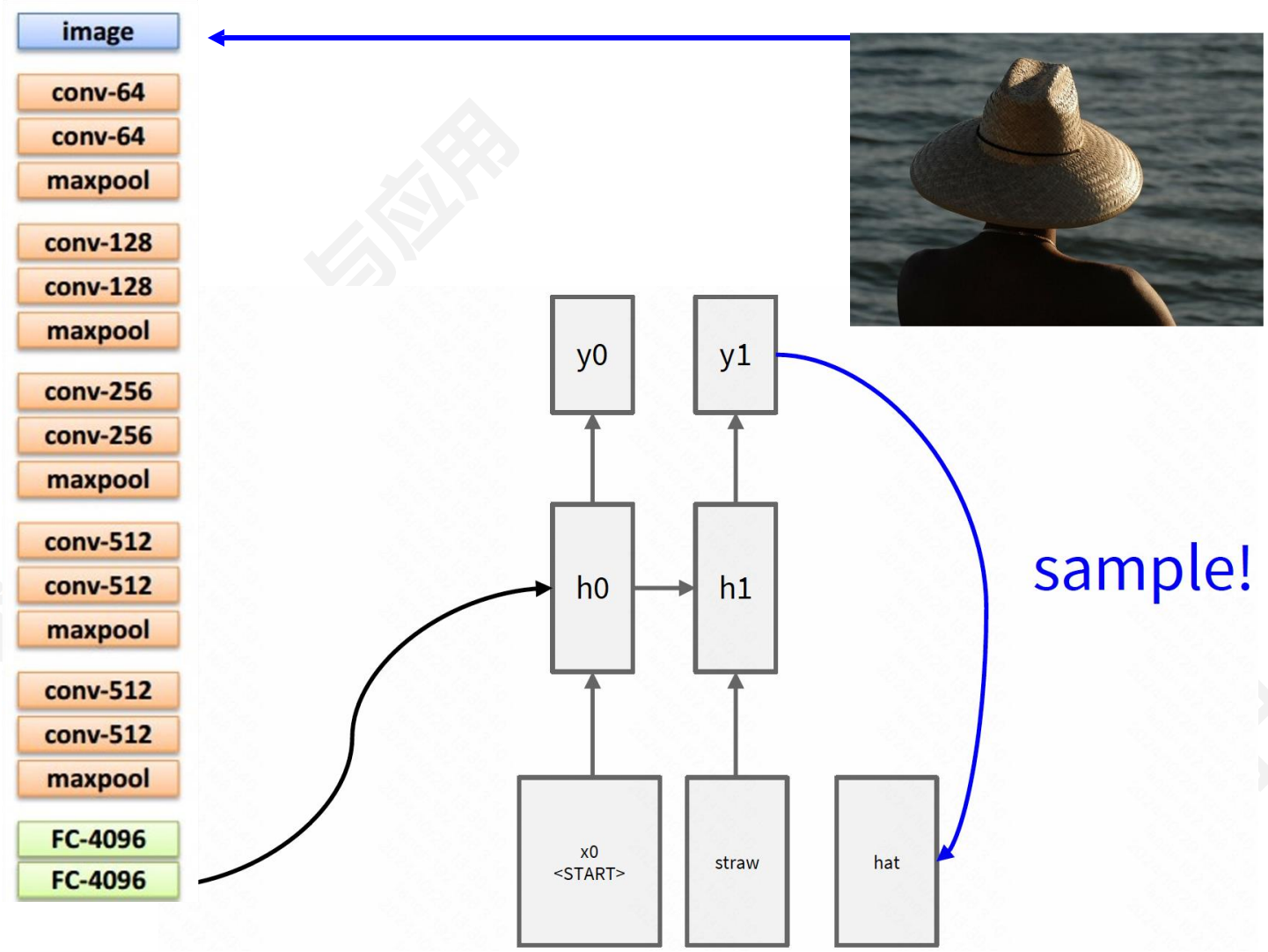


image
conv-64
conv-64
maxpool
conv-128
conv-128
maxpool
conv-256
conv-256
maxpool
conv-512
conv-512
maxpool
conv-512
conv-512
maxpool
FC-4096
FC-4096
y0
y1
h0
h1
x0
<START>
straw
hat
sample!

循环神经网络

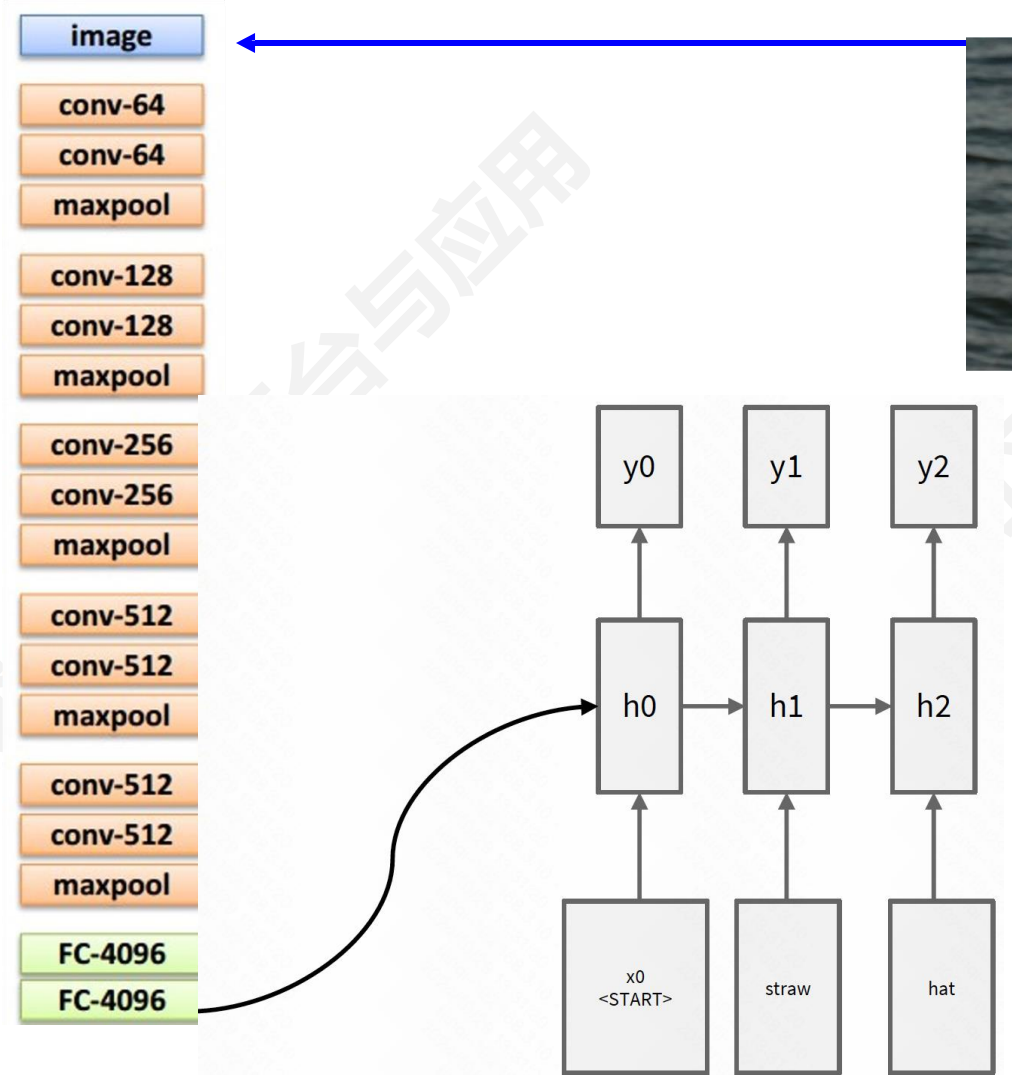


image
conv-64
conv-64
maxpool
conv-128
conv-128
maxpool
conv-256
conv-256
maxpool
conv-512
conv-512
maxpool
conv-512
conv-512
maxpool
FC-4096
FC-4096
y0
y1
y2
h0
h1
h2
x0
<START>
straw
hat

循环神经网络

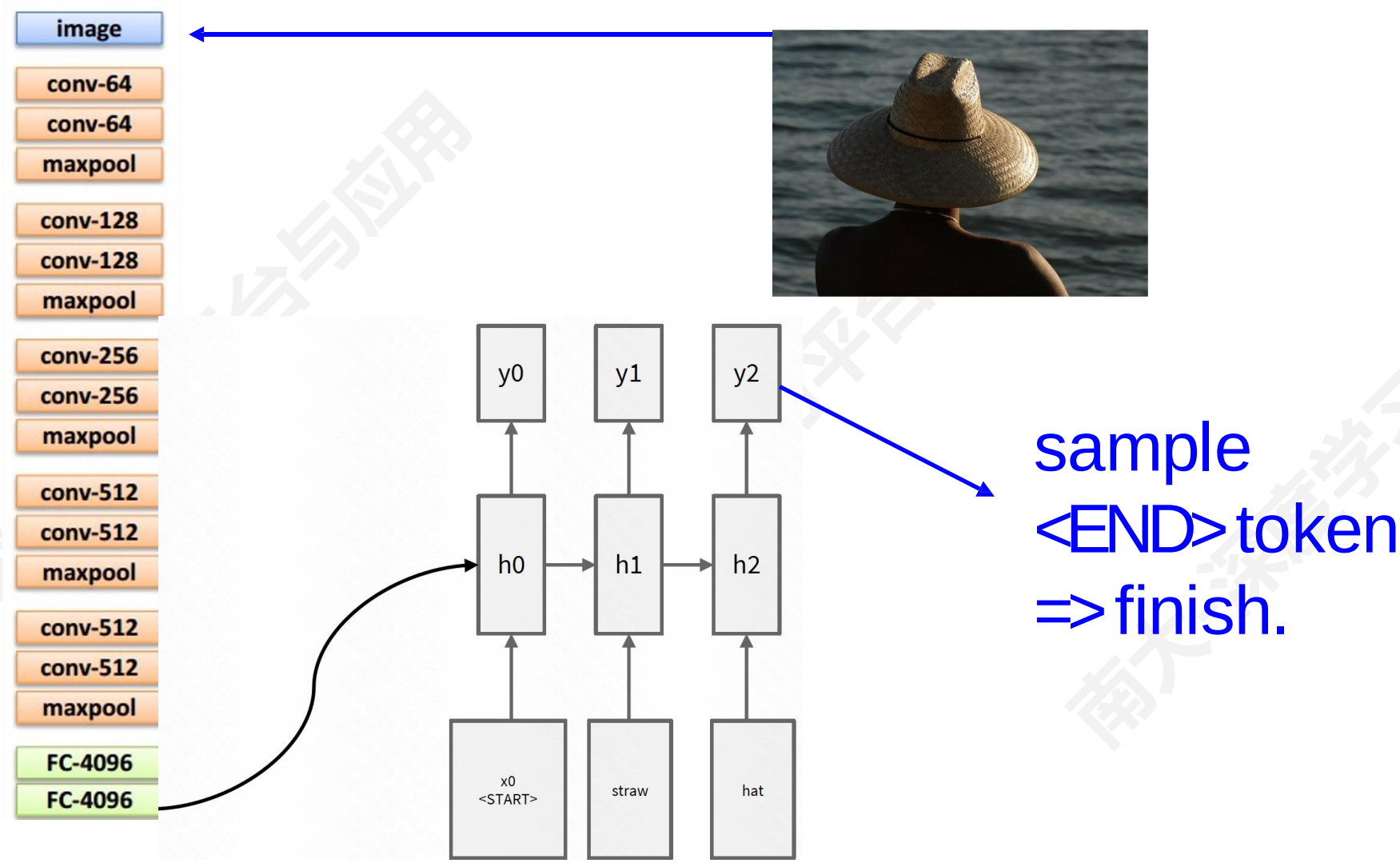


image
conv-64
conv-64
maxpool
conv-128
conv-128
maxpool
conv-256
conv-256
maxpool
conv-512
conv-512
maxpool
conv-512
conv-512
maxpool
FC-4096
FC-4096
y0
y1
y2
h0
h1
h2
x0
<START>
straw
hat
sample
<END> token
=> finish.

循环神经网络

A cat sitting on a suitcase on the floor

A cat is sitting on a tree branch

A dog is running in the grass with a frisbee

A white teddy bear sitting in the grass

Two people walking on the beach with surfboards

A tennis player in action on the court

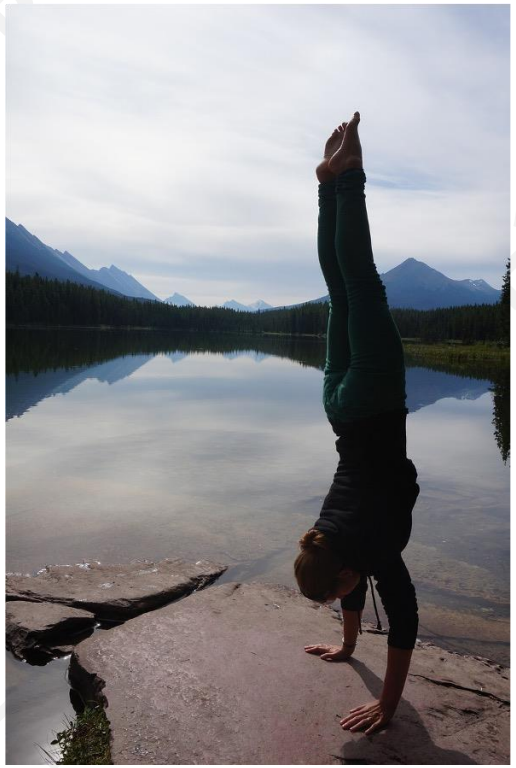
Two giraffes standing in a grassy field

A man riding a dirt bike on a dirt track

循环神经网络

A woman is holding a cat in her hand

A person holding a computer mouse on a desk



A woman standing on a beach holding a surfboard

A bird is perched on a tree branch

A man in a baseball uniform throwing a ball

Visual Question Answering (VQA)

Q: What endangered animal is featured on the truck?

A: A bald eagle.
A: A sparrow.
A: A humming bird.
A: A raven.

Q: Where will the driver go if turning right?

A: Onto 24 ¾ Rd.
A: Onto 25 ¾ Rd.
A: Onto 23 ¾ Rd.
A: Onto Main Street.

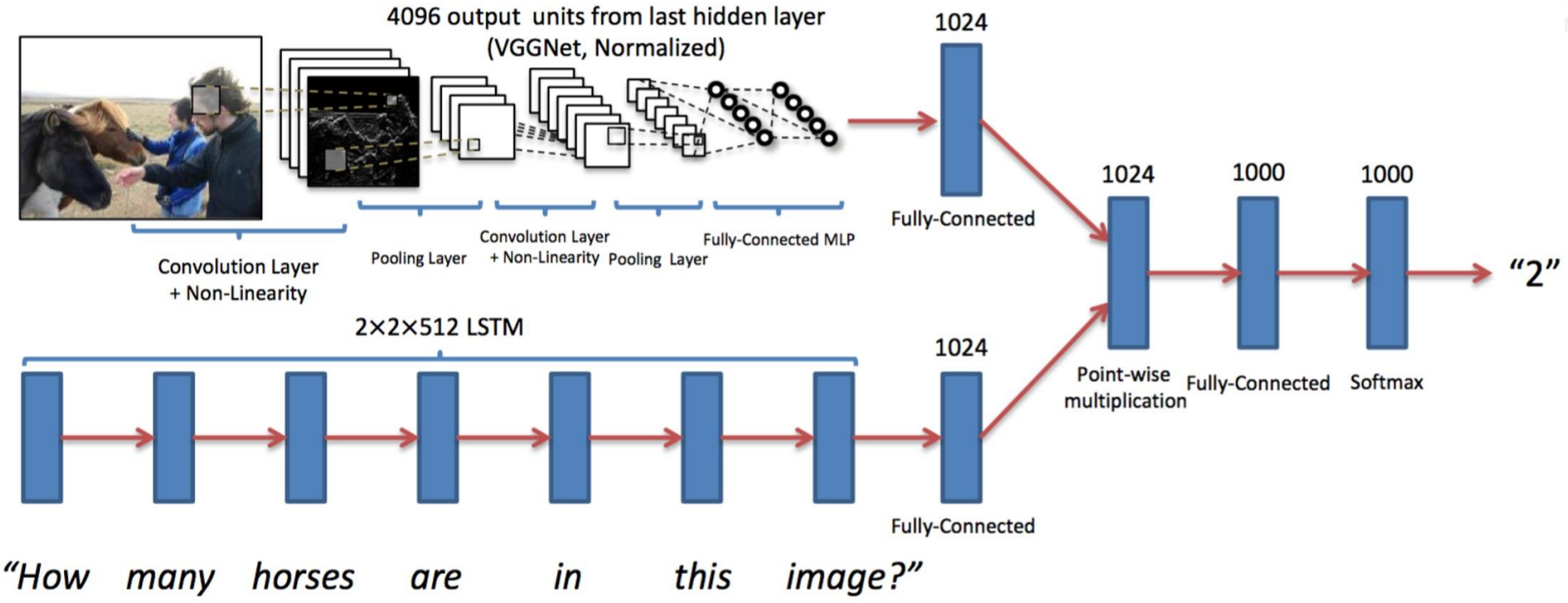
Q: When was the picture taken?

A: During a wedding.
A: During a bar mitzvah.
A: During a funeral.
A: During a Sunday church service

Q: Who is under the umbrella?

A: Two women.
A: A child.
A: An old man.
A: A husband and a wife.

Visual Question Answering (VQA)



循环神经网络

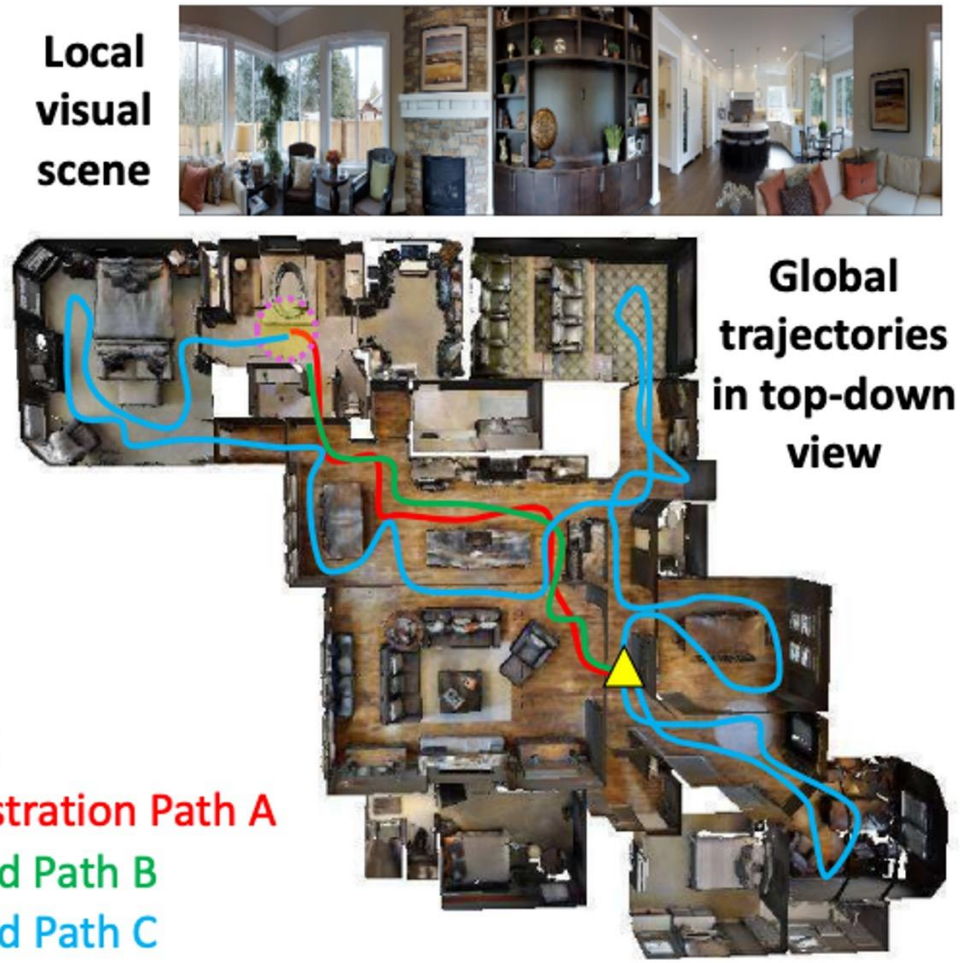
- Visual Dialog
- 基于图像的对话

循环神经网络

Visual Language Navigation

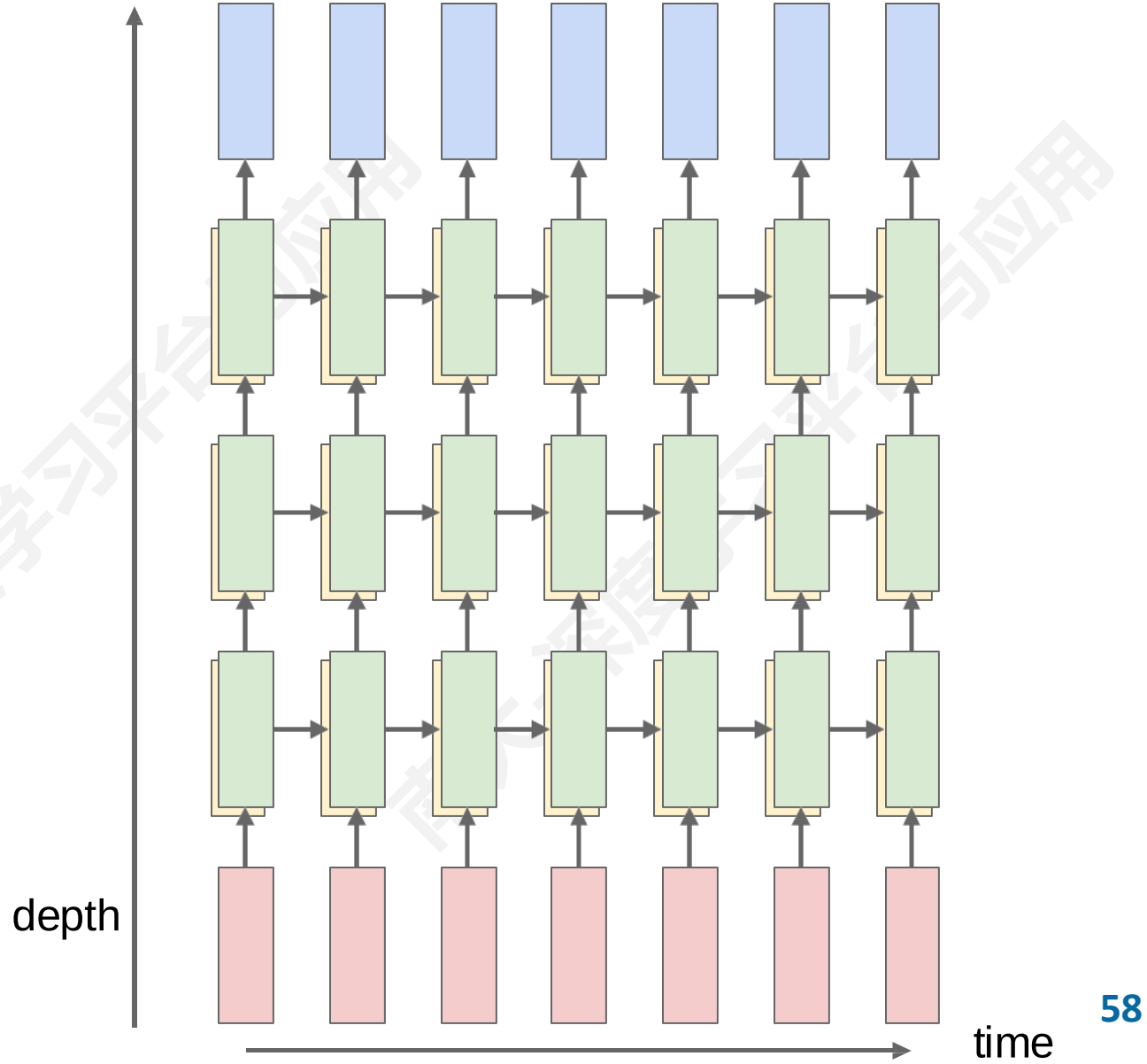
Instruction

Turn right and head towards the *kitchen*. Then turn left, pass a *table* and enter the *hallway*. Walk down the hallway and turn into the *entry way* to your right *without doors*. Stop in front of the *toilet*.



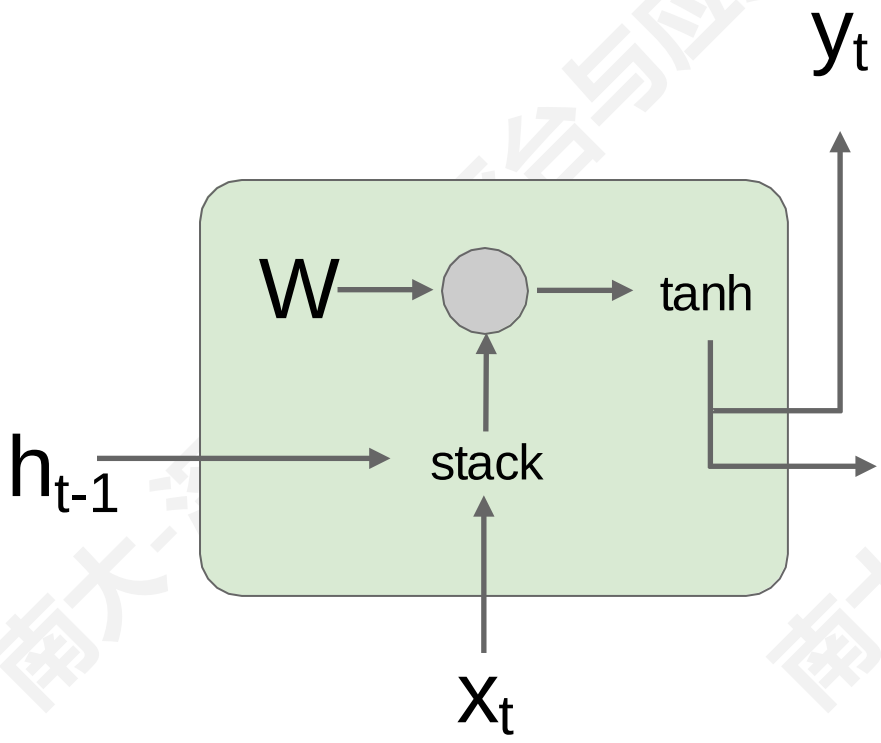
循环神经网络

- 多层 RNN



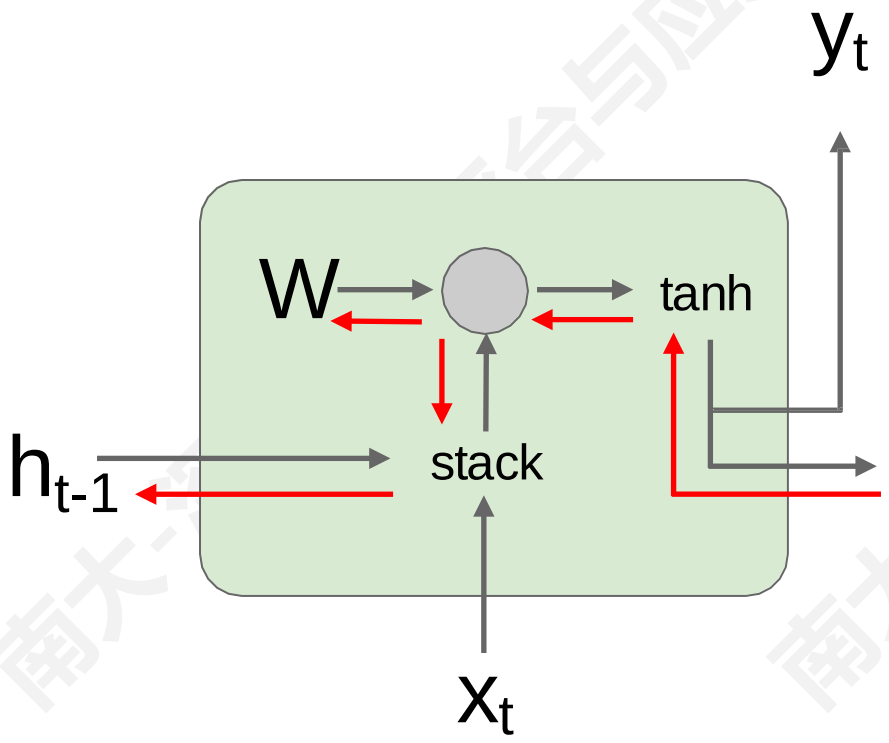
循环神经网络

- **RNN 的梯度计算**



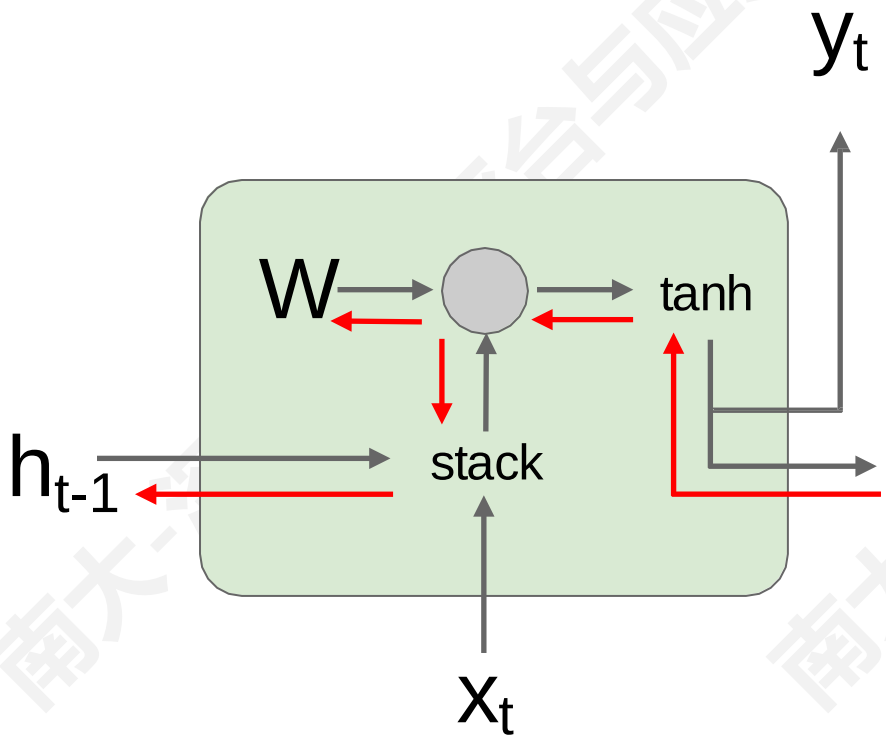
$$
\begin{aligned}
h_t &= \tanh(W_{hh}h_{t-1} + W_{xh}x_t) \\
&= \tanh\left((W_{hh} \quad W_{hx})\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\
&= \tanh\left(W\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)
\end{aligned}
$$

RNN 的梯度计算



$$
\begin{aligned}
h_t &= \tanh(W_{hh}h_{t-1} + W_{xh}x_t) \\
&= \tanh\left(\begin{pmatrix} W_{hh} & W_{hx} \end{pmatrix}\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right) \\
&= \tanh\left(W\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)
\end{aligned}
$$

RNN 的梯度计算



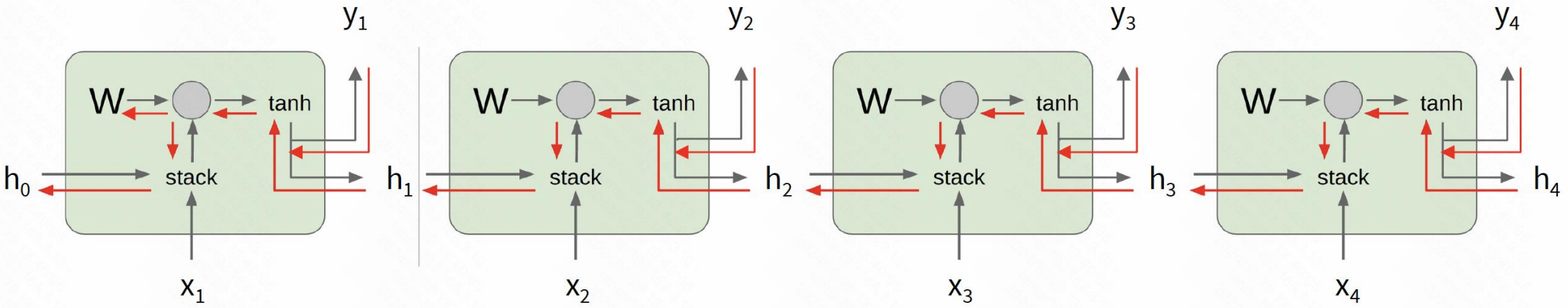
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$= \tanh\left((W_{hh} \quad W_{hx})\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$

$$= \tanh\left(W\begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$

$$\frac{\partial h_t}{\partial h_{t-1}} = tanh'(W_{hh}h_{t-1} + W_{xh}x_t)W_{hh}$$

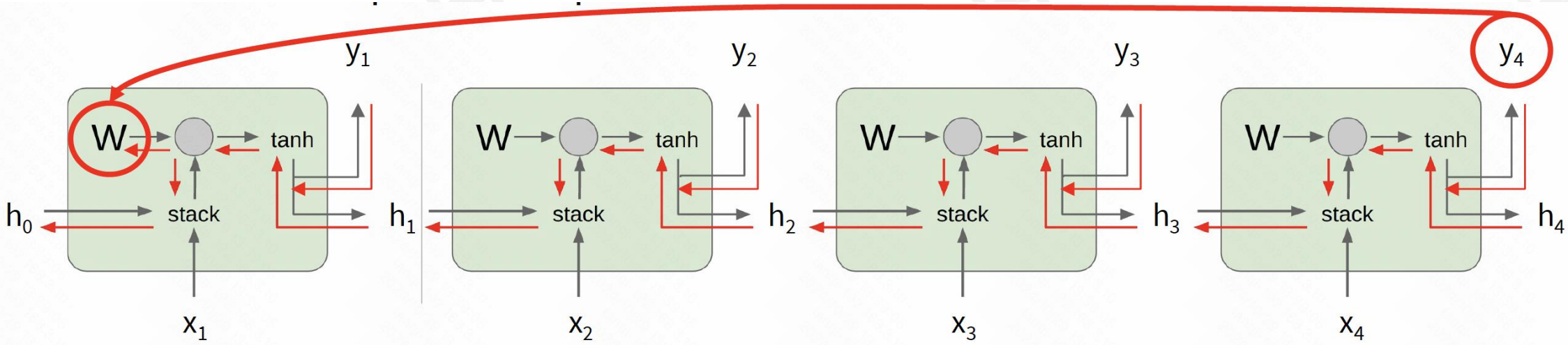
RNN 的梯度计算（多个时间步）



$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

循环神经网络

RNN 的梯度计算（多个时间步）

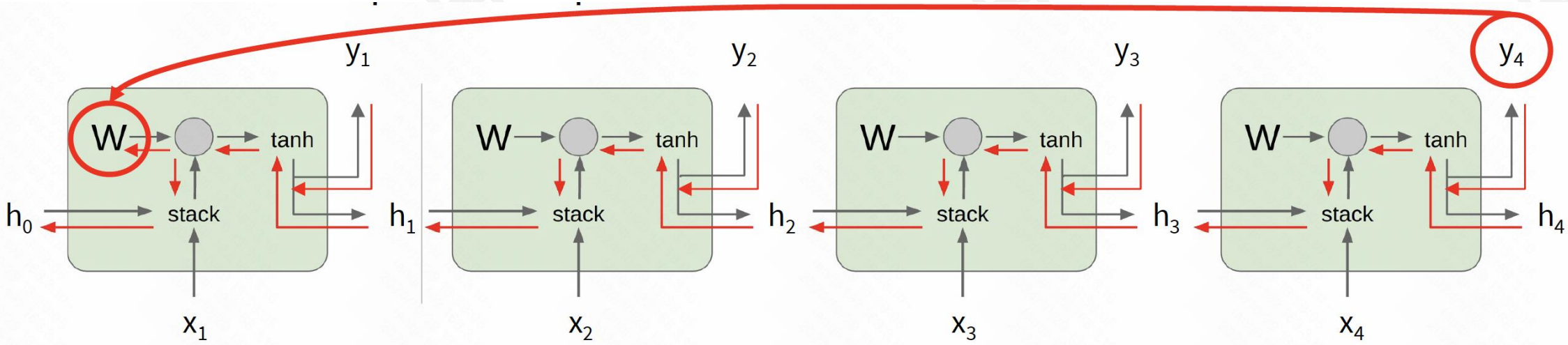


$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W}$$

循环神经网络

- RNN 的梯度计算（多个时间步）

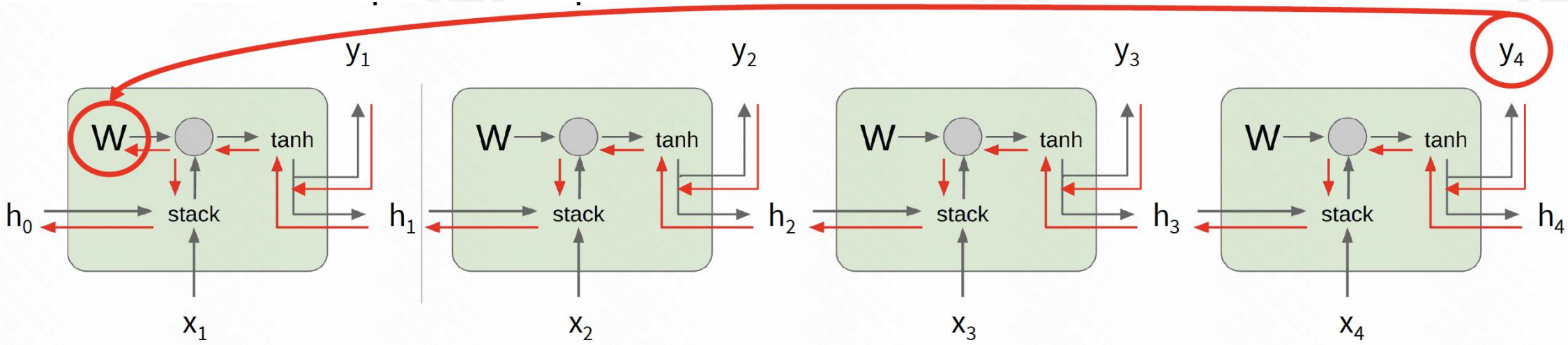


$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^{T} \boxed{\frac{\partial h_t}{\partial h_{t-1}}} \right) \frac{\partial h_1}{\partial W}$$

循环神经网络

RNN 的梯度计算（多个时间步）



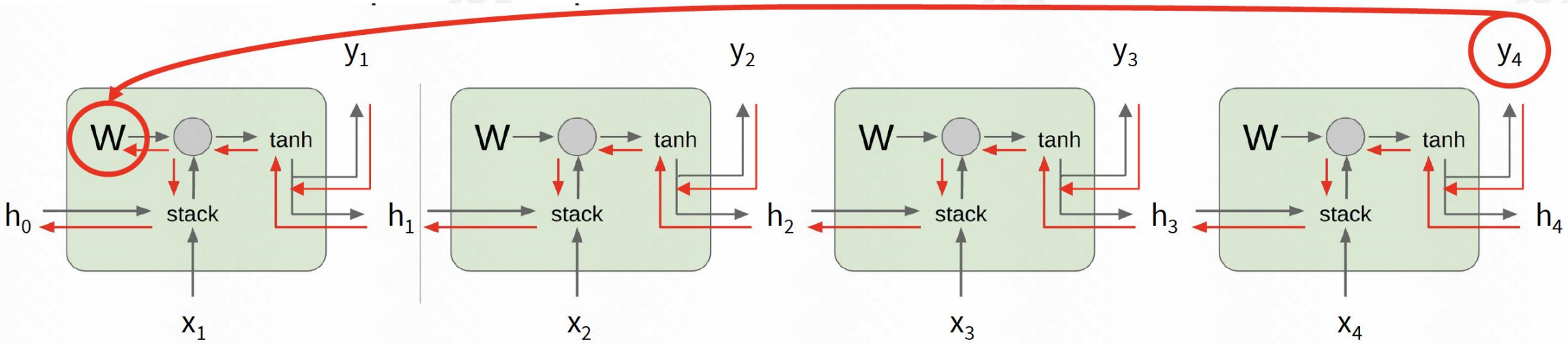
$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

$$\frac{\partial h_t}{\partial h_{t-1}} = tanh'(W_{hh} h_{t-1} + W_{xh} x_t) W_{hh}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \frac{\partial h_t}{\partial h_{t-1}} \cdots \frac{\partial h_1}{\partial W} = \frac{\partial L_T}{\partial h_T} \left(\prod_{t=2}^{T} \frac{\partial h_t}{\partial h_{t-1}} \right) \frac{\partial h_1}{\partial W}$$

循环神经网络

- RNN 的梯度计算（多个时间步）

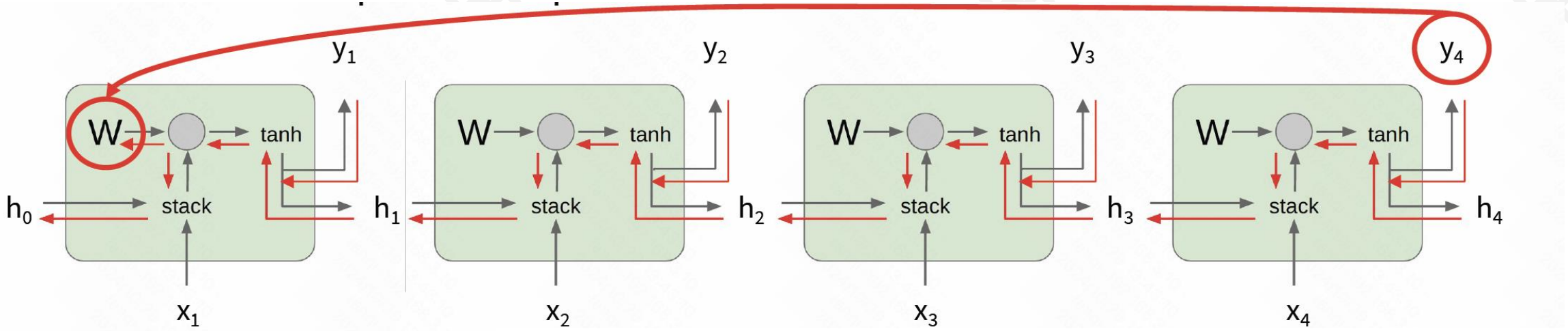


$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

总是小于1
梯度消失

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T}\left(\prod_{t=2}^{T} tanh'\left(W_{hh}h_{t-1} + W_{xh}x_t\right)\right)W_{hh}^{T-1}\frac{\partial h_1}{\partial W}$$

RNN 的梯度计算（多个时间步）

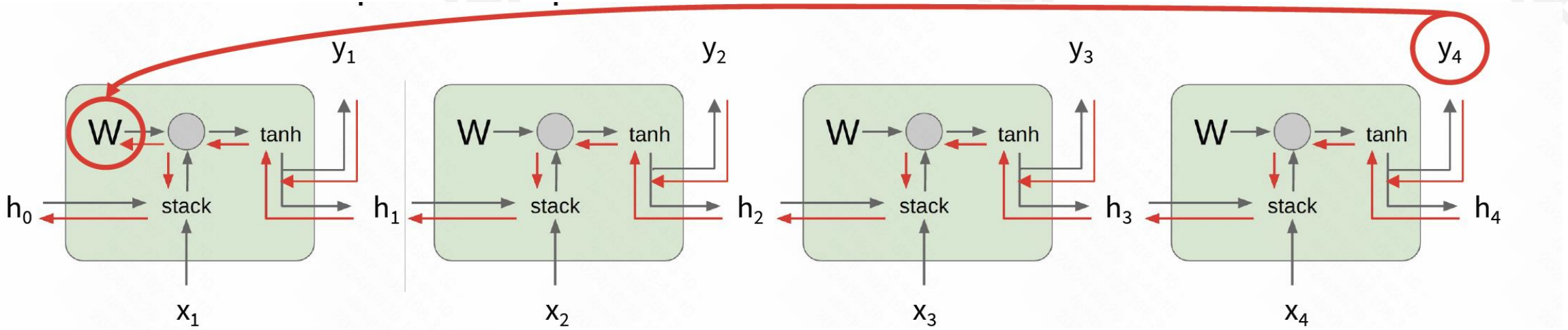


$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

去除非线性激活函数 tanh?

循环神经网络

- RNN 的梯度计算（多个时间步）



$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \boxed{W_{hh}^{T-1}} \frac{\partial h_1}{\partial W}$$

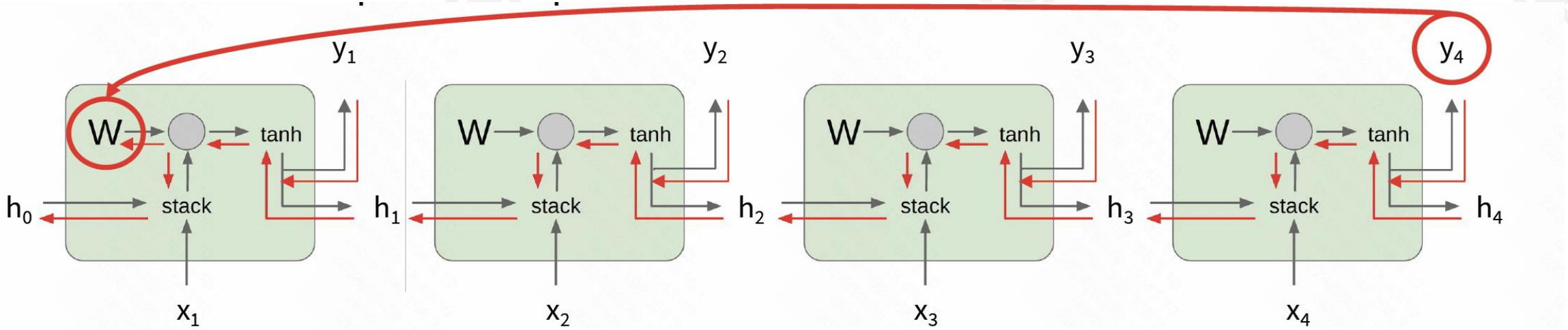
去除非线性激活函数 tanh?

最大奇异值大于1：梯度爆炸→梯度裁剪

```
grad_norm = np.sum(grad * grad)
if grad_norm > threshold:
  grad *= (threshold / grad_norm)
```

循环神经网络

RNN 的梯度计算（多个时间步）



$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L_t}{\partial W}$$

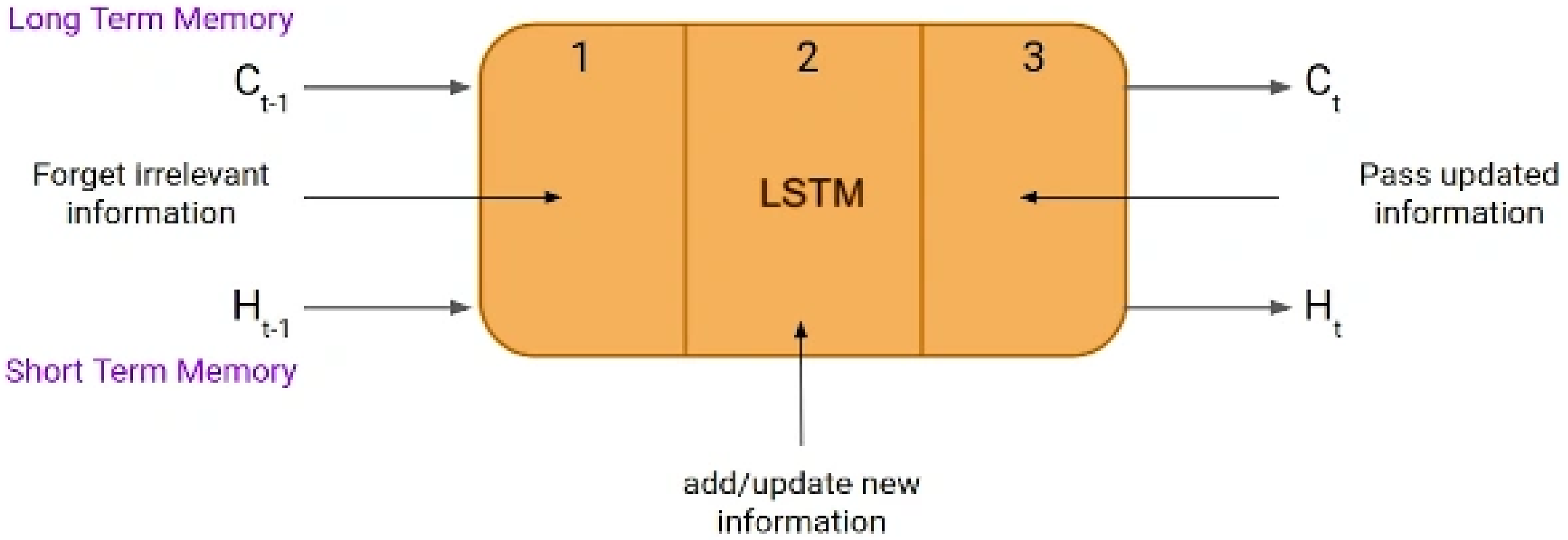
$$\frac{\partial L_T}{\partial W} = \frac{\partial L_T}{\partial h_T} \boxed{W_{hh}^{T-1}} \frac{\partial h_1}{\partial W}$$

去除非线性激活函数 tanh?

最大奇异值大于1：梯度爆炸→梯度裁剪

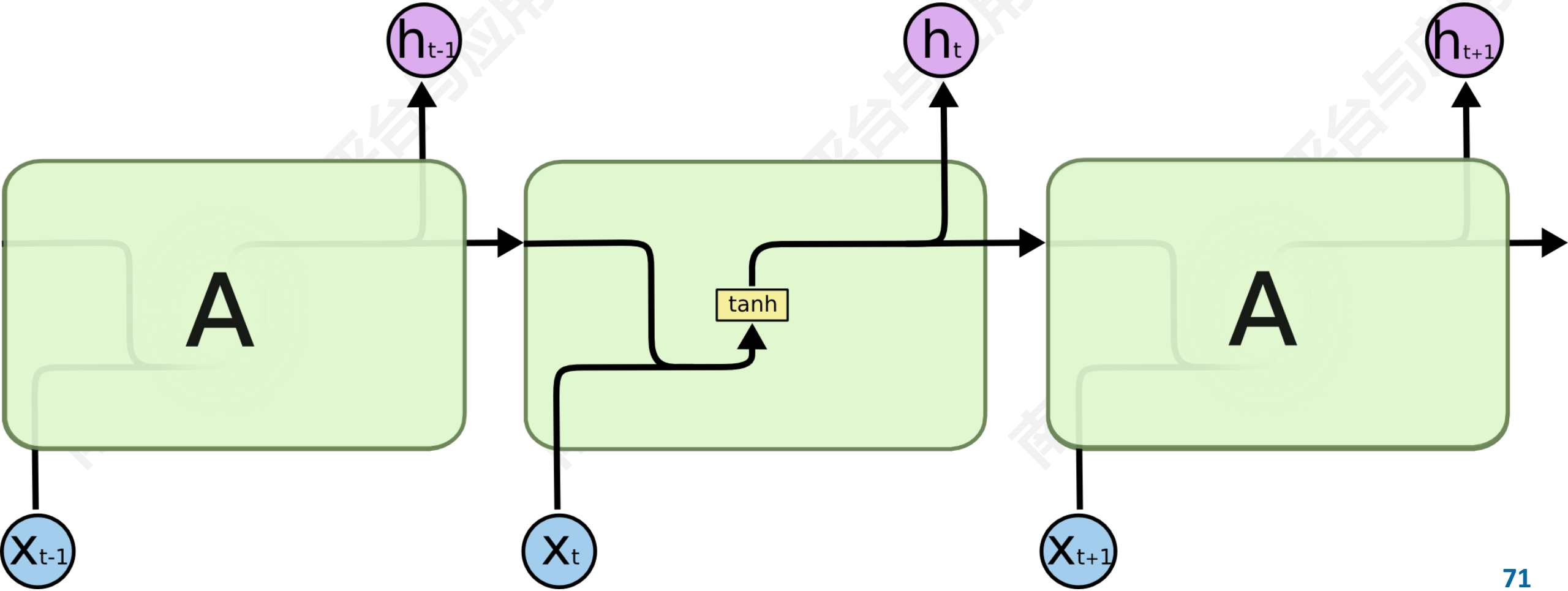
最大奇异值小于1：梯度消失→改进RNN结构

Long Short Term Memory (LSTM)

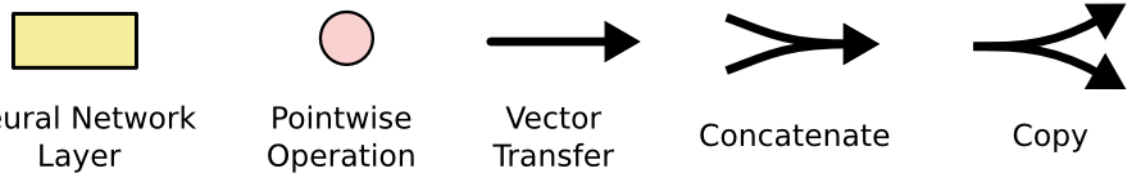


循环神经网络

- Long Short Term Memory (LSTM) 梯度计算

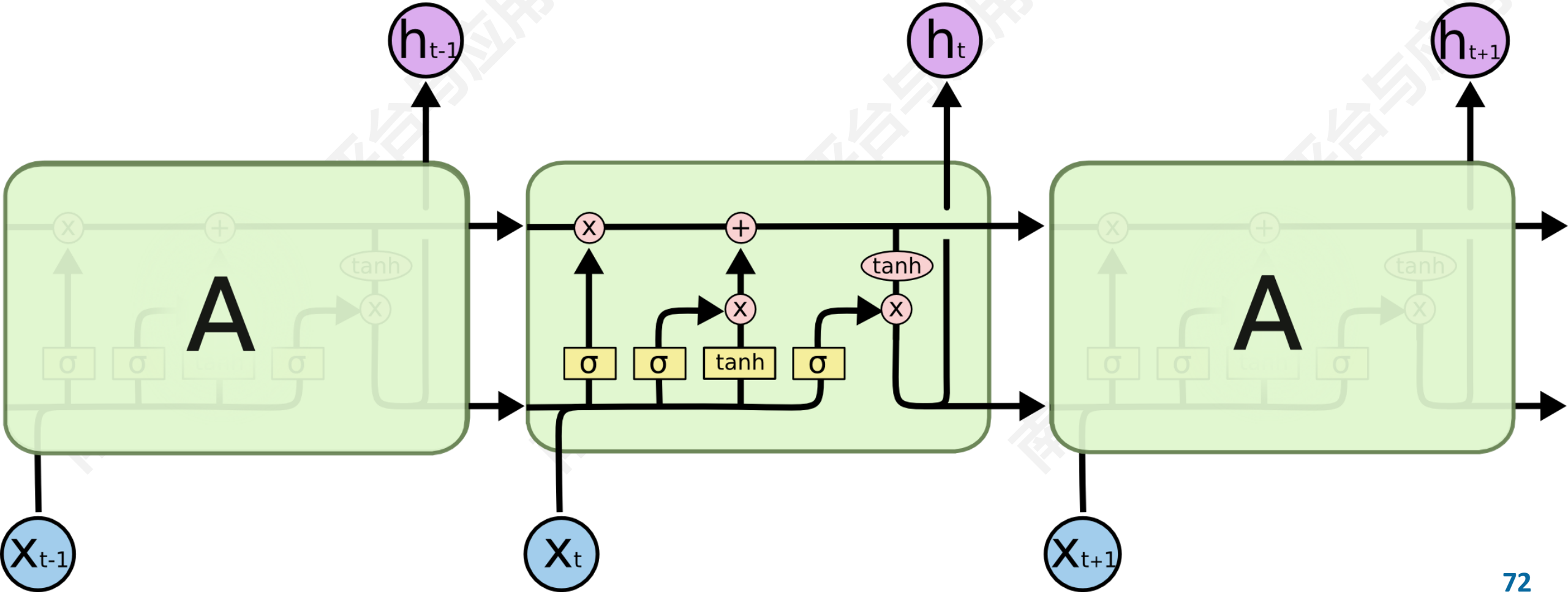


循环神经网络

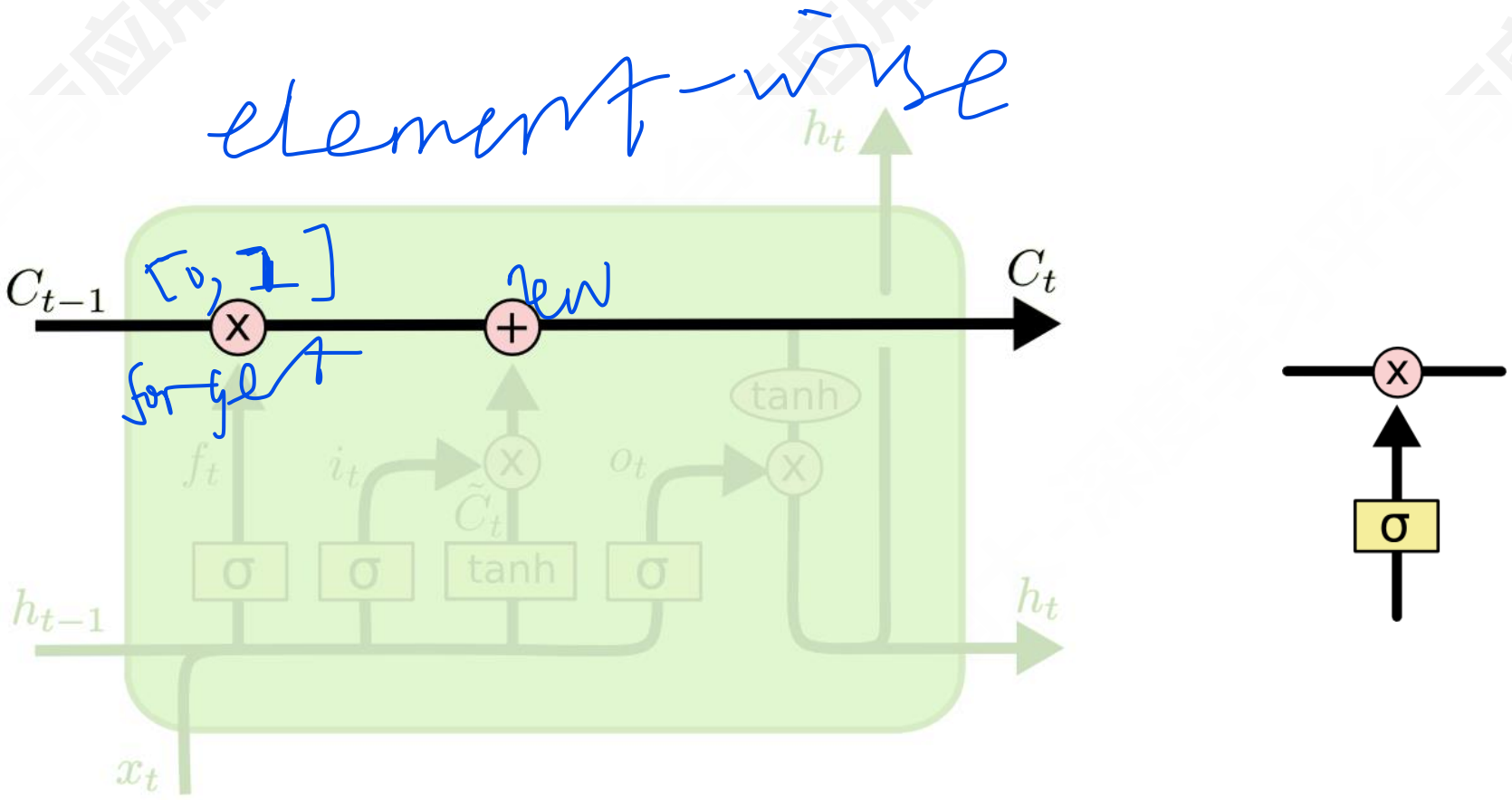


南京大学 NANJING UNIVERSITY

- Long Short Term Memory (LSTM)

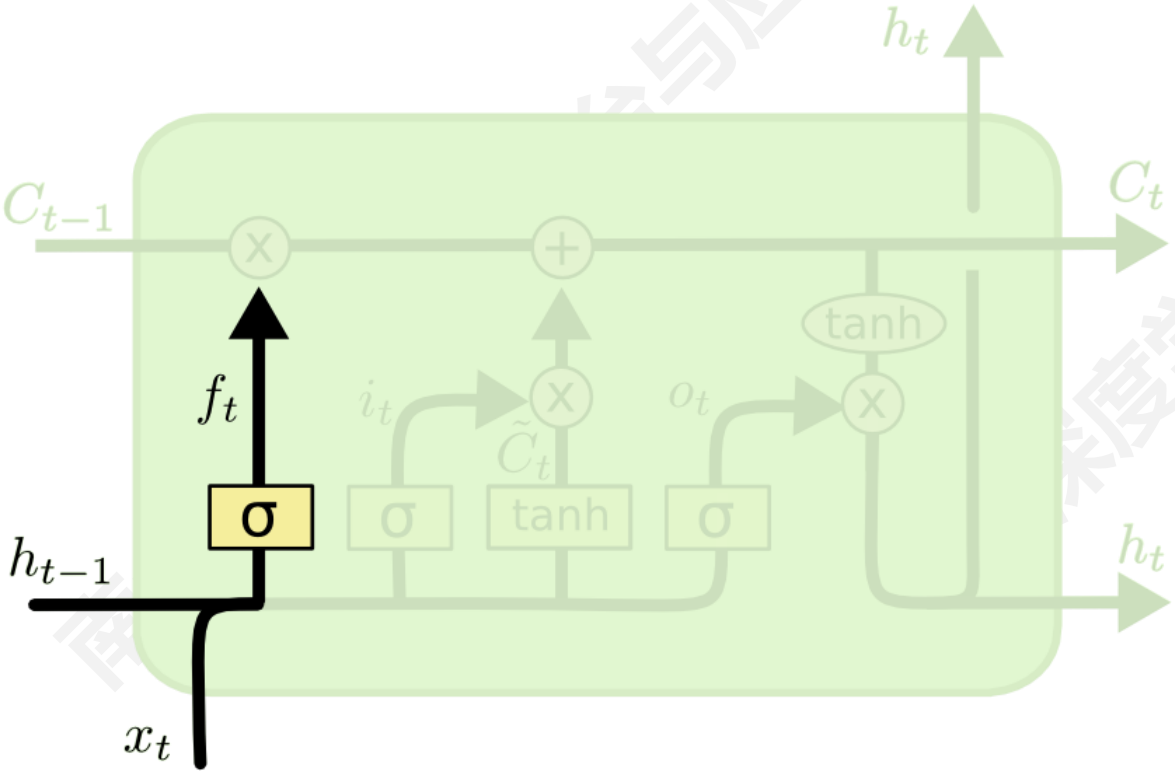


- Long Short Term Memory (LSTM)



循环神经网络

- Long Short Term Memory (LSTM)

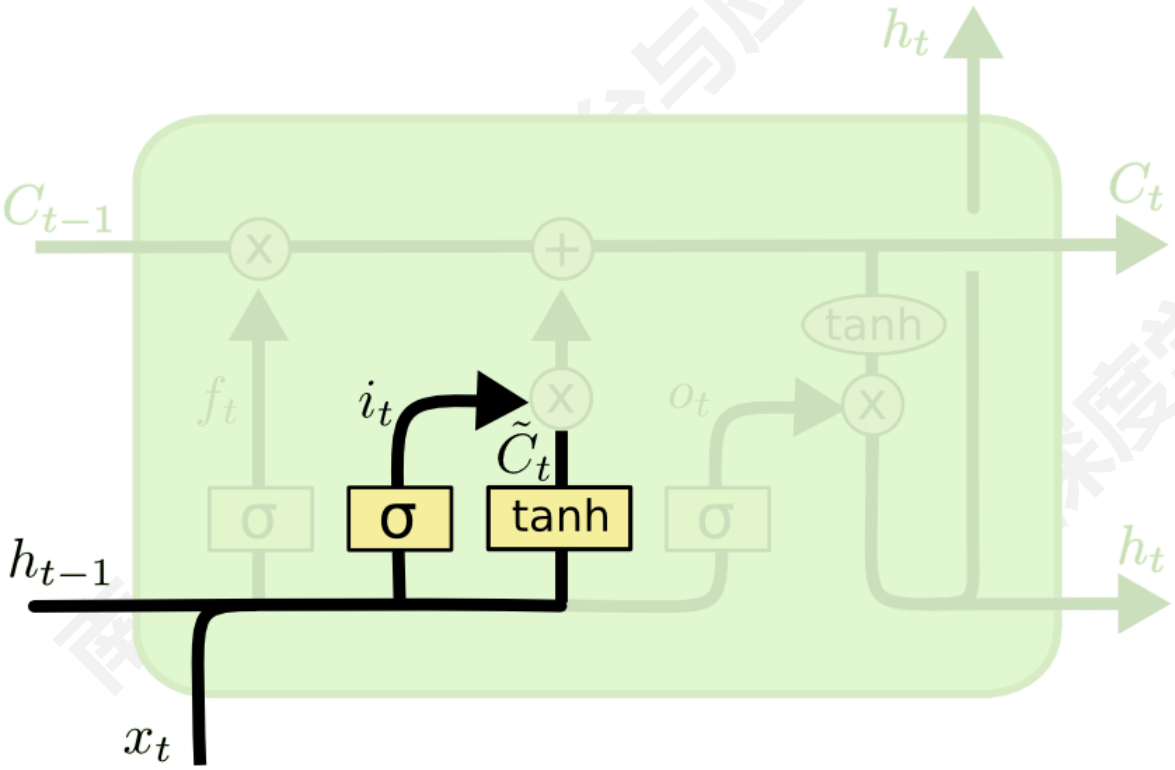


sigmoid

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

循环神经网络

- **Long Short Term Memory (LSTM)**



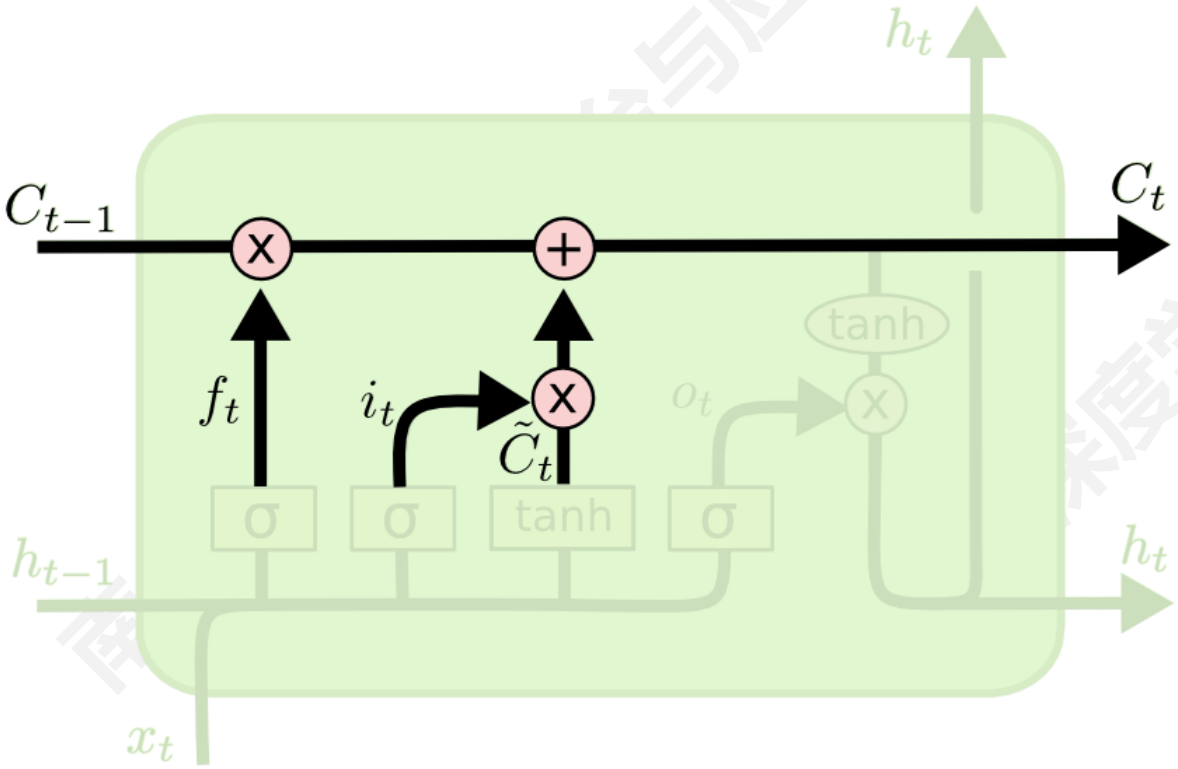
sigmoid

$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] + b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

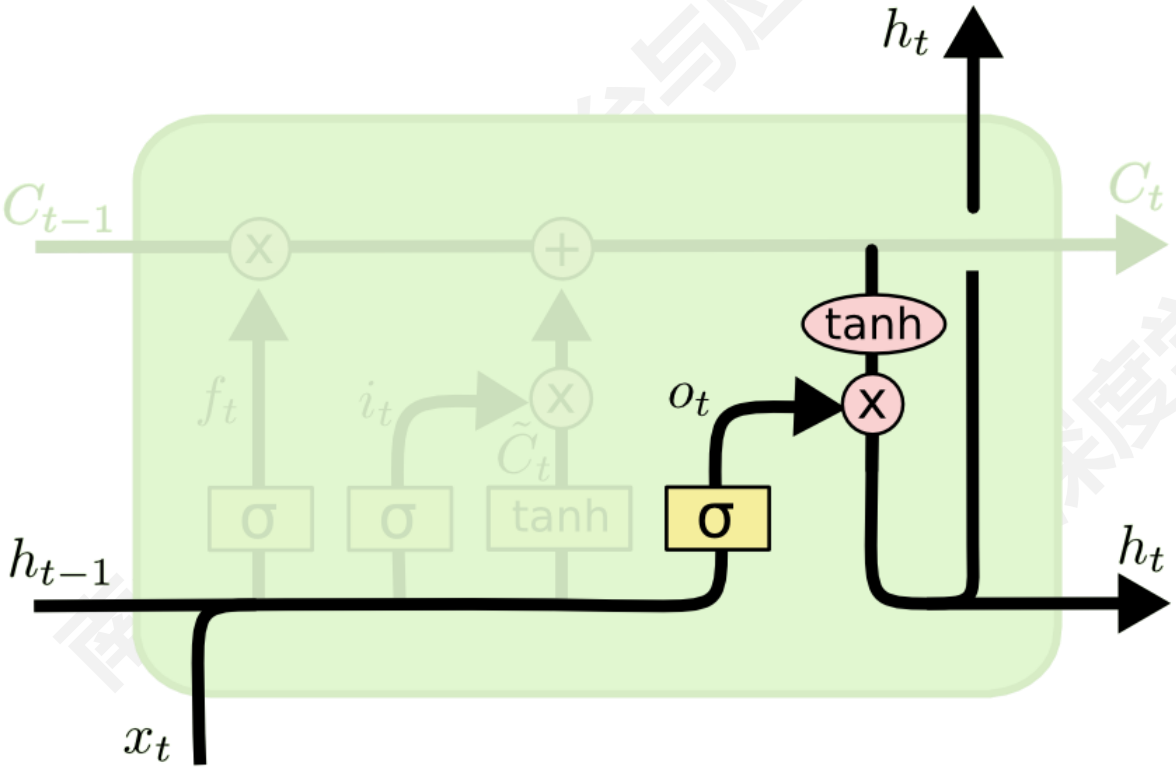
循环神经网络

- Long Short Term Memory (LSTM)



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

- Long Short Term Memory (LSTM)



$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

Long Short Term Memory (LSTM)

Vanilla RNN

$$h_t = \tanh\left(W\begin{pmatrix}h_{t-1}\\x_t\end{pmatrix}\right)$$

LSTM

$$\begin{pmatrix}i\\f\\o\\g\end{pmatrix} = \begin{pmatrix}\sigma\\\sigma\\\sigma\\\tanh\end{pmatrix} W\begin{pmatrix}h_{t-1}\\x_t\end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)

Vanilla RNN

$$h_t = \tanh\left(W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}\right)$$

LSTM

Four gates

$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

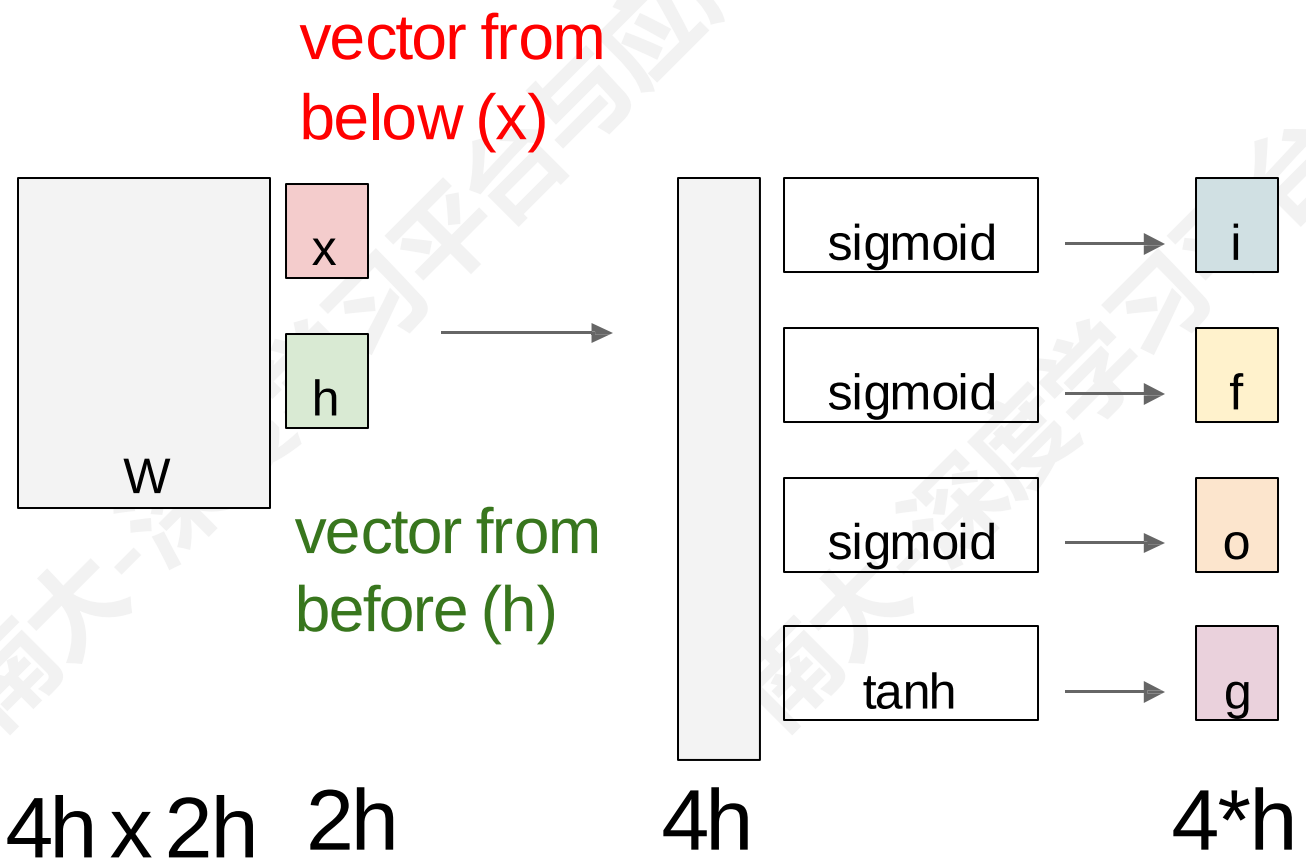
Cell state

$$c_t = f \odot c_{t-1} + i \odot g$$

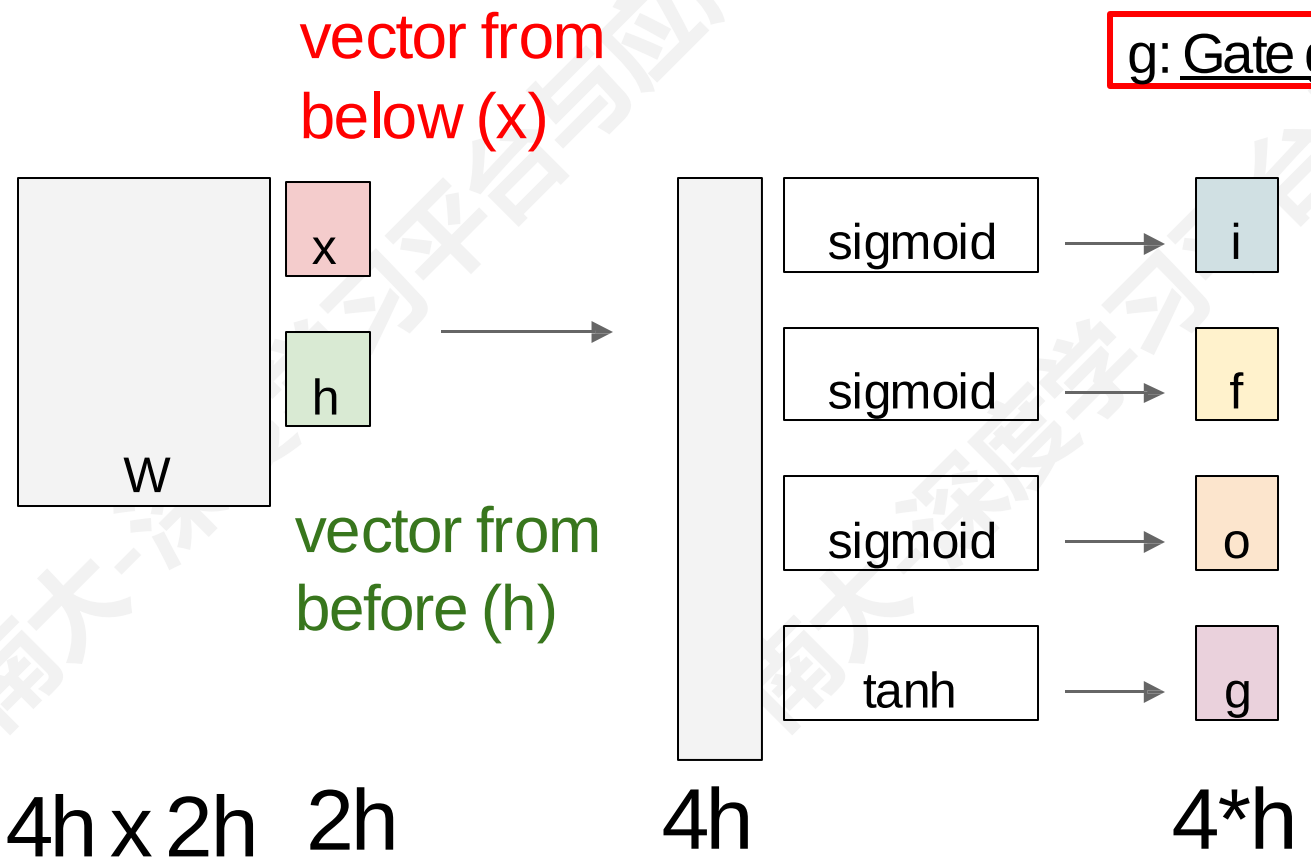
Hidden state

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM)



Long Short Term Memory (LSTM)



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

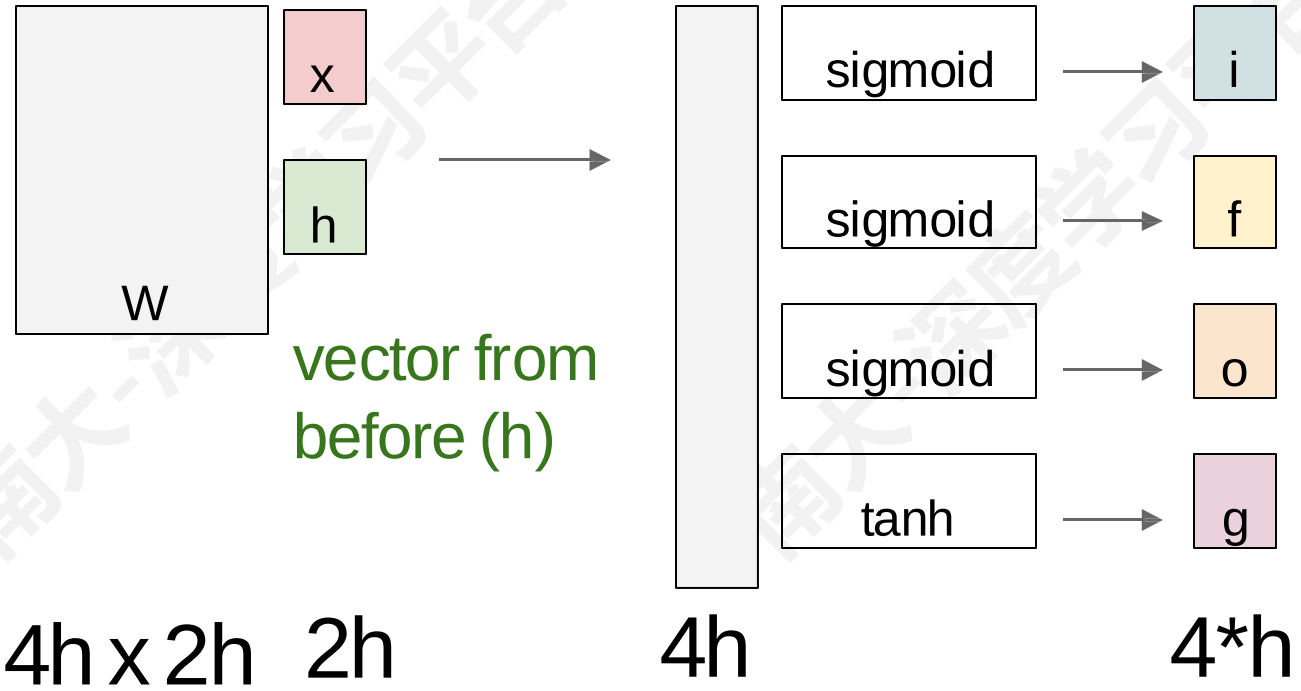
$$h_t = o \odot \tanh(c_t)$$

- **LSTM**

i: Input gate, whether to write to cell

vector from below (x)

g: Gate gate (?), How much to write to cell



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

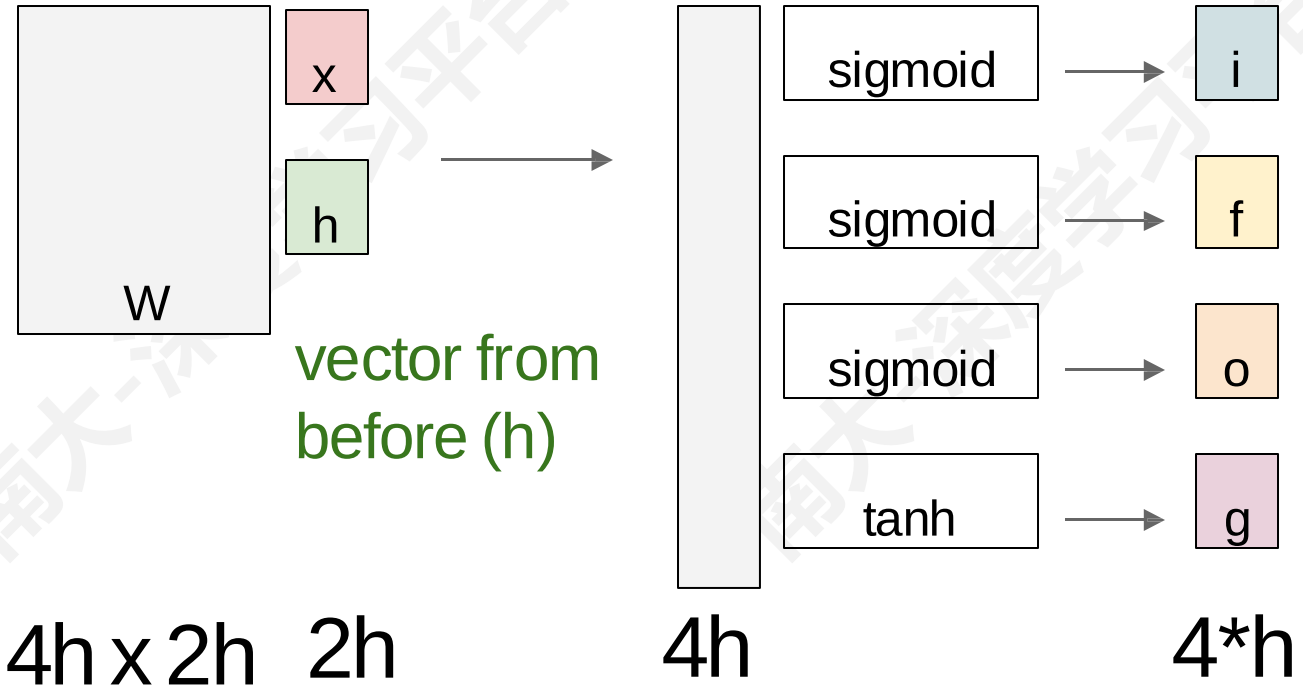
循环神经网络

- **LSTM**

i: Input gate, whether to write to cell

f: Forget gate, Whether to erase cell

g: Gate gate (?), How much to write to cell



$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

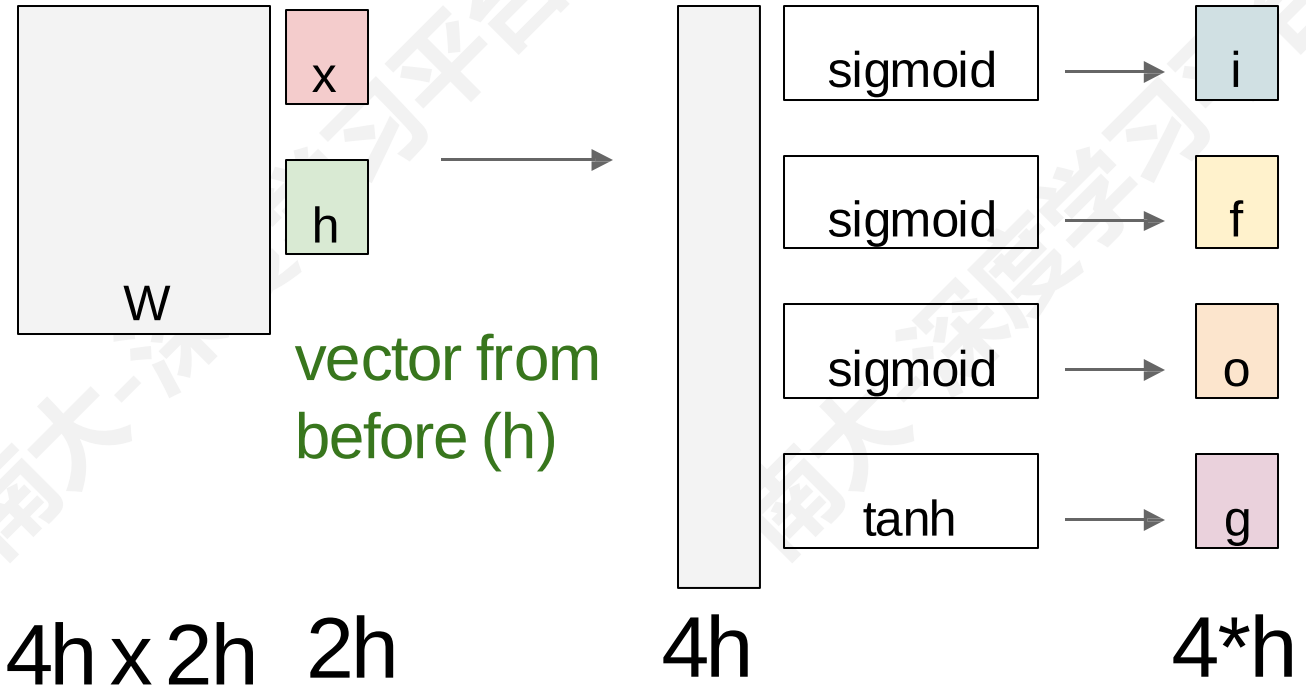
$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

循环神经网络

LSTM

i: Input gate, whether to write to cell
f: Forget gate, Whether to erase cell
o: Output gate, How much to reveal cell
g: Gate gate (?), How much to write to cell



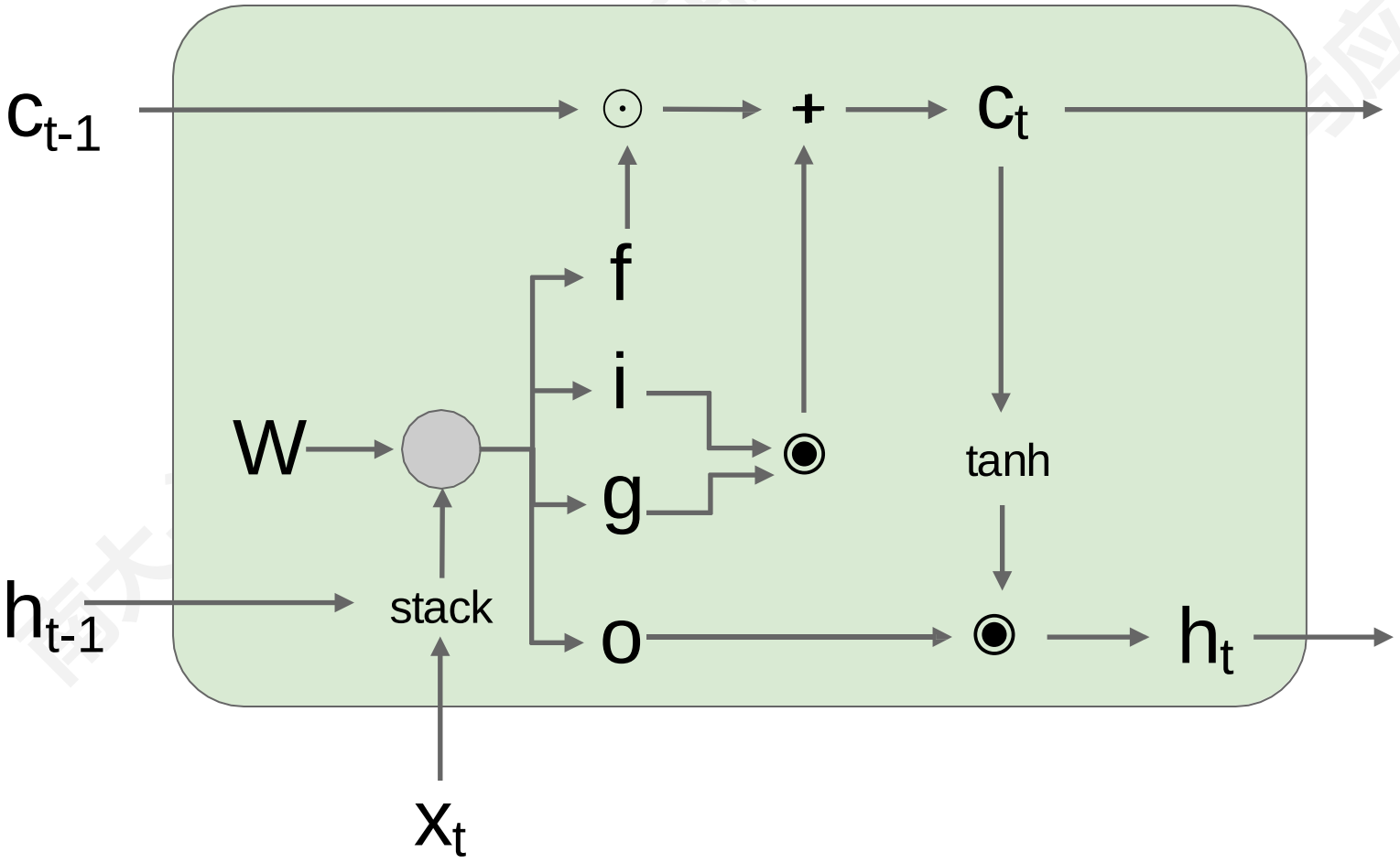
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

循环神经网络

Long Short Term Memory (LSTM)

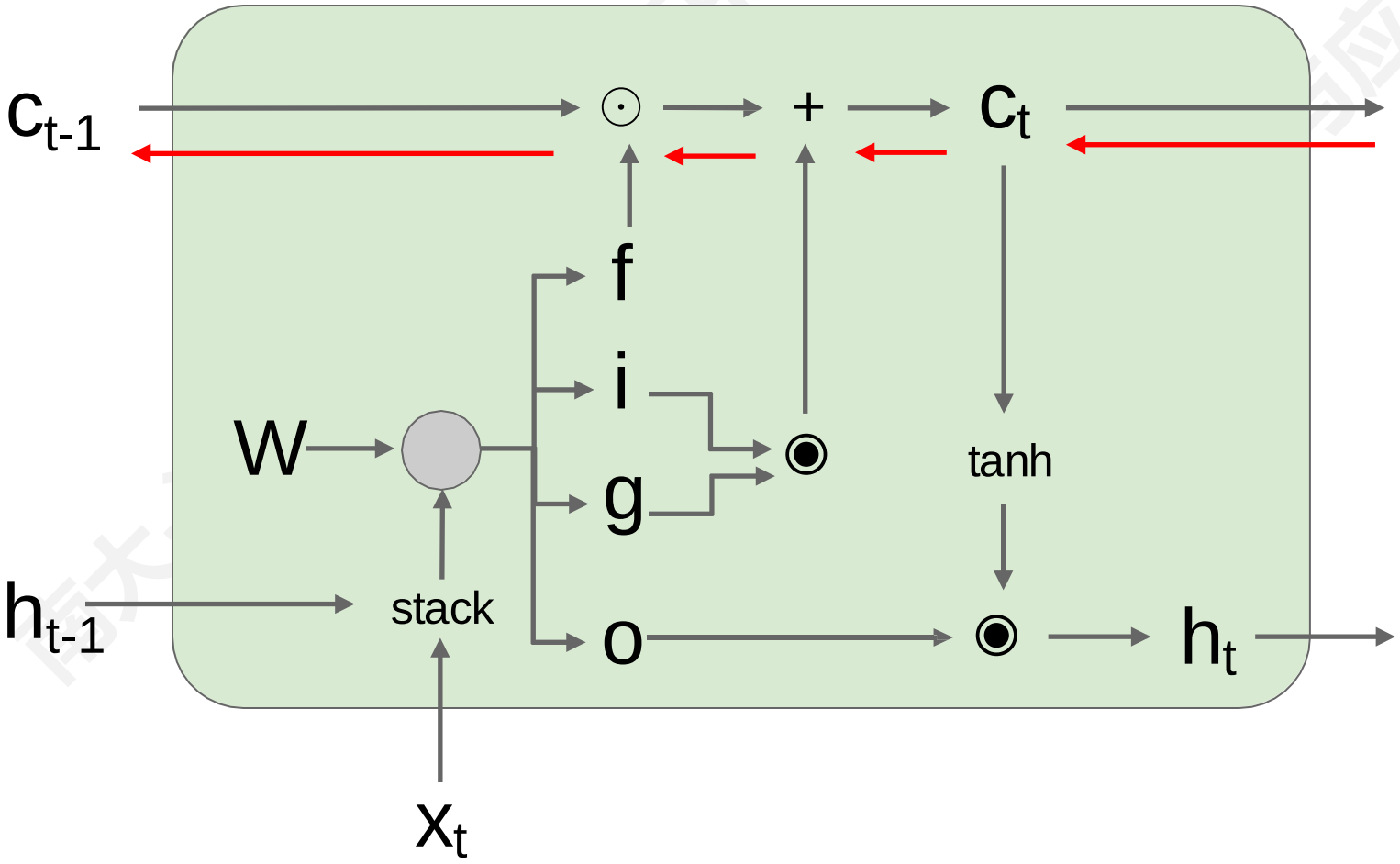


$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

Long Short Term Memory (LSTM) 梯度计算



从 c_t 到 c_{t-1} 的反向传播只与 f 有点乘的关系，与 W 无关

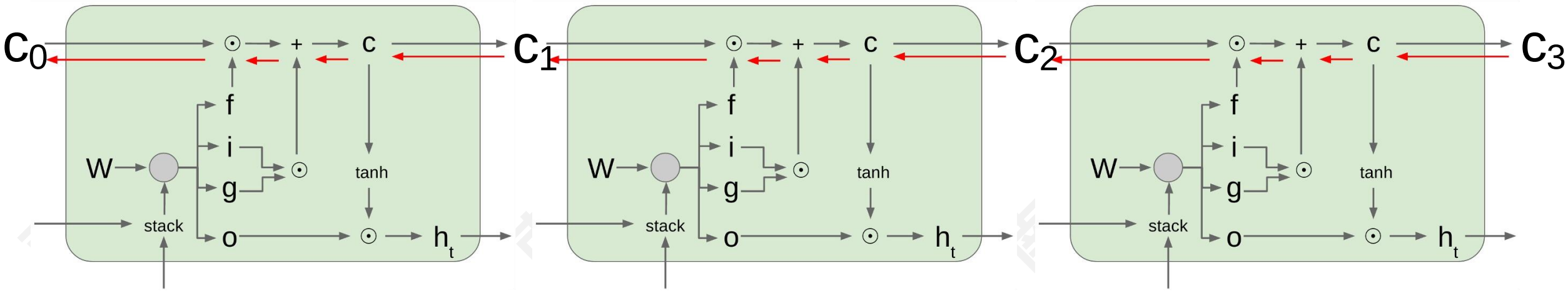
$$\begin{pmatrix} i \\ f \\ o \\ g \end{pmatrix} = \begin{pmatrix} \sigma \\ \sigma \\ \sigma \\ \tanh \end{pmatrix} W \begin{pmatrix} h_{t-1} \\ x_t \end{pmatrix}$$

$$c_t = f \odot c_{t-1} + i \odot g$$

$$h_t = o \odot \tanh(c_t)$$

- Long Short Term Memory (LSTM)

连续的梯度流!



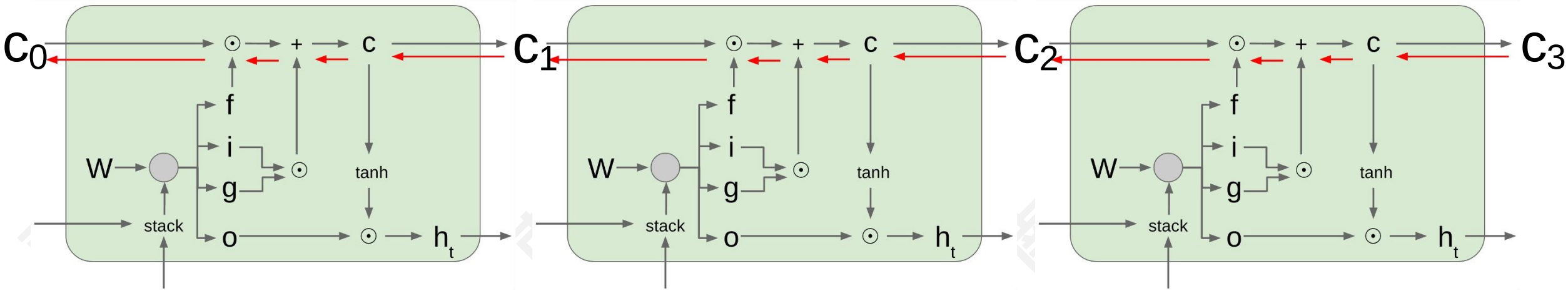
循环神经网络

- LSTM 解决梯度消失问题了吗?
  - LSTM 架构使 RNN 更容易在多个时间步长内保存信息
  - 例如，如果f=1且i=0，则该单元的信息将无限期保留
  - 相比之下，vanilla RNN更难学习循环权重矩阵 W_h 以保存信息
  - LSTM不能保证没有消失/爆炸梯度，但它确实为模型学习长距离依赖关系提供了一种更简单的方法

循环神经网络

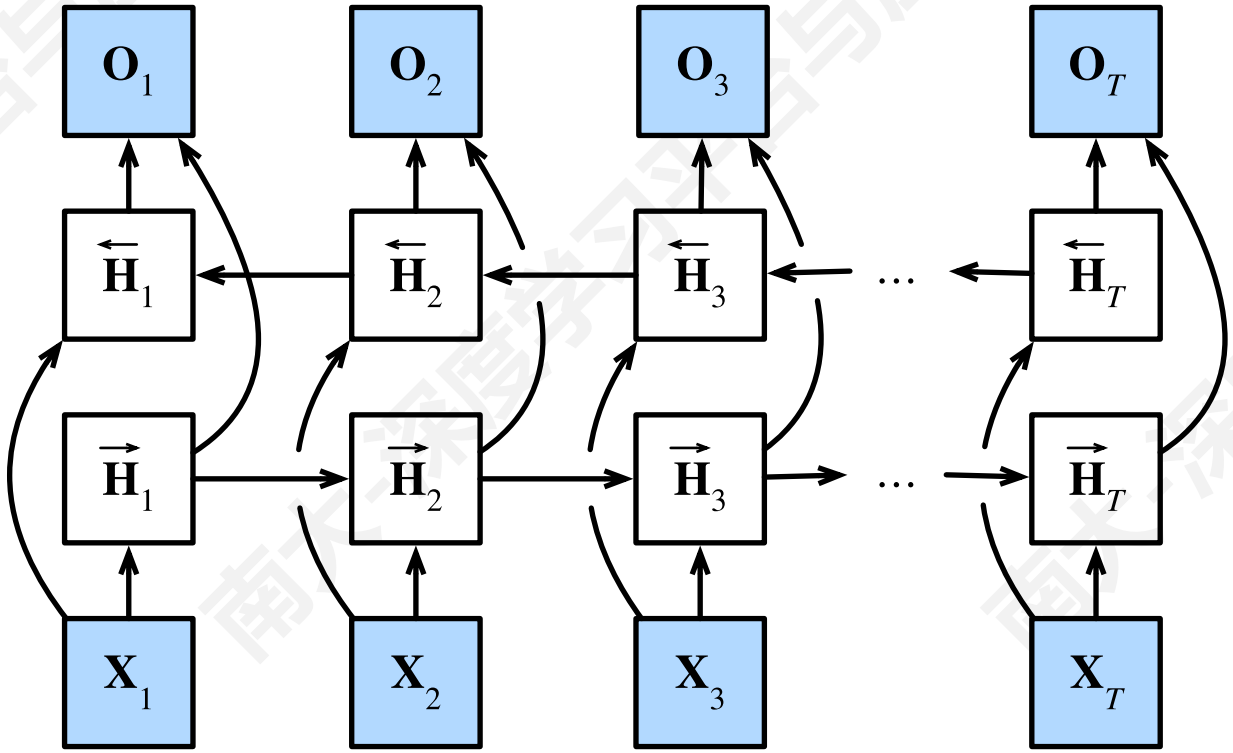
- Long Short Term Memory (LSTM)

连续的梯度流! 与 ResNet 很相似!



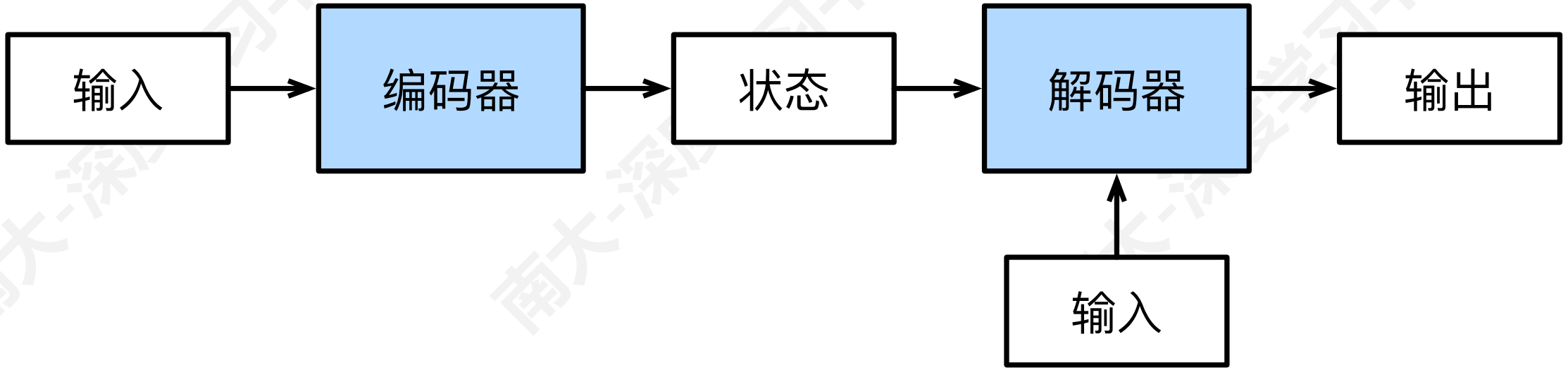
循环神经网络

双向循环网络



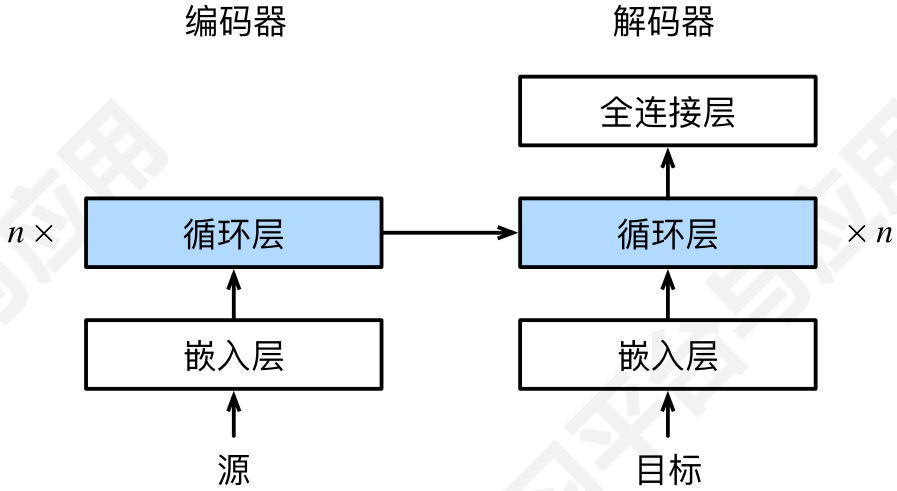
循环神经网络

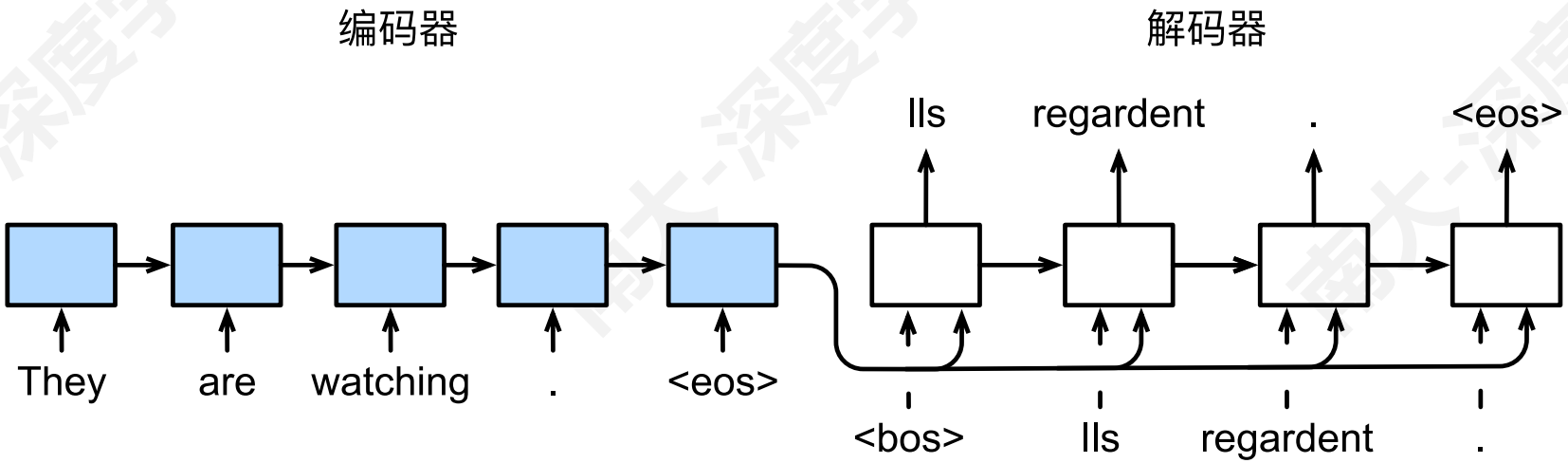
- 编码器-解码器架构
- 输入/输出长度可变，隐藏状态形状固定



循环神经网络

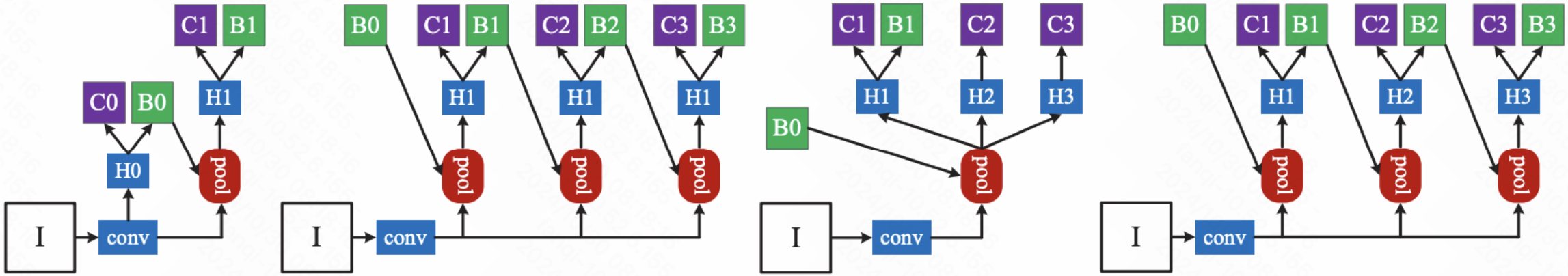
- seq2seq





南京大学 NANJING UNIVERSITY

Recurrent Idea



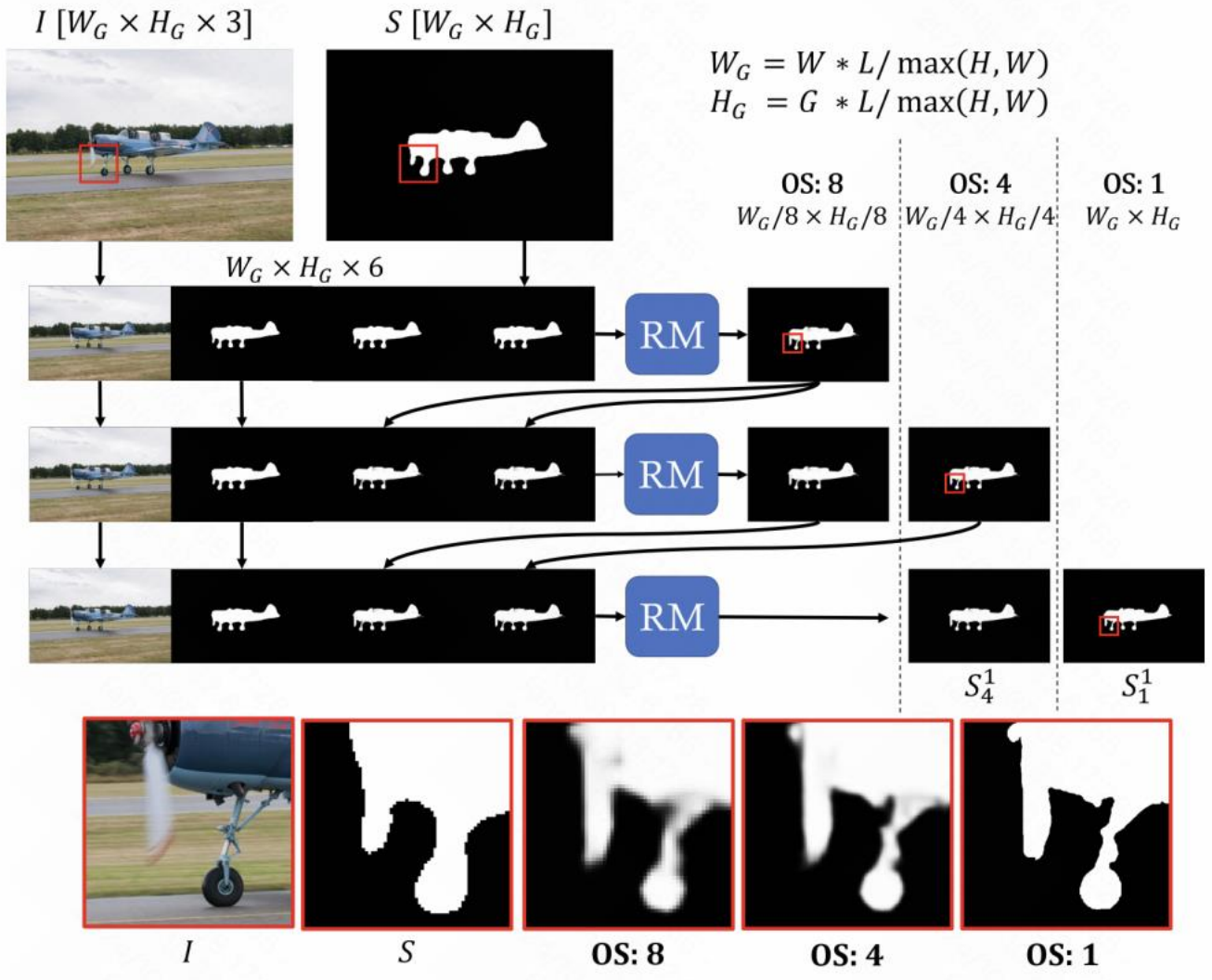
(a) Faster R-CNN (b) Iterative BBox at inference (c) Integral Loss (d) Cascade R-CNN

Figure 3. The architectures of different frameworks. "I" is input image, "conv" backbone convolutions, "pool" region-wise feature extraction, "H" network head, "B" bounding box, and "C" classification. "B0" is proposals in all architectures.

CascadePSP: Toward Class-Agnostic and Very Cascade R-CNN: Delving into High Quality Object Detection, CVPR 2018

循环神经网络

- **Recurrent Idea**



CascadePSP: Toward Class-Agnostic and Very High-Resolution Segmentation via Global and Local Refinement, CVPR 2020

南京大学 NANJING UNIVERSITY

Recurrent Idea

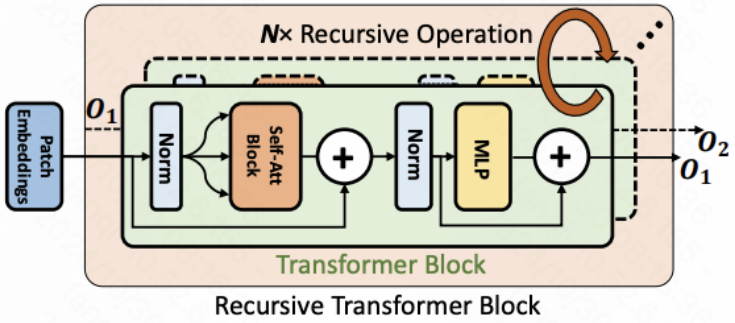


Fig. 2: Atomic Recursive Operation.

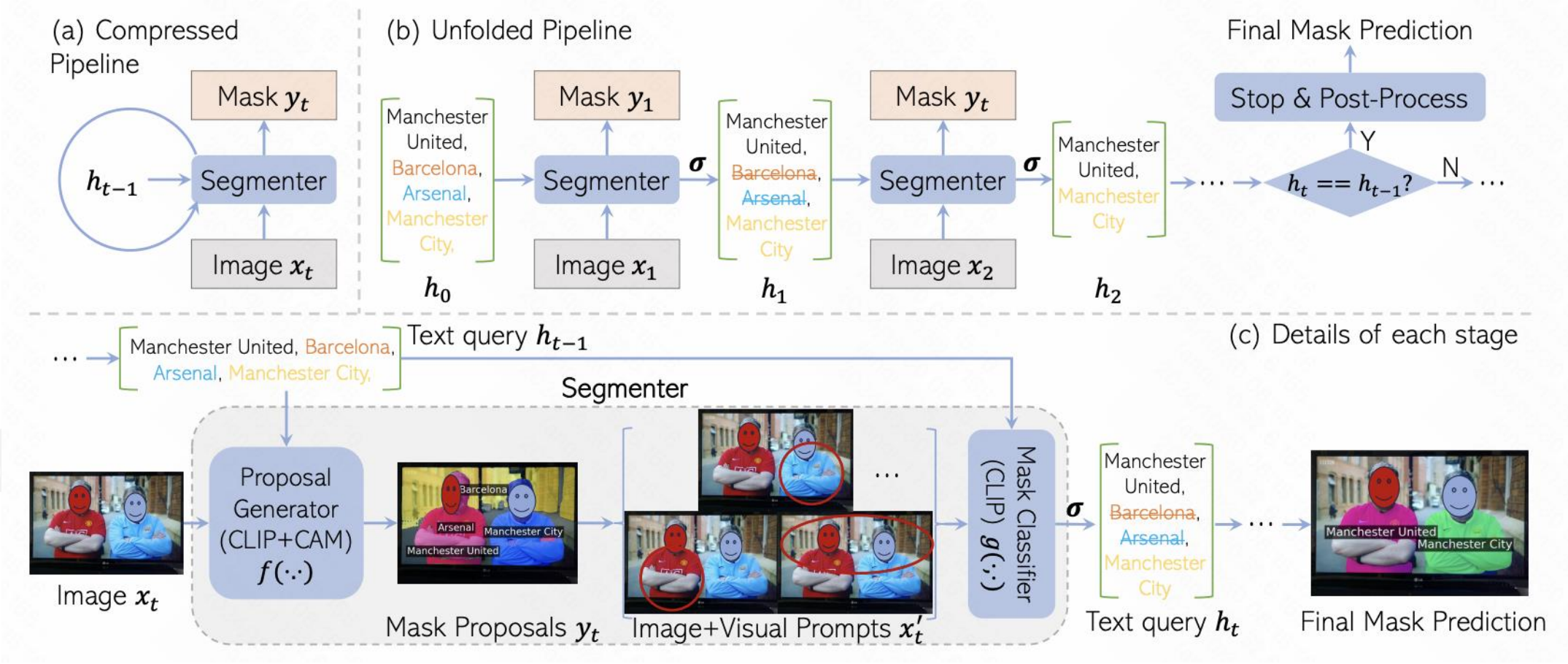
Table 1: Results using different numbers N of naïve recursive operation on ImageNet-1K dataset.

Method	Layers	#Params (M)	Top-1 Acc. (%)
DeiT-Tiny [52]	12	5.7	72.2
+ 1× naïve recursion	24	5.7	74.0
+ 2× naïve recursion	36	5.7	74.1
+ 3× naïve recursion	48	5.7	73.6

Sliced Recursive Transformer, ECCV 2022

循环神经网络

Recurrent Idea



CLIP as RNN: Segment Countless Visual Concepts without Training Endeavor, CVPR 2024

循环神经网络

南京大学 NANJING UNIVERSITY

- 总结：
  - RNN在架构设计中提供了很大的灵活性
  - Vanilla RNN很简单，但效果不太好， RNN 中梯度的反向流动可能会爆炸或消失
  - 常用 LSTM 或 GRU：它们改善了梯度流